

STAR RX72N - rx_frame Library
1.0.0

Generated by Doxygen 1.16.1

1 Directory Hierarchy	1
1.1 Directories	1
2 File Index	3
2.1 File List	3
3 Directory Documentation	5
3.1 libs/rx_frame/inc Directory Reference	5
3.2 libs Directory Reference	5
3.3 libs/rx_frame Directory Reference	6
3.4 libs/rx_frame/src Directory Reference	6
4 File Documentation	7
4.1 libs/rx_frame/inc/rx_frame.h File Reference	7
4.1.1 Detailed Description	9
4.1.1.1 Overview	9
4.1.1.2 Frame Format	10
4.1.1.3 Frame Size Calculations	10
4.1.1.4 Protocol Flow	10
4.1.1.5 CRC-32 Details	11
4.1.1.6 Thread Safety	11
4.1.1.7 Memory Usage	11
4.1.1.8 Performance Characteristics	11
4.1.1.9 NASA Power of 10 Compliance	12
4.1.1.10 SOLID Principles	12
4.1.1.11 Hardware Requirements	12
4.1.1.12 Integration Example	12
4.1.1.13 References	13
4.1.2 Data Structure Documentation	14
4.1.2.1 struct rx_frame_header_t	14
4.1.2.2 Memory Layout	14
4.1.2.3 Wire Format vs In-Memory Format	14
4.1.2.4 struct rx_frame_t	16
4.1.2.5 Memory Layout	16
4.1.2.6 Usage Patterns	16
4.1.2.7 struct rx_frame_encoder_t	18
4.1.2.8 Lifecycle	19
4.1.2.9 struct rx_frame_decoder_t	20
4.1.2.10 Lifecycle	20
4.1.3 Enumeration Type Documentation	21
4.1.3.1 rx_byte_order_constants_t	21
4.1.3.2 rx_frame_constants_t	21
4.1.3.3 Frame Layout	21

4.1.3.4 Size Calculations	22
4.1.3.5 rx_frame_flags_t	25
4.1.3.6 Flag Descriptions	25
4.1.3.7 Flag Combinations	25
4.1.3.8 rx_frame_type_t	27
4.1.3.9 Communication Pattern	27
4.1.3.10 Type Descriptions	28
4.1.3.11 rx_le16_byte_idx_t	31
4.1.3.12 rx_le32_byte_idx_t	32
4.1.3.13 rx_le32_shift_t	32
4.1.4 Function Documentation	33
4.1.4.1 rx_frame_create_ack()	33
4.1.4.2 rx_frame_create_nack()	34
4.1.4.3 rx_frame_create_ping()	35
4.1.4.4 rx_frame_create_pong()	36
4.1.4.5 rx_frame_create_reset()	38
4.1.4.6 rx_frame_create_reset_ack()	40
4.1.4.7 rx_frame_decode()	42
4.1.4.8 rx_frame_decode_with_resync()	44
4.1.4.9 rx_frame_decoder_deinit()	48
4.1.4.10 rx_frame_decoder_init()	49
4.1.4.11 rx_frame_encode()	50
4.1.4.12 rx_frame_encoded_size()	52
4.1.4.13 rx_frame_encoder_deinit()	52
4.1.4.14 rx_frame_encoder_init()	53
4.1.4.15 rx_frame_read_le16()	54
4.1.4.16 rx_frame_read_le32()	55
4.1.4.17 rx_frame_type_valid()	56
4.1.4.18 rx_frame_write_le16()	56
4.2 rx_frame.h	57
4.3 libs/rx_frame/src/rx_frame.c File Reference	72
4.3.1 Detailed Description	73
4.3.1.1 Overview	73
4.3.1.2 Module Architecture	73
4.3.1.3 Encoding Process	74
4.3.1.4 Decoding Process	74
4.3.1.5 Memory Usage	74
4.3.1.6 Performance (RX72N @ 240 MHz)	74
4.3.1.7 NASA Power of 10 Compliance	74
4.3.1.8 SOLID Principles	75
4.3.1.9 Test Coverage	75
4.3.2 Enumeration Type Documentation	76

4.3.2.1 byte_shift_t	76
4.3.2.2 frame_offset_t	76
4.3.2.3 frame_resync_discarded_t	77
4.3.2.4 init_state_t	78
4.3.3 Function Documentation	78
4.3.3.1 internal_decode_header()	78
4.3.3.2 internal_find_sync_offset()	80
4.3.3.3 internal_read_le32()	82
4.3.3.4 internal_verify_crc()	83
4.3.3.5 internal_write_le32()	84
4.3.3.6 rx_frame_create_ack()	85
4.3.3.7 rx_frame_create_nack()	86
4.3.3.8 rx_frame_create_ping()	87
4.3.3.9 rx_frame_create_pong()	88
4.3.3.10 rx_frame_create_reset()	90
4.3.3.11 rx_frame_create_reset_ack()	91
4.3.3.12 rx_frame_decode()	92
4.3.3.13 rx_frame_decode_with_resync()	94
4.3.3.14 rx_frame_decoder_deinit()	97
4.3.3.15 rx_frame_decoder_init()	97
4.3.3.16 rx_frame_encode()	98
4.3.3.17 rx_frame_encoder_deinit()	99
4.3.3.18 rx_frame_encoder_init()	100
4.4 rx_frame.c	101

Chapter 1

Directory Hierarchy

1.1 Directories

inc	5
rx_frame.h	7
libs	5
rx_frame	6
inc	5
rx_frame.h	7
src	6
rx_frame.c	72
rx_frame	6
inc	5
rx_frame.h	7
src	6
rx_frame.c	72
src	6
rx_frame.c	72

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

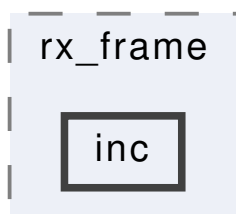
libs/rx_frame/inc/ rx_frame.h	
Frame Layer Protocol for SPI Communication	7
libs/rx_frame/src/ rx_frame.c	
Frame Layer Protocol Implementation	72

Chapter 3

Directory Documentation

3.1 libs/rx_frame/inc Directory Reference

Directory dependency graph for inc:

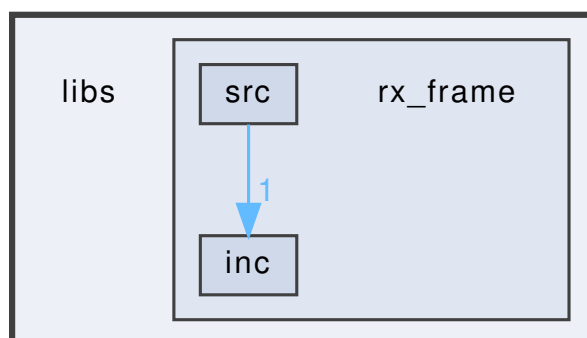


Files

- file [rx_frame.h](#)
Frame Layer Protocol for SPI Communication.

3.2 libs Directory Reference

Directory dependency graph for libs:

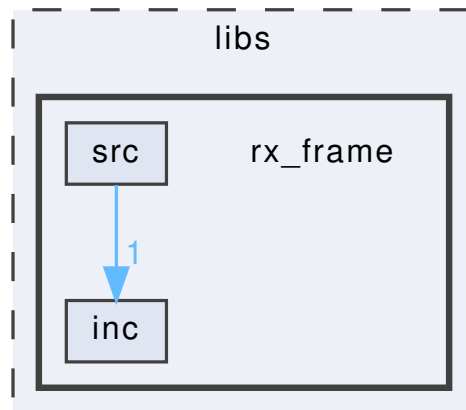


Directories

- directory [rx_frame](#)

3.3 libs/rx_frame Directory Reference

Directory dependency graph for rx_frame:

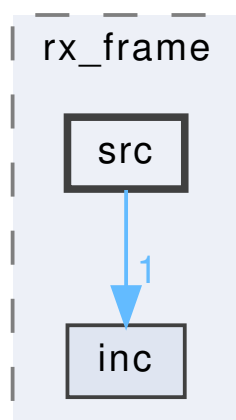


Directories

- directory [inc](#)
- directory [src](#)

3.4 libs/rx_frame/src Directory Reference

Directory dependency graph for src:



Files

- file [rx_frame.c](#)
Frame Layer Protocol Implementation.

Chapter 4

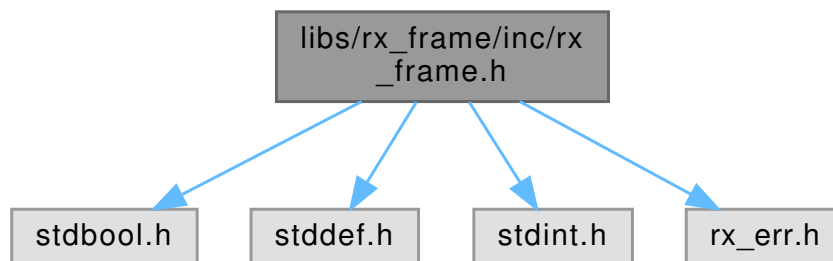
File Documentation

4.1 libs/rx_frame/inc/rx_frame.h File Reference

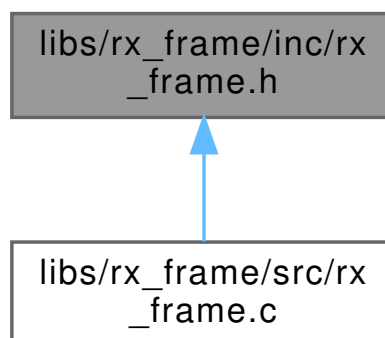
Frame Layer Protocol for SPI Communication.

```
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include "rx_err.h"
```

Include dependency graph for rx_frame.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [rx_frame_header_t](#)
Frame header containing control fields.
- struct [rx_frame_t](#)
Complete frame structure containing header, payload, and CRC.
- struct [rx_frame_encoder_t](#)
Frame encoder handle.
- struct [rx_frame_decoder_t](#)
Frame decoder handle.

Enumerations

- enum [rx_frame_constants_t](#): `uint16_t` {
`k_frame_sync_word = 0x55AA`, `k_frame_sync_size = 2`, `k_frame_seq_size = 2`, `k_frame_len_size = 2`,
`k_frame_type_size = 1`, `k_frame_flags_size = 1`, `k_frame_crc_size = 4`, `k_frame_header_size = 6`,
`k_frame_min_payload = 0`, `k_frame_max_payload = 1024`, `k_frame_min_size = 12`, `k_frame_max_size = 1036`,
`k_frame_max_scan_bytes` }
Frame structure constants for SPI protocol.
- enum [rx_frame_type_t](#): `uint8_t` {
`k_frame_type_ping = 0x00`, `k_frame_type_pong = 0x01`, `k_frame_type_command = 0x10`, `k_frame_type_response = 0x11`,
`k_frame_type_ack = 0x12`, `k_frame_type_nack = 0x13`, `k_frame_type_reset_ack = 0xFE`, `k_frame_type_reset = 0xFF` }
Frame types for SPI protocol (matches Go FrameType).
- enum [rx_frame_flags_t](#): `uint8_t` {
`k_frame_flag_none = 0x00`, `k_frame_flag_requires_ack = 0x01`, `k_frame_flag_retransmit = 0x02`,
`k_frame_flag_priority = 0x04`,
`k_frame_flag_fec_enabled = 0x08`, `k_frame_flag_soft_nack = 0x10` }
Frame control flags (matches Go FrameFlags).
- enum [rx_le16_byte_idx_t](#): `uint8_t` { `k_le16_byte_low = 0`, `k_le16_byte_high = 1` }
Byte indices for 16-bit little-endian serialization.
- enum [rx_byte_order_constants_t](#): `uint8_t` { `k_rx_le16_high_shift = (8)`, `k_rx_byte_mask = (0xFFU)` }
Byte manipulation constants for endianness conversions.
- enum [rx_le32_byte_idx_t](#): `uint8_t` { `k_le32_byte0 = 0`, `k_le32_byte1 = 1`, `k_le32_byte2 = 2`, `k_le32_byte3 = 3` }
Byte indices for 32-bit little-endian serialization.
- enum [rx_le32_shift_t](#): `uint8_t` { `k_rx_le32_shift0 = 0`, `k_rx_le32_shift1 = 8`, `k_rx_le32_shift2 = 16`,
`k_rx_le32_shift3 = 24` }
Bit shift amounts for extracting bytes from 32-bit values.

Functions

- `rx_err_t rx_frame_encoder_init(rx_frame_encoder_t *enc)`
Initialize frame encoder.
- `rx_err_t rx_frame_encoder_deinit(rx_frame_encoder_t *enc)`
Deinitialize frame encoder.
- `rx_err_t rx_frame_encode(const rx_frame_encoder_t *enc, const rx_frame_t *frame, uint8_t *output, uint32_t *output_len)`
Encode frame to wire format.
- static `uint32_t rx_frame_encoded_size(uint32_t payload_len)`

- Calculate encoded frame size.*
- static uint16_t `rx_frame_read_le16` (const uint8_t *buf)
 - Read uint16 from little-endian buffer.*
- static void `rx_frame_write_le16` (uint8_t *buf, uint16_t val)
 - Write uint16 to little-endian buffer.*
- static uint32_t `rx_frame_read_le32` (const uint8_t *buf)
 - Read uint32 from little-endian buffer.*
- rx_err_t `rx_frame_decoder_init` (rx_frame_decoder_t *dec)
 - Initialize frame decoder.*
- rx_err_t `rx_frame_decoder_deinit` (rx_frame_decoder_t *dec)
 - Deinitialize frame decoder.*
- rx_err_t `rx_frame_decode` (const rx_frame_decoder_t *dec, const uint8_t *data, const uint32_t data_len, rx_frame_t *frame)
 - Decode wire format to frame.*
- rx_err_t `rx_frame_decode_with_resync` (const rx_frame_decoder_t *dec, const uint8_t *data, const uint32_t data_len, rx_frame_t *frame, uint32_t *bytes_discarded_out)
 - Decode wire format to frame with bounded sync-word resynchronization.*
- rx_err_t `rx_frame_create_ack` (rx_frame_t *frame, uint16_t sequence)
 - Create ACK frame for given sequence.*
- rx_err_t `rx_frame_create_nack` (rx_frame_t *frame, uint16_t sequence, uint8_t flags)
 - Create NACK frame for given sequence.*
- rx_err_t `rx_frame_create_ping` (rx_frame_t *frame, const uint16_t sequence, const uint8_t *payload, const uint32_t payload_len)
 - Create PING frame for keepalive/heartbeat.*
- rx_err_t `rx_frame_create_pong` (rx_frame_t *frame, const uint16_t sequence, const uint8_t *payload, const uint32_t payload_len)
 - Create PONG frame (response to PING).*
- rx_err_t `rx_frame_create_reset` (rx_frame_t *frame, const uint16_t sequence)
 - Create RESET frame to reset communication state.*
- rx_err_t `rx_frame_create_reset_ack` (rx_frame_t *frame, const uint16_t sequence)
 - Create RESET_ACK frame (response to RESET).*
- static bool `rx_frame_type_valid` (uint8_t type)
 - Check if frame type is valid.*

4.1.1 Detailed Description

Frame Layer Protocol for SPI Communication.

4.1.1.1 Overview

This module implements the frame-level protocol for reliable SPI communication between the Raspberry Pi 5 gateway and the RX72N motor controller. It provides frame encoding, decoding, and CRC-32 integrity verification.

Key Features:

- Bit-exact compatible with star-gateway/internal/frame/ (Go implementation)
- IEEE 802.3 CRC-32 for data integrity
- Little-endian byte order for ALL multi-byte fields (SYNC, SEQ, LEN, CRC)
- Support for ACK/NACK flow control
- Optional forward error correction (FEC) flag

4.1.1.2 Frame Format

SYNC	SEQ	LEN	TYPE	FLAGS	PAYLOAD	CRC-32
2 bytes	2 bytes	2 bytes	1 byte	1 byte	0-1024 B	4 bytes
(LE)	(LE)	(LE)			(LE)	

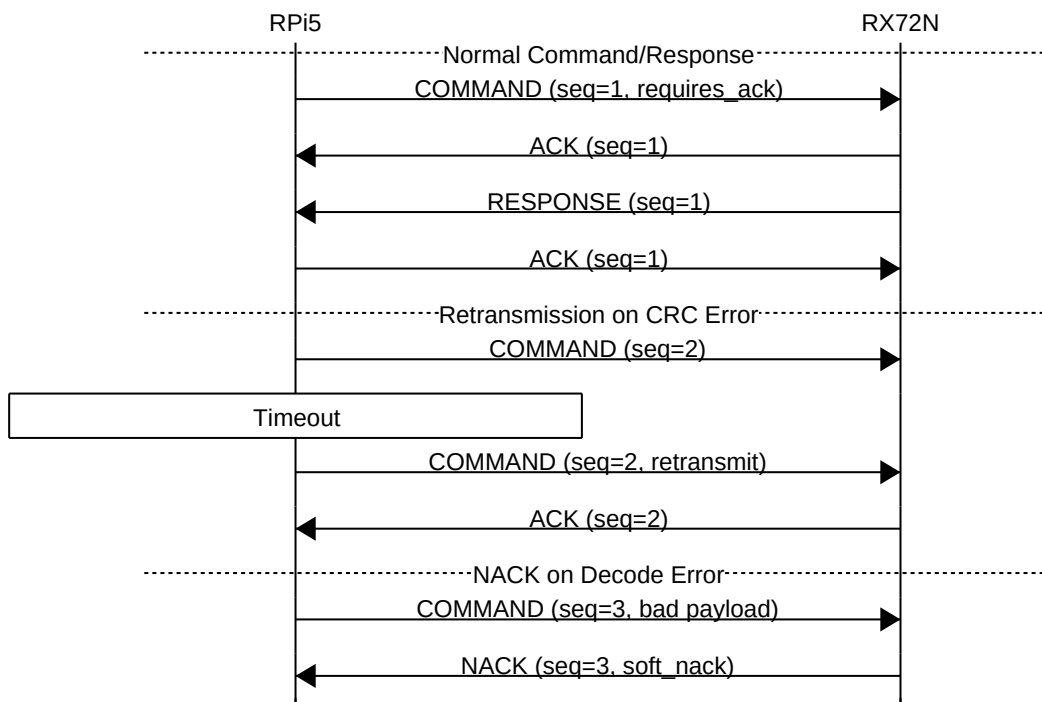
Field Descriptions:

Field	Size	Byte Order	Description
SYNC	2	Little-endian	Synchronization marker (0x55AA, wire: 0xAA 0x55)
SEQ	2	Little-endian	Sequence number for ordering/retransmit
LEN	2	Little-endian	Payload length in bytes (0-1024)
TYPE	1	N/A	Frame type (command, response, ack, nack)
FLAGS	1	N/A	Control flags (requires_ack, retransmit, etc.)
PAYLOAD	0-1024	N/A	Encoded protobuf message or empty
CRC-32	4	Little-endian	IEEE 802.3 CRC over SYNC+header+payload

4.1.1.3 Frame Size Calculations

- **Minimum frame:** 12 bytes (SYNC + header + CRC, no payload)
- **Maximum frame:** 1036 bytes (SYNC + header + 1024 payload + CRC)
- **Overhead:** 12 bytes fixed per frame

4.1.1.4 Protocol Flow



4.1.1.5 CRC-32 Details

The CRC-32 is computed using the IEEE 802.3 polynomial (0x04C11DB7) over the entire frame excluding the CRC field itself:

$$\text{CRC} = \text{CRC32_IEEE}(\text{SYNC} \parallel \text{Header} \parallel \text{Payload})$$

CRC Coverage:

- SYNC word (2 bytes)
- SEQ (2 bytes)
- LEN (2 bytes)
- TYPE (1 byte)
- FLAGS (1 byte)
- PAYLOAD (0-1024 bytes)

Why Little-Endian CRC? The CRC-32 is stored in little-endian format (LSB first) to match the IEEE 802.3 bit transmission order. This ensures compatibility with Go's `crc32.ChecksumIEEE()` function.

4.1.1.6 Thread Safety

Component	Thread Safe	Notes
Encoder	[PASS] Yes	Stateless after init
Decoder	[PASS] Yes	Stateless after init
Utility functions	[PASS] Yes	Pure functions

4.1.1.7 Memory Usage

Component	Size	Notes
rx_frame_t	1034 bytes	Header + 1024-byte payload + CRC
rx_frame_encoder_t	1 byte	Initialization flag only
rx_frame_decoder_t	1 byte	Initialization flag only
Encoded frame buffer	1036 bytes	Caller-provided

4.1.1.8 Performance Characteristics

Measured on RX72N @ 240 MHz, -O2 optimization:

Operation	Empty Payload	64B Payload	255B Payload
Encode	~5 us	~15 us	~50 us

Operation	Empty Payload	64B Payload	255B Payload
Decode	~5 us	~15 us	~50 us
CRC calc	~1 us	~6 us	~25 us

4.1.1.9 NASA Power of 10 Compliance

Rule	Status	Notes
1. Simple control flow	[PASS] Pass	No goto/setjmp/recursion
2. Fixed loop bounds	[PASS] Pass	Bounded by k_frame_max_payload
3. No dynamic memory	[PASS] Pass	All buffers caller-provided
4. Short functions	[PASS] Pass	All functions < 50 lines
5. Assertions	[PASS] Pass	NULL checks, state validation
6. Small scope	[PASS] Pass	Variables at minimal scope
7. Check returns	[PASS] Pass	All returns propagated
8. Limited preprocessor	[PASS] Pass	Only include guards
9. Restrict pointers	[PASS] Pass	No function pointers
10. Compiler warnings	[PASS] Pass	-Wall -Wextra -Werror

4.1.1.10 SOLID Principles

Principle	Implementation
Single Responsibility	Frame layer only - no protobuf knowledge
Open/Closed	New frame types via enum, no code changes
Liskov Substitution	Encoder/decoder handles are interchangeable
Interface Segregation	Separate encoder/decoder APIs
Dependency Inversion	Depends on rx_crc.h abstraction

4.1.1.11 Hardware Requirements

Resource	Requirement
SPI	10 Mbps, Full-duplex
DMA	Optional, improves throughput
CRC	Uses rx_crc module (HW or SW)

4.1.1.12 Integration Example

```
#include "rx_frame.h"
#include "rx_spi_comm.h"

// Transmit a command frame
void send_motor_command(uint8_t* protobuf_payload, uint32_t payload_len)
{
    rx_frame_encoder_t encoder;
    rx_frame_t frame;
```

```

uint8_t wire_buffer[1036];
uint32_t wire_len;

// Initialize encoder
rx_frame_encoder_init(&encoder);

// Build frame
frame.header.sequence = get_next_sequence();
frame.header.length = payload_len;
frame.header.type = k_frame_type_command;
frame.header.flags = k_frame_flag_requires_ack;
memcpy(frame.payload, protobuf_payload, payload_len);

// Encode to wire format
rx_err_t err = rx_frame_encode(&encoder, &frame, wire_buffer, &wire_len);
if (err != k_rx_ok) {
    rx_log_error("FRAME", "Encode failed: %d", err);
    return;
}

// Send via SPI
spi_transmit(wire_buffer, wire_len);
}

// Receive and decode a frame
rx_err_t receive_frame(uint8_t* wire_data, uint32_t wire_len, rx_frame_t* frame)
{
    rx_frame_decoder_t decoder;

    // Initialize decoder
    rx_frame_decoder_init(&decoder);

    // Decode frame (validates SYNC, parses header, verifies CRC)
    rx_err_t err = rx_frame_decode(&decoder, wire_data, wire_len, frame);
    if (err == k_rx_err_crc_mismatch) {
        rx_log_error("FRAME", "CRC mismatch, requesting retransmit");
        send_nack(frame->header.sequence);
        return err;
    }
    return err;
}

```

4.1.1.13 References

- star-gateway/internal/frame/: Go reference implementation
- docs/sections/01_nanopb_protocol.tex: Protocol specification
- IEEE 802.3: CRC-32 polynomial definition

See also

[rx_frame.c](#) Implementation file
[rx_crc.h](#) CRC-32 computation
[rx_spi_comm.h](#) SPI transport layer
[rx_nanopb.h](#) Protobuf encoding for payloads

Author

STAR Team

Date

2026-01-29

Copyright

Copyright (c) 2026 STAR Project. MIT License.

Since

Version 1.0.0

Definition in file [rx_frame.h](#).

4.1.2 Data Structure Documentation

4.1.2.1 struct rx_frame_header_t

Frame header containing control fields.

The frame header contains all control information needed to process a frame. All 16-bit fields are stored in host byte order within this structure; conversion to/from little-endian wire format is handled by the encoder/decoder.

4.1.2.2 Memory Layout

Offset	Size	Field	Type	Description
0	2	sequence	uint16_t	Sequence number
2	2	length	uint16_t	Payload length
4	1	type	uint8_t	Frame type
5	1	flags	uint8_t	Control flags
Total	6			No padding required

4.1.2.3 Wire Format vs In-Memory Format

Wire Format (little-endian):

- sequence: LSB at lower address
- length: LSB at lower address

In-Memory (this struct):

- sequence: Host byte order (little-endian on RX72N, matches wire)
- length: Host byte order

Usage Example:

```
rx_frame_header_t header = {
    .sequence = 0x1234,           // Will be sent as 0x34, 0x12 on wire (LE)
    .length = 64,                // 64-byte payload
    .type = k_frame_type_command,
    .flags = k_frame_flag_requires_ack,
};
```

Invariant

length <= k_frame_max_payload (1024)
 type is valid [rx_frame_type_t](#) value

See also

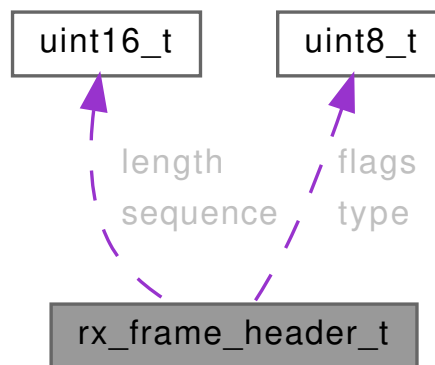
[rx_frame_t](#) Complete frame structure

Since

Version 1.0.0

Definition at line 639 of file [rx_frame.h](#).

Collaboration diagram for rx_frame_header_t:



Data Fields

uint8_t	flags	Control flags. Bitfield of rx_frame_flags_t values controlling frame handling. Multiple flags can be combined with bitwise OR. Valid Values: Combination of <code>rx_frame_flags_t</code> values
uint16_t	length	Payload length in bytes. Length of the payload field (not including header or CRC). Zero for ACK/NACK frames. Wire Format: Little-endian Valid Range: 0-1024 (<code>k_frame_max_payload</code>) Invariant $\text{length} \leq \text{k_frame_max_payload}$

uint16_t	sequence	Frame sequence number. 16-bit sequence number for ordering, duplicate detection, and ACK/NACK correlation. Wraps around at 65535. Wire Format: Little-endian Valid Range: 0-65535
uint8_t	type	Frame type identifier. Identifies the message type (command, response, ack, nack). Valid Values: rx_frame_type_t enumeration See also rx_frame_type_valid() Validate type

4.1.2.4 struct rx_frame_t

Complete frame structure containing header, payload, and CRC.

This structure represents a complete decoded frame ready for processing. The encoder serializes this to wire format; the decoder populates it from wire format.

4.1.2.5 Memory Layout

Offset	Size	Field	Description
0	6	header	Frame header (rx_frame_header_t)
6	1024	payload	Payload buffer (max size)
1030	4	crc	CRC-32 checksum (for decoded frames)
Total	1034		

Note: Structure size is 1034 bytes. The actual payload length is indicated by header.length, not the buffer size.

4.1.2.6 Usage Patterns

Encoding (TX):

1. Populate header fields (sequence, length, type, flags)
2. Copy payload data to payload[] array

3. Call [rx_frame_encode\(\)](#) - CRC is computed automatically

Decoding (RX):

1. Call [rx_frame_decode\(\)](#) with wire data
2. Access header fields to determine frame type
3. Read payload[0..header.length-1] for message content
4. CRC field contains the received checksum (already verified)

Usage Example - Building a Frame:

```
rx_frame_t frame;
memset(&frame, 0, sizeof(frame));

// Set header
frame.header.sequence = 1;
frame.header.length = protobuf_len;
frame.header.type = k_frame_type_command;
frame.header.flags = k_frame_flag_requires_ack;

// Copy payload
memcpy(frame.payload, protobuf_data, protobuf_len);

// Encode (CRC computed internally)
uint8_t wire[1036];
uint32_t wire_len;
rx_frame_encode(&encoder, &frame, wire, &wire_len);
```

Usage Example - Processing Decoded Frame:

```
rx_frame_t frame;
rx_err_t err = rx_frame_decode(&decoder, wire_data, wire_len, &frame);

if (err == k_rx_ok) {
    // Frame valid, CRC verified
    if (frame.header.type == k_frame_type_command) {
        // Process payload[0..header.length-1]
        handle_command(frame.payload, frame.header.length);
    }
}
```

Invariant

header.length <= k_frame_max_payload

Note

The crc field is populated during decode, not used during encode

Warning

Payload data beyond header.length is undefined

See also

[rx_frame_header_t](#) Header structure

[rx_frame_encode\(\)](#) Encode frame to wire format

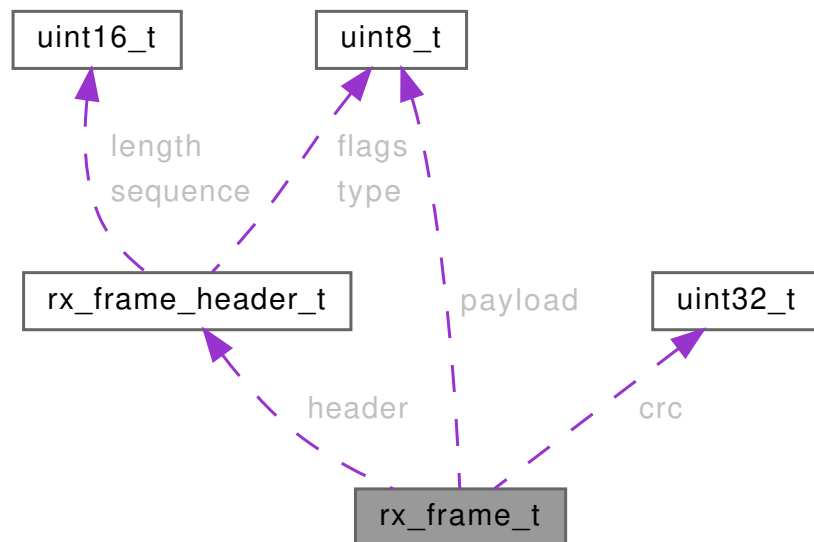
[rx_frame_decode\(\)](#) Decode frame from wire format

Since

Version 1.0.0

Definition at line 757 of file [rx_frame.h](#).

Collaboration diagram for `rx_frame_t`:



Data Fields

<code>uint32_t</code>	<code>crc</code>	CRC-32 checksum (IEEE 802.3). For received frames: Contains the CRC read from wire (verified). For transmitted frames: Not used (computed by encoder). Wire Format: Little-endian
rx_frame_header_t	<code>header</code>	Frame header containing control fields. Populated by caller (encode) or decoder (decode).
<code>uint8_t</code>	<code>payload[k_frame_max_payload]</code>	Payload data buffer (maximum 1024 bytes). For transmit: Contains data to send (length in <code>header.length</code>). For receive: Contains decoded payload (length in <code>header.length</code>). Note Only <code>payload[0..header.length-1]</code> is valid data.

4.1.2.7 struct `rx_frame_encoder_t`

Frame encoder handle.

Lightweight handle structure for frame encoding operations. Must be initialized before use with [rx_frame_encoder_init\(\)](#).

4.1.2.8 Lifecycle

1. Declare: `rx_frame_encoder_t enc;`
2. Initialize: `rx_frame_encoder_init(&enc);`
3. Use: `rx_frame_encode(&enc, &frame, output, &output_len);`
4. Deinitialize: `rx_frame_encoder_deinit(&enc);` (optional)

Memory Layout:

Offset	Size	Field	Description
0	1	initialized	Initialization flag
Total	1		No padding

Invariant

initialized is 0 or 1

Note

Structure is stateless after init; thread-safe for concurrent use

See also

[rx_frame_encoder_init\(\)](#) Initialize encoder

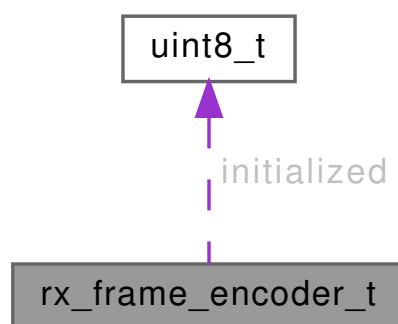
[rx_frame_encode\(\)](#) Encode frame

Since

Version 1.0.0

Definition at line 811 of file [rx_frame.h](#).

Collaboration diagram for rx_frame_encoder_t:



Data Fields

uint8_t	initialized	Initialization state flag. Non-zero (1) if initialized and ready for use. Valid Values: 0 (uninitialized), 1 (initialized)
---------	-------------	--

4.1.2.9 struct rx_frame_decoder_t

Frame decoder handle.

Lightweight handle structure for frame decoding operations. Must be initialized before use with [rx_frame_decoder_init\(\)](#).

4.1.2.10 Lifecycle

1. Declare: [rx_frame_decoder_t](#) dec;
2. Initialize: [rx_frame_decoder_init\(&dec\)](#);
3. Use: [rx_frame_decode\(&dec, data, len, &frame\)](#);
4. Deinitialize: [rx_frame_decoder_deinit\(&dec\)](#); (optional)

Memory Layout:

Offset	Size	Field	Description
0	1	initialized	Initialization flag
Total	1		No padding

Invariant

initialized is 0 or 1

Note

Structure is stateless after init; thread-safe for concurrent use

See also

[rx_frame_decoder_init\(\)](#) Initialize decoder

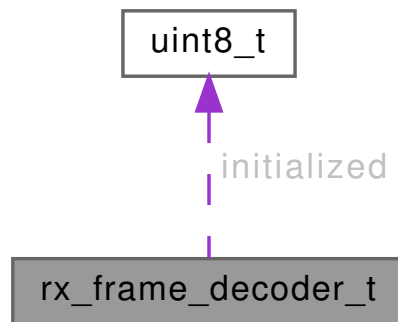
[rx_frame_decode\(\)](#) Decode frame

Since

Version 1.0.0

Definition at line 848 of file [rx_frame.h](#).

Collaboration diagram for rx_frame_decoder_t:

**Data Fields**

uint8_t	initialized	Initialization state flag. Non-zero (1) if initialized and ready for use. Valid Values: 0 (uninitialized), 1 (initialized)
---------	-------------	--

4.1.3 Enumeration Type Documentation**4.1.3.1 rx_byte_order_constants_t**

```
enum rx_byte_order_constants_t : uint8_t
```

Byte manipulation constants for endianness conversions.

Enumerator

k_rx_le16_high_shift	Bit shift for high byte in 16-bit value
k_rx_byte_mask	Mask to extract one byte

Definition at line 955 of file `rx_frame.h`.

```

00955     : uint8_t {
00956     k_rx_le16_high_shift = (8),    /**< Bit shift for high byte in 16-bit value */
00957     k_rx_byte_mask      = (0xFFU), /**< Mask to extract one byte */
00958 } rx_byte_order_constants_t;
  
```

4.1.3.2 rx_frame_constants_t

```
enum rx_frame_constants_t : uint16_t
```

Frame structure constants for SPI protocol.

These constants define the frame format parameters that must exactly match the Go implementation in `star-gateway/internal/frame/`. Any mismatch will cause protocol interoperability failures.

4.1.3.3 Frame Layout

Offset:	0	2	4	6	7	8	8+len
Field:	SYNC	SEQ	LEN	TYPE	FLAGS	PAYLOAD...	CRC
Size:	2	2	2	1	1	0-1024	4

4.1.3.4 Size Calculations

Frame Type	Formula	Bytes
Minimum (no payload)	SYNC + Header + CRC	12

Frame Type	Formula	Bytes
With N-byte payload	12 + N	12 - 1036
Maximum (1KB payload)	SYNC + Header + 1024 + CRC	1036

Usage Example:

```
// Validate frame size
if (wire_len < k_frame_min_size || wire_len > k_frame_max_size) {
    return k_rx_err_invalid_size;
}

// Check sync word
uint16_t sync = rx_frame_read_le16(wire_data);
if (sync != k_frame_sync_word) {
    return k_rx_err_protocol_error;
}

// Calculate expected size from payload length
uint16_t payload_len = rx_frame_read_le16(&wire_data[4]);
uint32_t expected = rx_frame_encoded_size(payload_len);
```

Warning

These values MUST match star-gateway/internal/frame/constants.go

Note

Using uint16_t underlying type to accommodate k_frame_max_payload (1024)

See also

star-gateway/internal/frame/ Go reference implementation

Since

Version 1.0.0

Enumerator

k_frame_sync_word	Frame synchronization marker (0x55AA). The sync word marks the start of a valid frame. Receivers scan for this pattern to establish frame boundaries. The value 0x55AA was chosen for: • Alternating bit pattern aids clock recovery • Unique enough to avoid false positives in random data • Matches common SPI debug patterns Wire Format: Little-endian (0xAA first, then 0x55)
k_frame_sync_size	SYNC field size in bytes (2). Two-byte sync marker at frame start.

k_frame_seq_size	SEQ field size in bytes (2). 16-bit sequence number for ordering and retransmit tracking.
k_frame_len_size	LEN field size in bytes (2). 16-bit payload length (max 1024).
k_frame_type_size	TYPE field size in bytes (1). Single byte for frame type (command, response, ack, nack).
k_frame_flags_size	FLAGS field size in bytes (1). Single byte for control flags (requires_ack, retransmit, etc.).
k_frame_crc_size	CRC-32 field size in bytes (4). IEEE 802.3 CRC-32 checksum.
k_frame_header_size	Header size excluding SYNC (6 bytes). SEQ(2) + LEN(2) + TYPE(1) + FLAGS(1) = 6 bytes.
k_frame_min_payload	Minimum payload length (0 bytes). ACK and NACK frames have empty payloads.
k_frame_max_payload	Maximum payload length (1024 bytes). Limited by SPI DMA buffer size and protobuf message limits.
k_frame_min_size	Minimum frame size (12 bytes). SYNC(2) + Header(6) + CRC(4) = 12 bytes (no payload).
k_frame_max_size	Maximum frame size (1036 bytes). SYNC(2) + Header(6) + Payload(1024) + CRC(4) = 1036 bytes.
k_frame_max_scan_bytes	Maximum bytes to scan forward during sync recovery (1036). When the sync word is not found at offset 0, the decoder scans forward up to this many bytes looking for the sync pattern. The upper bound equals k_frame_max_size so recovery never reads more data than one complete maximum-size frame. This ensures bounded execution time (NASA Rule 2) and prevents unbounded stream consumption on permanently corrupt inputs. The scan discards all bytes before the discovered sync word and reattempts decoding from that position.

Definition at line 290 of file rx_frame.h.

```

00290     : uint16_t {
00291 k_frame_sync_word = 0x55AA, /**< @brief Frame synchronization marker (0x55AA)
00292     * @details
00293     * The sync word marks the start of a valid frame. Receivers scan for this
00294     * pattern to establish frame boundaries. The value 0x55AA was chosen for:
00295     * - Alternating bit pattern aids clock recovery
00296     * - Unique enough to avoid false positives in random data
00297     * - Matches common SPI debug patterns
00298     * @par Wire Format: Little-endian (0xAA first, then 0x55)
00299     */
00300
00301 k_frame_sync_size = 2, /**< @brief SYNC field size in bytes (2)
00302     * @details Two-byte sync marker at frame start.
00303     */
00304
00305 k_frame_seq_size = 2, /**< @brief SEQ field size in bytes (2)
00306     * @details 16-bit sequence number for ordering and retransmit tracking.
00307     */
00308
00309 k_frame_len_size = 2, /**< @brief LEN field size in bytes (2)
00310     * @details 16-bit payload length (max 1024).
00311     */
00312
00313 k_frame_type_size = 1, /**< @brief TYPE field size in bytes (1)
00314     * @details Single byte for frame type (command, response, ack, nack).
00315     */
00316
00317 k_frame_flags_size = 1, /**< @brief FLAGS field size in bytes (1)
00318     * @details Single byte for control flags (requires_ack, retransmit, etc.).
00319     */
00320
00321 k_frame_crc_size = 4, /**< @brief CRC-32 field size in bytes (4)
00322     * @details IEEE 802.3 CRC-32 checksum.
00323     */
00324
00325 k_frame_header_size = 6, /**< @brief Header size excluding SYNC (6 bytes)
00326     * @details SEQ(2) + LEN(2) + TYPE(1) + FLAGS(1) = 6 bytes.
00327     */
00328

```

```

00329 k_frame_min_payload = 0, /**< @brief Minimum payload length (0 bytes)
00330      * @details ACK and NACK frames have empty payloads.
00331      */
00332
00333 k_frame_max_payload = 1024, /**< @brief Maximum payload length (1024 bytes)
00334      * @details Limited by SPI DMA buffer size and protobuf message limits.
00335      */
00336
00337 k_frame_min_size = 12, /**< @brief Minimum frame size (12 bytes)
00338      * @details SYNC(2) + Header(6) + CRC(4) = 12 bytes (no payload).
00339      */
00340
00341 k_frame_max_size = 1036, /**< @brief Maximum frame size (1036 bytes)
00342      * @details SYNC(2) + Header(6) + Payload(1024) + CRC(4) = 1036 bytes.
00343      */
00344
00345 k_frame_max_scan_bytes =
00346     k_frame_max_size, /**< @brief Maximum bytes to scan forward during sync recovery (1036)
00347      * @details
00348      * When the sync word is not found at offset 0, the decoder scans
00349      * forward up to this many bytes looking for the sync pattern. The
00350      * upper bound equals k_frame_max_size so recovery never reads more
00351      * data than one complete maximum-size frame. This ensures bounded
00352      * execution time (NASA Rule 2) and prevents unbounded stream
00353      * consumption on permanently corrupt inputs.
00354      *
00355      * The scan discards all bytes before the discovered sync word and
00356      * reattempts decoding from that position.
00357      */
00358 } rx_frame_constants_t;

```

4.1.3.5 rx_frame_flags_t

```
enum rx_frame_flags_t : uint8_t
```

Frame control flags (matches Go FrameFlags).

Bit flags that modify frame handling behavior. Multiple flags can be combined using bitwise OR.

4.1.3.6 Flag Descriptions

Flag	Bit	Purpose
REQUIRES_ACK	0	Sender expects ACK/NACK response
RETRANSMIT	1	This is a retransmission of previous frame
PRIORITY	2	Process before normal-priority frames
FEC_ENABLED	3	Payload uses forward error correction
SOFT_NACK	4	NACK with recoverable error info

4.1.3.7 Flag Combinations

```

// Typical command frame
frame.header.flags = k_frame_flag_requires_ack;

// Retransmission after timeout
frame.header.flags = k_frame_flag_requires_ack | k_frame_flag_retransmit;

// High-priority motor command
frame.header.flags = k_frame_flag_requires_ack | k_frame_flag_priority;

// FEC-protected data transfer
frame.header.flags = k_frame_flag_fec_enabled;

// NACK with error details
nack_frame.header.flags = k_frame_flag_soft_nack;

```

Usage Example:

```
// Check if ACK required
if (frame->header.flags & k_frame_flag_requires_ack) {
    send_ack(frame->header.sequence);
}

// Check if retransmit
if (frame->header.flags & k_frame_flag_retransmit) {
    rx_log_warn("FRAME", "Received retransmit seq=%u",
        frame->header.sequence);
    stats.retransmit_count++;
}
```

Warning

Values MUST match star-gateway/internal/frame/flags.go

Since

Version 1.0.0

Enumerator

k_frame_flag_none	No flags set (0x00). Default for frames that don't require special handling.
k_frame_flag_requires_ack	Frame requires acknowledgment (0x01, bit 0). Sender expects an ACK or NACK response. If no response within timeout, sender should retransmit with RETRANSMIT flag.
k_frame_flag_retransmit	Retransmission of previous frame (0x02, bit 1). Indicates this frame is a retry after timeout or NACK. Receiver should check for duplicate sequence numbers.
k_frame_flag_priority	High-priority frame (0x04, bit 2). Process before normal-priority frames in queue. Used for emergency stop and critical motor commands.
k_frame_flag_fec_enabled	Forward error correction enabled (0x08, bit 3). Payload is FEC-encoded (convolutional rate 1/2). FEC is enabled by default on the HARQ path (e.g., SPI transport with rx_spi_link). This flag indicates FEC encoding is present and receiver should apply Viterbi decoding before processing payload. The flag is ignored for transports that do not use HARQ (e.g., USB CDC, which relies on built-in hardware CRC and retry).
k_frame_flag_soft_nack	NACK with soft error information (0x10, bit 4). Used in NACK frames to indicate recoverable error. May include confidence bits or partial decode info.

Definition at line 550 of file rx_frame.h.

```
00550     : uint8_t {
00551     k_frame_flag_none = 0x00, /**< @brief No flags set (0x00)
00552         * @details Default for frames that don't require special handling.
00553         */
00554
00555     k_frame_flag_requires_ack = 0x01, /**< @brief Frame requires acknowledgment (0x01, bit 0)
00556         * @details
00557         * Sender expects an ACK or NACK response. If no response within
00558         * timeout, sender should retransmit with RETRANSMIT flag.
00559         */
00560
00561     k_frame_flag_retransmit = 0x02, /**< @brief Retransmission of previous frame (0x02, bit 1)
00562         * @details
00563         * Indicates this frame is a retry after timeout or NACK.
00564         * Receiver should check for duplicate sequence numbers.
00565         */
00566
00567     k_frame_flag_priority = 0x04, /**< @brief High-priority frame (0x04, bit 2)
00568         * @details
00569         * Process before normal-priority frames in queue.
00570         * Used for emergency stop and critical motor commands.
00571         */
```



```

00572
00573 k_frame_flag_fec_enabled = 0x08, /**< @brief Forward error correction enabled (0x08, bit 3)
00574 * @details
00575 * Payload is FEC-encoded (convolutional rate 1/2). FEC is enabled by default
00576 * on the HARQ path (e.g., SPI transport with rx_spi_link). This flag indicates
00577 * FEC encoding is present and receiver should apply Viterbi decoding before
00578 * processing payload. The flag is ignored for transports that do not use HARQ
00579 * (e.g., USB CDC, which relies on built-in hardware CRC and retry).
00580 */
00581
00582 k_frame_flag_soft_nack = 0x10, /**< @brief NACK with soft error information (0x10, bit 4)
00583 * @details
00584 * Used in NACK frames to indicate recoverable error.
00585 * May include confidence bits or partial decode info.
00586 */
00587 } rx_frame_flags_t;

```

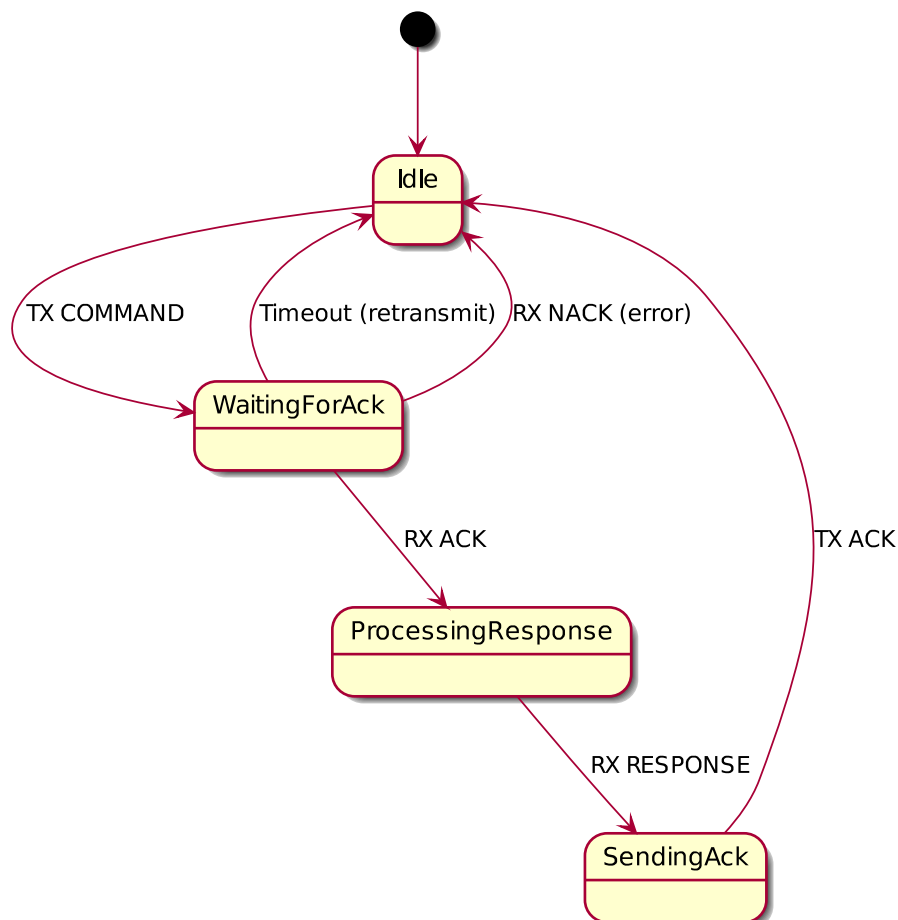
4.1.3.8 rx_frame_type_t

```
enum rx_frame_type_t : uint8_t
```

Frame types for SPI protocol (matches Go FrameType).

Defines the type of message contained in a frame. The frame type determines the expected payload content and required response handling.

4.1.3.9 Communication Pattern



4.1.3.10 Type Descriptions

Type	Direction	Payload	Response
COMMAND	RPi5 -> RX72N	Protobuf	ACK, then RESPONSE

Type	Direction	Payload	Response
RESPONSE	RX72N -> RPi5	Protobuf	ACK
ACK	Either	None	None
NACK	Either	None (flags indicate reason)	Retransmit

Usage Example:

```
switch (frame->header.type) {
    case k_frame_type_command:
        // Process command, send ACK then RESPONSE
        send_ack(frame->header.sequence);
        process_command(frame->payload, frame->header.length);
        send_response(frame->header.sequence);
        break;
    case k_frame_type_ack:
        // Mark pending command as acknowledged
        mark_acked(frame->header.sequence);
        break;
    case k_frame_type_nack:
        // Retransmit the failed frame
        retransmit(frame->header.sequence);
        break;
    default:
        rx_log_error("FRAME", "Unknown type: %u", frame->header.type);
}
```

Warning

Values MUST match star-gateway/internal/frame/types.go

See also

[rx_frame_type_valid\(\)](#) Validate frame type

Since

Version 1.0.0

Enumerator

k_frame_type_ping	Ping request (0x00). Keepalive/heartbeat message sent to check connection status. Receiver responds with PONG echoing the payload. Payload: 4-byte counter (little-endian uint32) or empty
k_frame_type_pong	Pong response (0x01). Response to PING request, confirms connection is alive. Echoes the PING counter payload for validation. Payload: Echo of PING counter or empty

k_frame_type_command	Command from controller to peripheral (0x10). Contains a protobuf-encoded command message from RPi5 to RX72N. On SPI: typically requires ACK and generates a RESPONSE frame. On USB CDC: no ACK expected (hardware reliability). Typical Payloads: MotorCommand, TelemetryRequest, ConfigWrite
k_frame_type_response	Response from peripheral to controller (0x11). Contains a protobuf-encoded response message from RX72N to RPi5. Sent after processing a COMMAND. Typical Payloads: MotorStatus, TelemetryData, ConfigAck
k_frame_type_ack	Positive acknowledgment (0x12, SPI only). Confirms successful reception and CRC validation of a frame. Empty payload; sequence number matches the acknowledged frame. Used only on SPI transport (USB CDC relies on hardware reliability). Payload: None (length = 0)
k_frame_type_nack	Negative acknowledgment (0x13, SPI only). Indicates a problem with received frame (CRC error, decode failure). Empty payload; flags indicate error type. Sender should retransmit. Used only on SPI transport (USB CDC relies on hardware reliability). Payload: None (length = 0) Flags: May include k_frame_flag_soft_nack
k_frame_type_reset_ack	Reset acknowledgment (0xFE). Confirms reset operation completed successfully. Sequence number matches the reset request. Payload: None (length = 0)
k_frame_type_reset	Reset request (0xFF). Request to reset communication state (sequence numbers, buffers, etc.). Receiver responds with RESET_ACK. Payload: None (length = 0)

Definition at line 417 of file [rx_frame.h](#).

```

00417         : uint8_t {
00418     /**
00419      * @brief Ping request (0x00)
00420      * @details
00421      * Keepalive/heartbeat message sent to check connection status.
00422      * Receiver responds with PONG echoing the payload.
00423      * @par Payload: 4-byte counter (little-endian uint32) or empty
00424      */
00425     k_frame_type_ping = 0x00,
00426
00427     /**
00428      * @brief Pong response (0x01)

```

```

00429     * @details
00430     * Response to PING request, confirms connection is alive.
00431     * Echoes the PING counter payload for validation.
00432     * @par Payload: Echo of PING counter or empty
00433     */
00434     k_frame_type_pong = 0x01,
00435
00436     /**
00437     * @brief Command from controller to peripheral (0x10)
00438     * @details
00439     * Contains a protobuf-encoded command message from RPi5 to RX72N.
00440     * On SPI: typically requires ACK and generates a RESPONSE frame.
00441     * On USB CDC: no ACK expected (hardware reliability).
00442     * @par Typical Payloads: MotorCommand, TelemetryRequest, ConfigWrite
00443     */
00444     k_frame_type_command = 0x10,
00445
00446     /**
00447     * @brief Response from peripheral to controller (0x11)
00448     * @details
00449     * Contains a protobuf-encoded response message from RX72N to RPi5.
00450     * Sent after processing a COMMAND.
00451     * @par Typical Payloads: MotorStatus, TelemetryData, ConfigAck
00452     */
00453     k_frame_type_response = 0x11,
00454
00455     /**
00456     * @brief Positive acknowledgment (0x12, SPI only)
00457     * @details
00458     * Confirms successful reception and CRC validation of a frame.
00459     * Empty payload; sequence number matches the acknowledged frame.
00460     * Used only on SPI transport (USB CDC relies on hardware reliability).
00461     * @par Payload: None (length = 0)
00462     */
00463     k_frame_type_ack = 0x12,
00464
00465     /**
00466     * @brief Negative acknowledgment (0x13, SPI only)
00467     * @details
00468     * Indicates a problem with received frame (CRC error, decode failure).
00469     * Empty payload; flags indicate error type. Sender should retransmit.
00470     * Used only on SPI transport (USB CDC relies on hardware reliability).
00471     * @par Payload: None (length = 0)
00472     * @par Flags: May include k_frame_flag_soft_nack
00473     */
00474     k_frame_type_nack = 0x13,
00475
00476     /**
00477     * @brief Reset acknowledgment (0xFE)
00478     * @details
00479     * Confirms reset operation completed successfully.
00480     * Sequence number matches the reset request.
00481     * @par Payload: None (length = 0)
00482     */
00483     k_frame_type_reset_ack = 0xFE,
00484
00485     /**
00486     * @brief Reset request (0xFF)
00487     * @details
00488     * Request to reset communication state (sequence numbers, buffers, etc.).
00489     * Receiver responds with RESET_ACK.
00490     * @par Payload: None (length = 0)
00491     */
00492     k_frame_type_reset = 0xFF,
00493 } rx_frame_type_t;

```

4.1.3.11 rx_le16_byte_idx_t

```
enum rx_le16_byte_idx_t : uint8_t
```

Byte indices for 16-bit little-endian serialization.

In little-endian format, the low byte (LSB) is stored at index 0 and the high byte (MSB) is stored at index 1.

Enumerator

k_le16_byte_low	Low byte (LSB) at index 0
-----------------	---------------------------

k_le16_byte_high	High byte (MSB) at index 1
------------------	----------------------------

Definition at line 947 of file [rx_frame.h](#).

```
00947     : uint8_t {
00948     k_le16_byte_low = 0, /**< Low byte (LSB) at index 0 */
00949     k_le16_byte_high = 1, /**< High byte (MSB) at index 1 */
00950 } rx_le16_byte_idx_t;
```

4.1.3.12 rx_le32_byte_idx_t

```
enum rx_le32_byte_idx_t : uint8_t
```

Byte indices for 32-bit little-endian serialization.

Enumerator

k_le32_byteζ	Byte 0 (LSB)
k_le32_byteη	Byte 1
k_le32_byteθ	Byte 2
k_le32_byteι	Byte 3 (MSB)

Definition at line 963 of file [rx_frame.h](#).

```
00963     : uint8_t {
00964     k_le32_byte = 0, /**< Byte 0 (LSB) */
00965     k_le32_byte = 1, /**< Byte 1 */
00966     k_le32_byte = 2, /**< Byte 2 */
00967     k_le32_byte = 3, /**< Byte 3 (MSB) */
00968 } rx_le32_byte_idx_t;
```

4.1.3.13 rx_le32_shift_t

```
enum rx_le32_shift_t : uint8_t
```

Bit shift amounts for extracting bytes from 32-bit values.

Enumerator

k_rx_le32_shiftζ	Shift 0 bits for byte 0
k_rx_le32_shiftη	Shift 8 bits for byte 1
k_rx_le32_shiftθ	Shift 16 bits for byte 2
k_rx_le32_shiftι	Shift 24 bits for byte 3

Definition at line 973 of file [rx_frame.h](#).

```
00973     : uint8_t {
00974     k_rx_le32_shift = 0, /**< Shift 0 bits for byte 0 */
00975     k_rx_le32_shift = 8, /**< Shift 8 bits for byte 1 */
00976     k_rx_le32_shift = 16, /**< Shift 16 bits for byte 2 */
00977     k_rx_le32_shift = 24, /**< Shift 24 bits for byte 3 */
00978 } rx_le32_shift_t;
```

4.1.4 Function Documentation

4.1.4.1 rx_frame_create_ack()

```
rx_err_t rx_frame_create_ack (
    rx_frame_t * frame,
    const uint16_t sequence) [nodiscard]
```

Create ACK frame for given sequence.

Parameters

out	<i>frame</i>	Frame to initialize
in	<i>sequence</i>	Sequence number to acknowledge

Returns

k_rx_ok on success

Create ACK frame for given sequence.

Constructs a frame with type=ACK, empty payload, and the specified sequence number. The ACK response frames are used by the receiver to confirm successful reception of a command or data frame from the sender.

Parameters

out	<i>frame</i>	Frame structure to populate with ACK
in	<i>sequence</i>	Sequence number of the frame being acknowledged

Returns

k_rx_ok on successful frame creation

Return values

<i>k_rx_err_invalid_arg</i>	if frame is nullptr
<i>k_rx_err_validation_failed</i>	if ACK type verification fails

Definition at line 892 of file [rx_frame.c](#).

```
00893 {
00894     if (frame == nullptr) {
00895         return k_rx_err_invalid_arg;
00896     }
00897     *frame = (rx_frame_t){0};
00898     frame->header.sequence = sequence;
00899     frame->header.length = 0;
00900     frame->header.type = k_frame_type_ack;
00901     frame->header.flags = k_frame_flag_none;
00902     if (frame->header.type != k_frame_type_ack) {
00903         return k_rx_err_validation_failed;
00904     }
00905     return k_rx_ok;
00906 }
```

References [rx_frame_header_t::flags](#), [rx_frame_t::header](#), [k_frame_flag_none](#), [k_frame_type_ack](#), [rx_frame_header_t::length](#), [rx_frame_header_t::sequence](#), and [rx_frame_header_t::type](#).

4.1.4.2 rx_frame_create_nack()

```
rx_err_t rx_frame_create_nack (
    rx_frame_t * frame,
    const uint16_t sequence,
    uint8_t flags)  [nodiscard]
```

Create NACK frame for given sequence.

Parameters

out	<i>frame</i>	Frame to initialize
in	<i>sequence</i>	Sequence number
in	<i>flags</i>	Additional flags (e.g., <code>k_frame_flag_soft_nack</code>)

Returns

`k_rx_ok` on success

Create NACK frame for given sequence.

Constructs a frame with type=NACK, empty payload, the specified sequence number, and error/status flags. NACK frames are sent by the receiver to indicate that a frame was not processed successfully due to an error condition (specified in flags).

Parameters

out	<i>frame</i>	Frame structure to populate with NACK
in	<i>sequence</i>	Sequence number of the frame being negative-acknowledged
in	<i>flags</i>	Status/error flags indicating reason for NACK

Returns

`k_rx_ok` on successful frame creation

Return values

<i>k_rx_err_invalid_arg</i>	if frame is nullptr
<i>k_rx_err_validation_failed</i>	if NACK type verification fails

Definition at line 926 of file [rx_frame.c](#).

```
00927 {
00928     if (frame == nullptr) {
00929         return k_rx_err_invalid_arg;
00930     }
00931
00932     *frame = (rx_frame_t){0};
00933     frame->header.sequence = sequence;
00934     frame->header.length = 0;
00935     frame->header.type = k_frame_type_nack;
00936     frame->header.flags = flags;
00937
00938     if (frame->header.type != k_frame_type_nack) {
00939         return k_rx_err_validation_failed;
00940     }
00941
00942     return k_rx_ok;
00943 }
```

References [rx_frame_header_t::flags](#), [rx_frame_t::header](#), [k_frame_type_nack](#), [rx_frame_header_t::length](#), [rx_frame_header_t::sequence](#), and [rx_frame_header_t::type](#).

4.1.4.3 rx_frame_create_ping()

```
rx_err_t rx_frame_create_ping (
    rx_frame_t * frame,
    const uint16_t sequence,
    const uint8_t * payload,
    const uint32_t payload_len) [nodiscard]
```

Create PING frame for keepalive/heartbeat.

Parameters

out	<i>frame</i>	Frame to initialize
in	<i>sequence</i>	Sequence number
in	<i>payload</i>	Optional payload to include (may be NULL if payload_len is 0)
in	<i>payload_len</i>	Payload length in bytes

Returns

k_rx_ok on success

Create PING frame for keepalive/heartbeat.

Initializes frame as a PING request for connection liveness checks. Receiver should respond with a PONG frame echoing the payload. Payload is optional; a 4-byte LE counter is typical.

Parameters

out	<i>frame</i>	Frame to populate (zeroed then filled)
in	<i>sequence</i>	Sequence number for this frame
in	<i>payload</i>	Optional payload (NULL when payload_len is 0)
in	<i>payload_len</i>	Payload length in bytes [0, k_frame_max_payload]

Return values

<i>k_rx_ok</i>	Frame created successfully
<i>k_rx_err_invalid_arg</i>	frame is NULL, or payload NULL with len > 0
<i>k_rx_err_invalid_size</i>	payload_len exceeds k_frame_max_payload
<i>k_rx_err_validation_failed</i>	Post-condition type check failed

Precondition

frame != nullptr

payload != NULL when payload_len > 0

Postcondition

```
frame->header.type == k_frame_type_ping
frame->header.length == payload_len
```

Note

Stateless; safe to call from any context.

See also

[rx_frame_create_pong\(\)](#) Corresponding response creator

Since

Version 1.0.0

Definition at line 972 of file [rx_frame.c](#).

```
00976 {
00977     if (frame == nullptr) {
00978         return k_rx_err_invalid_arg;
00979     }
00980
00981     if (payload_len > 0 && payload == nullptr) {
00982         return k_rx_err_invalid_arg;
00983     }
00984
00985     if (payload_len > k_frame_max_payload) {
00986         return k_rx_err_invalid_size;
00987     }
00988
00989     *frame = (rx_frame_t){0};
00990     frame->header.sequence = sequence;
00991     frame->header.length = (uint16_t)payload_len;
00992     frame->header.type = k_frame_type_ping;
00993     frame->header.flags = k_frame_flag_none;
00994
00995     if (payload_len > 0) {
00996         memcpy(frame->payload, payload, payload_len);
00997     }
00998
00999     if (frame->header.type != k_frame_type_ping) {
01000         return k_rx_err_validation_failed;
01001     }
01002
01003     return k_rx_ok;
01004 }
```

References [rx_frame_header_t::flags](#), [rx_frame_t::header](#), [k_frame_flag_none](#), [k_frame_max_payload](#), [k_frame_type_ping](#), [rx_frame_header_t::length](#), [rx_frame_t::payload](#), [rx_frame_header_t::sequence](#), and [rx_frame_header_t::type](#).

4.1.4.4 rx_frame_create_pong()

```
rx_err_t rx_frame_create_pong (
    rx_frame_t * frame,
    const uint16_t sequence,
    const uint8_t * payload,
    const uint32_t payload_len) [nodiscard]
```

Create PONG frame (response to PING).

Parameters

out	<i>frame</i>	Frame to initialize
-----	--------------	---------------------

in	<i>sequence</i>	Sequence number (matches ping)
in	<i>payload</i>	Optional payload to include (may be NULL if <i>payload_len</i> is 0)
in	<i>payload_len</i>	Payload length in bytes

Returns

`k_rx_ok` on success

Create PONG frame (response to PING).

Initializes frame as a PONG response to a received PING. Echoes the PING payload so the sender can validate round-trip data.

Parameters

out	<i>frame</i>	Frame to populate (zeroed then filled)
in	<i>sequence</i>	Sequence number for this frame
in	<i>payload</i>	Payload to echo (NULL when <i>payload_len</i> is 0)
in	<i>payload_len</i>	Payload length in bytes [0, <code>k_frame_max_payload</code>]

Return values

<i>k_rx_ok</i>	Frame created successfully
<i>k_rx_err_invalid_arg</i>	frame is NULL, or payload NULL with len > 0
<i>k_rx_err_invalid_size</i>	payload_len exceeds <code>k_frame_max_payload</code>
<i>k_rx_err_validation_failed</i>	Post-condition type check failed

Precondition

`frame != nullptr`

`payload != NULL` when `payload_len > 0`

Postcondition

`frame->header.type == k_frame_type_pong`

`frame->header.length == payload_len`

Note

Stateless; safe to call from any context.

See also

[rx_frame_create_ping\(\)](#) Corresponding request creator

Since

Version 1.0.0

Definition at line 1032 of file [rx_frame.c](#).

```

01036 {
01037     if (frame == nullptr) {
01038         return k_rx_err_invalid_arg;
01039     }
01040
01041     if (payload_len > 0 && payload == nullptr) {
01042         return k_rx_err_invalid_arg;
01043     }
01044
01045     if (payload_len > k_frame_max_payload) {
01046         return k_rx_err_invalid_size;
01047     }
01048
01049     *frame = (rx_frame_t){0};
01050     frame->header.sequence = sequence;
01051     frame->header.length = (uint16_t)payload_len;
01052     frame->header.type = k_frame_type_pong;
01053     frame->header.flags = k_frame_flag_none;
01054
01055     if (payload_len > 0) {
01056         memcpy(frame->payload, payload, payload_len);
01057     }
01058
01059     if (frame->header.type != k_frame_type_pong) {
01060         return k_rx_err_validation_failed;
01061     }
01062
01063     return k_rx_ok;
01064 }

```

References [rx_frame_header_t::flags](#), [rx_frame_t::header](#), [k_frame_flag_none](#), [k_frame_max_payload](#), [k_frame_type_pong](#), [rx_frame_header_t::length](#), [rx_frame_t::payload](#), [rx_frame_header_t::sequence](#), and [rx_frame_header_t::type](#).

4.1.4.5 rx_frame_create_reset()

```

rx_err_t rx_frame_create_reset (
    rx_frame_t * frame,
    const uint16_t sequence) [nodiscard]

```

Create RESET frame to reset communication state.

Parameters

out	<i>frame</i>	Frame to initialize
in	<i>sequence</i>	Sequence number

Returns

k_rx_ok on success

Create RESET frame to reset communication state.

Initializes frame as a RESET request that asks the peer to reset communication state (sequence numbers, buffers). The receiver should respond with a RESET_ACK frame. Payload is always empty.

Parameters

out	<i>frame</i>	Frame to populate (zeroed then filled)
-----	--------------	--

in	<i>sequence</i>	Sequence number for this frame
----	-----------------	--------------------------------

Return values

<i>k_rx_ok</i>	Frame created successfully
<i>k_rx_err_invalid_arg</i>	frame is nullptr
<i>k_rx_err_validation_failed</i>	Post-condition type check failed

Precondition

frame != nullptr

Caller has determined that a reset is necessary

Postcondition

frame->header.type == k_frame_type_reset

frame->header.length == 0

Note

Stateless; safe to call from any context.

Warning

Reset clears all sequence number state on both sides.

See also

[rx_frame_create_reset_ack\(\)](#) Corresponding acknowledgment creator

Since

Version 1.0.0

Definition at line 1091 of file [rx_frame.c](#).

```

01092 {
01093     if (frame == nullptr) {
01094         return k_rx_err_invalid_arg;
01095     }
01096     *frame = (rx_frame_t){0};
01097     frame->header.sequence = sequence;
01098     frame->header.length = 0;
01099     frame->header.type = k_frame_type_reset;
01100     frame->header.flags = k_frame_flag_none;
01101     if (frame->header.type != k_frame_type_reset) {
01102         return k_rx_err_validation_failed;
01103     }
01104     return k_rx_ok;
01105 }

```

References [rx_frame_header_t::flags](#), [rx_frame_t::header](#), [k_frame_flag_none](#), [k_frame_type_reset](#), [rx_frame_header_t::length](#), [rx_frame_header_t::sequence](#), and [rx_frame_header_t::type](#).

4.1.4.6 rx_frame_create_reset_ack()

```
rx_err_t rx_frame_create_reset_ack (  
    rx_frame_t * frame,  
    const uint16_t sequence)  [nodiscard]
```

Create RESET_ACK frame (response to RESET).

Parameters

out	<i>frame</i>	Frame to initialize
-----	--------------	---------------------

in	<i>sequence</i>	Sequence number (matches reset)
----	-----------------	---------------------------------

Returns

`k_rx_ok` on success

Create RESET_ACK frame (response to RESET).

Initializes frame as a RESET_ACK response to a received RESET request. Confirms that the receiver has completed its reset procedure. Payload is always empty; sequence matches the RESET request.

Parameters

out	<i>frame</i>	Frame to populate (zeroed then filled)
in	<i>sequence</i>	Sequence number (should match received RESET)

Return values

<i>k_rx_ok</i>	Frame created successfully
<i>k_rx_err_invalid_arg</i>	frame is nullptr
<i>k_rx_err_validation_failed</i>	Post-condition type check failed

Precondition

`frame != nullptr`

A RESET frame was received and processed

Postcondition

`frame->header.type == k_frame_type_reset_ack`

`frame->header.length == 0`

Note

Stateless; safe to call from any context.

See also

[rx_frame_create_reset\(\)](#) Corresponding request creator

Since

Version 1.0.0

Definition at line 1134 of file [rx_frame.c](#).

```

01135 {
01136     if (frame == nullptr) {
01137         return k_rx_err_invalid_arg;
01138     }
01139     *frame
01140         = (rx_frame_t){0};
01141     frame->header.sequence = sequence;
01142     frame->header.length   = 0;
01143     frame->header.type     = k_frame_type_reset_ack;
01144     frame->header.flags    = k_frame_flag_none;
01145     if (frame->header.type != k_frame_type_reset_ack) {
01146         return k_rx_err_validation_failed;
01147     }
01148     return k_rx_ok;
01150 }
01151 }
```

References [rx_frame_header_t::flags](#), [rx_frame_t::header](#), [k_frame_flag_none](#), [k_frame_type_reset_ack](#), [rx_frame_header_t::length](#), [rx_frame_header_t::sequence](#), and [rx_frame_header_t::type](#).

4.1.4.7 rx_frame_decode()

```

rx_err_t rx_frame_decode (
    const rx_frame_decoder_t * dec,
    const uint8_t * data,
    const uint32_t data_len,
    rx_frame_t * frame) [nodiscard]
```

Decode wire format to frame.

Parses wire format and validates SYNC word and CRC-32.

Parameters

in	<i>dec</i>	Decoder handle
in	<i>data</i>	Wire format data
in	<i>data_len</i>	Data length
out	<i>frame</i>	Decoded frame

Returns

- k_rx_ok on success
- k_rx_err_invalid_arg if any pointer is nullptr
- k_rx_err_invalid_state if decoder not initialized
- k_rx_err_invalid_size if data too short
- k_rx_err_protocol_error if SYNC word invalid
- k_rx_err_crc_mismatch if CRC validation fails

Decode wire format to frame.

Validates the decoder state, extracts the frame header, copies the payload, and verifies the CRC-32 checksum.

Parameters

in	<i>dec</i>	Initialized frame decoder instance
----	------------	------------------------------------

in	<i>data</i>	Raw frame data buffer
in	<i>data_len</i>	Length of data buffer in bytes
out	<i>frame</i>	Decoded frame (header, payload, CRC)

Return values

<i>k_rx_ok</i>	Success - frame decoded and CRC verified
<i>k_rx_err_invalid_arg</i>	Any pointer parameter is nullptr or other invalid arguments
<i>k_rx_err_invalid_state</i>	Decoder not initialized
<i>k_rx_err_crc</i>	CRC-32 verification failed
<i>k_rx_err_invalid_size</i>	Payload exceeds maximum frame size

Note

This function performs validation at multiple points:

- Null pointer checks on all parameters
- Decoder initialization check
- Header parsing via [internal_decode_header\(\)](#)
- CRC verification via [internal_verify_crc\(\)](#)

Definition at line 726 of file [rx_frame.c](#).

```

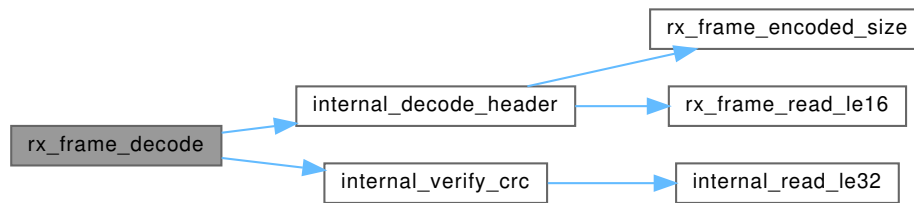
00730 {
00731     if (dec == nullptr || data == nullptr || frame == nullptr) {
00732         return k_rx_err_invalid_arg;
00733     }
00734
00735     if (dec->initialized != k_state_initialized) {
00736         return k_rx_err_invalid_state;
00737     }
00738
00739     uint32_t offset;
00740     rx_err_t err = internal_decode_header(data, data_len, frame, &offset);
00741     if (err != k_rx_ok) {
00742         return err;
00743     }
00744
00745     /* Read PAYLOAD */
00746     if (frame->header.length > 0) {
00747         memcpy(frame->payload, &data[offset], frame->header.length);
00748         offset += frame->header.length;
00749     }
00750
00751     err = internal_verify_crc(data, data_len, offset, &frame->crc);
00752     if (err != k_rx_ok) {
00753         return err;
00754     }
00755     return k_rx_ok;
00756 }

```

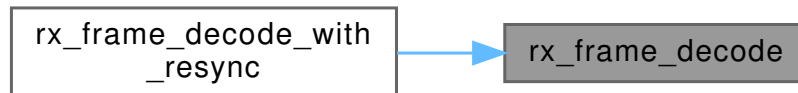
References [rx_frame_t::crc](#), [rx_frame_t::header](#), [rx_frame_decoder_t::initialized](#), [internal_decode_header\(\)](#), [internal_verify_crc\(\)](#), [k_state_initialized](#), [rx_frame_header_t::length](#), and [rx_frame_t::payload](#).

Referenced by [rx_frame_decode_with_resync\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.4.8 rx_frame_decode_with_resync()

```

rx_err_t rx_frame_decode_with_resync (
    const rx_frame_decoder_t * dec,
    const uint8_t * data,
    const uint32_t data_len,
    rx_frame_t * frame,
    uint32_t * bytes_discarded_out) [nodiscard]
  
```

Decode wire format to frame with bounded sync-word resynchronization.

Extends [rx_frame_decode\(\)](#) with automatic stream recovery. When the sync word is not found at offset 0 (i.e., the stream is byte-misaligned due to a dropped or inserted byte), this function scans forward up to `k_frame_max_scan_bytes` positions looking for the next sync word occurrence. Bytes discarded during the scan are reported via `bytes_discarded_out` for diagnostic logging by the caller.

If the sync word is located within the scan window, decoding proceeds from that offset as if data had been provided aligned. If no sync word is found within the bounded window, `k_rx_err_protocol_error` is returned.

Algorithm:

1. Attempt normal decode at offset 0.
2. On `k_rx_err_protocol_error` (sync mismatch), invoke [internal_find_sync_offset\(\)](#) to locate the next sync word.
3. If found, set `*bytes_discarded_out = sync_offset` – caller must advance the stream read pointer by this value even if the subsequent decode fails (e.g., `k_rx_err_crc_mismatch`), to avoid infinite re-scan.
4. Decode from the discovered offset (trimmed view of data).
5. All other errors (CRC, size) are propagated unchanged.

Parameters

in	<i>dec</i>	Initialized frame decoder handle
in	<i>data</i>	Raw byte buffer (may be misaligned)
in	<i>data_len</i>	Length of data buffer in bytes

out	<i>frame</i>	Decoded frame (header, payload, CRC)
out	<i>bytes_discarded_out</i>	Number of bytes skipped to reach sync word (0 when sync was found at offset 0)

Returns

`rx_err_t` Error code

Return values

<i>k_rx_ok</i>	Frame decoded and CRC verified
<i>k_rx_err_invalid_arg</i>	Any pointer parameter is nullptr
<i>k_rx_err_invalid_state</i>	Decoder not initialized
<i>k_rx_err_invalid_size</i>	Data buffer too short to contain a valid frame
<i>k_rx_err_protocol_error</i>	No sync word found within <code>k_frame_max_scan_bytes</code>
<i>k_rx_err_crc_mismatch</i>	Sync found but CRC validation failed

Precondition

dec must be initialized via [rx_frame_decoder_init\(\)](#)

data must point to a buffer of at least `data_len` bytes

frame must be a non-null pointer to a writable [rx_frame_t](#)

`bytes_discarded_out` must be a non-null pointer to a writable `uint32_t`

Postcondition

On `k_rx_ok`, frame contains the decoded frame with verified CRC

`bytes_discarded_out` contains the number of bytes skipped (≥ 0)

On `k_rx_err_crc_mismatch`, `*bytes_discarded_out == bytes skipped`; caller must still advance stream pointer

Note

Thread-safe after init; the decoder handle is read-only and the function uses no shared mutable state (consistent with the file-level "Stateless after init").

The scan window is bounded at `k_frame_max_scan_bytes` (NASA Rule 2).

Caller should log `bytes_discarded_out > 0` as a diagnostic warning.

Warning

Caller must advance its read pointer by `*bytes_discarded_out` bytes after any call that returns `k_rx_ok` or `k_rx_err_crc_mismatch` to avoid infinite re-scan; `*bytes_discarded_out` is unmodified only when `k_rx_err_invalid_arg` is returned.

Performance:

Worst case: scans `k_frame_max_scan_bytes` (1036) before failing. Best case: identical to [rx_frame_decode\(\)](#) (0 bytes discarded).

Example:

```
rx_frame_decoder_t dec;
rx_frame_decoder_init(&dec);

rx_frame_t frame;
uint32_t discarded = 0;
rx_err_t err = rx_frame_decode_with_resync(&dec, wire_buf, wire_len,
                                           &frame, &discarded);
if (discarded > 0) {
    rx_log_warn("FRAME", "Stream resync: discarded %u bytes", discarded);
}
if (err == k_rx_ok) {
    process_frame(&frame);
}
```

See also

[rx_frame_decode\(\)](#) Simple decode without resynchronization

[k_frame_max_scan_bytes](#) Maximum scan window (1036 bytes)

Since

Version 1.0.0

Decode wire format to frame with bounded sync-word resynchronization.

Provides stream recovery when the byte stream has become misaligned due to a dropped or inserted byte. On sync mismatch, scans forward up to `k_frame_max_scan_bytes` positions for the next occurrence of the sync word, discards the leading bytes, and reattempts decoding from that position.

This function is safe for repeated calls on a stream: the caller must advance its stream read pointer by `*bytes_discarded_out` on `k_rx_ok` and also on `k_rx_err_crc_mismatch` (any case where `bytes_discarded_out` is set) to avoid re-processing discarded bytes in subsequent calls.

Recovery algorithm:

1. Attempt [rx_frame_decode\(\)](#) on `data[0..data_len-1]`.
2. If result is not `k_rx_err_protocol_error`, return immediately.
3. Call [internal_find_sync_offset\(\)](#) to locate next sync word in `data[1..]`.
4. If not found, return `k_rx_err_protocol_error` with `*bytes_discarded_out = 0`.
5. Decode from `data[sync_offset..data_len-1]` using the trimmed sub-buffer.
6. Write `sync_offset` to `*bytes_discarded_out` and return result.

Parameters

in	<i>dec</i>	Initialized frame decoder handle
in	<i>data</i>	Raw byte buffer (may be stream-misaligned)
in	<i>data_len</i>	Length of data buffer in bytes
out	<i>frame</i>	Decoded frame (header, payload, CRC)
out	<i>bytes_discarded_out</i>	Bytes skipped before sync word (0 = aligned)

Returns

`rx_err_t` `k_rx_ok` on success, or `k_rx_err_invalid_arg`, `k_rx_err_invalid_state`, `k_rx_err_invalid_size`, `k_rx_err_protocol_error`, or `k_rx_err_crc_mismatch` on failure

Return values

<i>k_rx_ok</i>	Frame decoded and CRC verified
----------------	--------------------------------

<i>k_rx_err_invalid_arg</i>	Any pointer parameter is nullptr
<i>k_rx_err_invalid_state</i>	Decoder not initialized
<i>k_rx_err_invalid_size</i>	Buffer too short to contain a valid frame
<i>k_rx_err_protocol_error</i>	No sync word found within scan window
<i>k_rx_err_crc_mismatch</i>	Sync found but CRC verification failed

Precondition

dec must be initialized via [rx_frame_decoder_init\(\)](#)
data must point to a buffer of at least data_len bytes
frame must point to a valid [rx_frame_t](#) storage location (not NULL)
bytes_discarded_out must point to a valid uint32_t storage location (not NULL)

Postcondition

On k_rx_ok, *bytes_discarded_out == bytes skipped to reach sync word
On k_rx_ok, frame contains a fully verified decoded frame
On k_rx_err_crc_mismatch, *bytes_discarded_out == bytes skipped; caller must still advance stream pointer

Note

Parameter order follows the canonical convention: decoder input -> data input -> frame output -> diagnostic output (matching [rx_frame_decode\(\)](#)).
Not thread-safe. Caller must synchronize access.
The scan is bounded at k_frame_max_scan_bytes (NASA Rule 2).
Log *bytes_discarded_out > 0 as a stream-health diagnostic warning.

Warning

Caller must advance stream read pointer by *bytes_discarded_out after any call that returns k_rx_ok or k_rx_err_crc_mismatch to avoid infinite re-scan; *bytes_discarded_out is unmodified only when k_rx_err_invalid_arg is returned.

Example:

```
uint32_t discarded = k_bytes_discarded_none;
rx_err_t err = rx_frame_decode_with_resync(dec, data, data_len, &frame, &discarded);
if (err == k_rx_ok) {
    // Advance stream read pointer past any discarded bytes
    stream_read_ptr += discarded;
} else if (err == k_rx_err_crc_mismatch) {
    // Sync was found but CRC failed -- still advance to avoid infinite re-scan
    stream_read_ptr += discarded;
    rx_log_error(TAG, "sync found but CRC failed");
} else {
    rx_log_error(TAG, "no sync word within scan window");
}
```

See also

[rx_frame_decode\(\)](#) Simple decode without stream recovery
[internal_find_sync_offset\(\)](#) Bounded sync word scanner
[k_frame_max_scan_bytes](#) Scan window upper bound

Since

Version 1.0.0

Definition at line 836 of file [rx_frame.c](#).

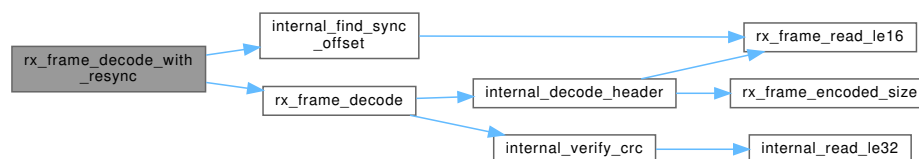
```

00841 {
00842     if (dec == nullptr || data == nullptr || frame == nullptr || bytes_discarded_out == nullptr) {
00843         return k_rx_err_invalid_arg;
00844     }
00845     *bytes_discarded_out = k_bytes_discarded_none;
00846     /* Fast path: attempt aligned decode first */
00847     rx_err_t err = rx_frame_decode(dec, data, data_len, frame);
00848     if (err != k_rx_err_protocol_error) {
00849         /* Either success, or an error other than sync mismatch (CRC, size, etc.) */
00850         return err;
00851     }
00852     /* Sync word not at offset 0: scan forward for next sync word */
00853     uint32_t sync_offset = (uint32_t)k_frame_offset_start;
00854     err = internal_find_sync_offset(data, data_len, &sync_offset);
00855     if (err != k_rx_ok) {
00856         /* No sync word found within bounded window */
00857         return k_rx_err_protocol_error;
00858     }
00859     /* Report discarded bytes now: caller needs this to advance past the sync
00860      * even if the subsequent decode fails (e.g. CRC mismatch at new offset) */
00861     *bytes_discarded_out = sync_offset;
00862     /* Reattempt decode from the discovered sync offset */
00863     err = rx_frame_decode(dec, &data[sync_offset], data_len - sync_offset, frame);
00864     return err;
00865 }

```

References [internal_find_sync_offset\(\)](#), [k_bytes_discarded_none](#), [k_frame_offset_start](#), and [rx_frame_decode\(\)](#).

Here is the call graph for this function:



4.1.4.9 rx_frame_decoder_deinit()

```

rx_err_t rx_frame_decoder_deinit (
    rx_frame_decoder_t * dec) [nodiscard]

```

Deinitialize frame decoder.

Parameters

in,out	<i>dec</i>	Pointer to decoder handle
--------	------------	---------------------------

Returns

k_rx_ok on success

Deinitialize frame decoder.

Marks the decoder as uninitialized, preventing further use. Verifies deinitialization completed successfully (redundant validation per Rule 5).

Parameters

out	<i>dec</i>	Decoder instance to deinitialize
-----	------------	----------------------------------

Returns

k_rx_ok on successful deinitialization

Return values

<i>k_rx_err_invalid_arg</i>	if dec is nullptr
<i>k_rx_err_validation_failed</i>	if deinitialization verification fails

Definition at line 690 of file [rx_frame.c](#).

```
00691 {
00692     if (dec == nullptr) {
00693         return k_rx_err_invalid_arg;
00694     }
00695     dec->initialized = k_state_uninitialized;
00696     if (dec->initialized != k_state_uninitialized) {
00697         return k_rx_err_validation_failed;
00698     }
00699     return k_rx_ok;
00700 }
00701 }
```

References [rx_frame_decoder_t::initialized](#), and [k_state_uninitialized](#).

4.1.4.10 rx_frame_decoder_init()

```
rx_err_t rx_frame_decoder_init (
    rx_frame_decoder_t * dec) [nodiscard]
```

Initialize frame decoder.

Parameters

out	<i>dec</i>	Pointer to decoder handle
-----	------------	---------------------------

Returns

`k_rx_ok` on success, `k_rx_err_invalid_arg` if `dec` is nullptr

Initialize frame decoder.

Prepares the decoder for use by setting the initialized flag and verifying the initialization completed successfully (redundant validation per Rule 5).

Parameters

out	<i>dec</i>	Decoder instance to initialize
-----	------------	--------------------------------

Returns

`k_rx_ok` on successful initialization

Return values

<i>k_rx_err_invalid_arg</i>	if <code>dec</code> is nullptr
<i>k_rx_err_validation_failed</i>	if initialization verification fails

Definition at line 665 of file `rx_frame.c`.

```

00666 {
00667     if (dec == nullptr) {
00668         return k_rx_err_invalid_arg;
00669     }
00670     dec->initialized = k_state_initialized;
00671     if (dec->initialized != k_state_initialized) {
00672         return k_rx_err_validation_failed;
00673     }
00674     return k_rx_ok;
00675 }
00676 }
```

References `rx_frame_decoder_t::initialized`, and `k_state_initialized`.

4.1.4.11 rx_frame_encode()

```

rx_err_t rx_frame_encode (
    const rx_frame_encoder_t * enc,
    const rx_frame_t * frame,
    uint8_t * output,
    uint32_t * output_len) [nodiscard]
```

Encode frame to wire format.

Serializes frame to wire format with SYNC, header, payload, and CRC-32. All multi-byte fields are little-endian (SYNC, SEQ, LEN, CRC-32).

Parameters

in	<i>enc</i>	Encoder handle
in	<i>frame</i>	Frame to encode

out	<i>output</i>	Output buffer (min k_frame_min_size + payload bytes)
out	<i>output_len</i>	Actual number of bytes written

Returns

k_rx_ok on success
 k_rx_err_invalid_arg if any pointer is nullptr
 k_rx_err_invalid_state if encoder not initialized
 k_rx_err_invalid_size if payload exceeds max

Definition at line 586 of file [rx_frame.c](#).

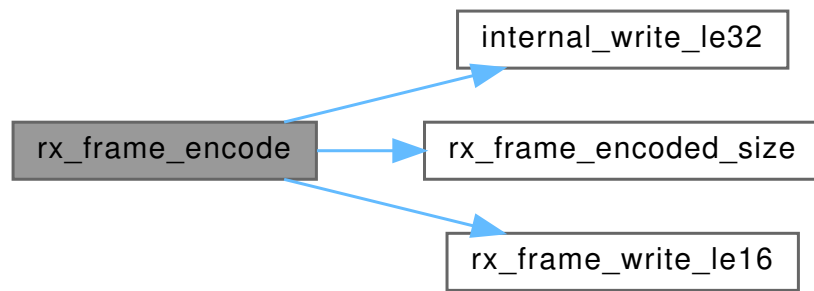
```

00590 {
00591   if (enc == nullptr || frame == nullptr || output == nullptr || output_len == nullptr) {
00592     return k_rx_err_invalid_arg;
00593   }
00594
00595   if (enc->initialized != k_state_initialized) {
00596     return k_rx_err_invalid_state;
00597   }
00598
00599   /* Validate payload size */
00600   if (frame->header.length > k_frame_max_payload) {
00601     return k_rx_err_invalid_size;
00602   }
00603
00604   /* Calculate total frame size */
00605   uint32_t frame_size = rx_frame_encoded_size(frame->header.length);
00606   uint32_t offset = k_frame_offset_start;
00607
00608   /* Write SYNC word (little-endian: wire bytes 0xAA, 0x55) */
00609   rx_frame_write_le16(&output[offset], k_frame_sync_word);
00610   offset += k_frame_sync_size;
00611
00612   /* Write SEQ (little-endian) */
00613   rx_frame_write_le16(&output[offset], frame->header.sequence);
00614   offset += k_frame_seq_size;
00615
00616   /* Write LEN (little-endian) */
00617   rx_frame_write_le16(&output[offset], frame->header.length);
00618   offset += k_frame_len_size;
00619
00620   /* Write TYPE (1 byte) */
00621   output[offset] = frame->header.type;
00622   offset += k_frame_type_size;
00623
00624   /* Write FLAGS (1 byte) */
00625   output[offset] = frame->header.flags;
00626   offset += k_frame_flags_size;
00627
00628   /* Write PAYLOAD */
00629   if (frame->header.length > 0) {
00630     memcpy(&output[offset], frame->payload, frame->header.length);
00631     offset += frame->header.length;
00632   }
00633
00634   /* Calculate CRC-32 over SYNC + Header + Payload (IEEE 802.3 polynomial) */
00635   uint32_t crc = rx_crc32_ieee(output, offset);
00636
00637   /* Write CRC-32 (little-endian to match IEEE 802.3 LSB-first order) */
00638   rx_err_t err = internal_write_le32(&output[offset], frame_size - offset, crc);
00639   if (err != k_rx_ok) {
00640     return err;
00641   }
00642   offset += k_frame_crc_size;
00643
00644   *output_len = frame_size;
00645   return k_rx_ok;
00646 }

```

References [rx_frame_header_t::flags](#), [rx_frame_t::header](#), [rx_frame_encoder_t::initialized](#), [internal_write_le32\(\)](#), [k_frame_crc_size](#), [k_frame_flags_size](#), [k_frame_len_size](#), [k_frame_max_payload](#), [k_frame_offset_start](#), [k_frame_seq_size](#), [k_frame_sync_size](#), [k_frame_sync_word](#), [k_frame_type_size](#), [k_state_initialized](#), [rx_frame_header_t::length](#), [rx_frame_t::payload](#), [rx_frame_encoded_size\(\)](#), [rx_frame_write_le16\(\)](#), [rx_frame_header_t::sequence](#), and [rx_frame_header_t::type](#).

Here is the call graph for this function:



4.1.4.12 rx_frame_encoded_size()

```
uint32_t rx_frame_encoded_size (
    uint32_t payload_len)  [inline], [static]
```

Calculate encoded frame size.

Parameters

in	<i>payload_len</i>	Payload length
----	--------------------	----------------

Returns

Total frame size in bytes

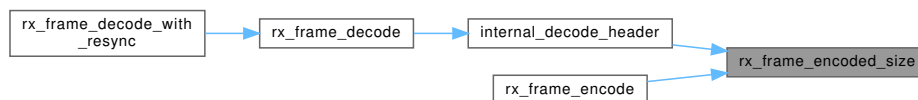
Definition at line 919 of file [rx_frame.h](#).

```
00920 {
00921     return k_frame_sync_size + k_frame_header_size + payload_len + k_frame_crc_size;
00922 }
```

References [k_frame_crc_size](#), [k_frame_header_size](#), and [k_frame_sync_size](#).

Referenced by [internal_decode_header\(\)](#), and [rx_frame_encode\(\)](#).

Here is the caller graph for this function:



4.1.4.13 rx_frame_encoder_deinit()

```
rx_err_t rx_frame_encoder_deinit (
    rx_frame_encoder_t * enc)  [nodiscard]
```

Deinitialize frame encoder.

Parameters

in,out	<i>enc</i>	Pointer to encoder handle
--------	------------	---------------------------

Returns

k_rx_ok on success

Deinitialize frame encoder.

Marks the encoder as uninitialized, preventing further use. Verifies deinitialization completed successfully (redundant validation per Rule 5).

Parameters

out	<i>enc</i>	Encoder instance to deinitialize
-----	------------	----------------------------------

Returns

k_rx_ok on successful deinitialization

Return values

<i>k_rx_err_invalid_arg</i>	if <i>enc</i> is nullptr
<i>k_rx_err_validation_failed</i>	if deinitialization verification fails

Definition at line 573 of file [rx_frame.c](#).

```
00574 {  
00575     if (enc == nullptr) {  
00576         return k_rx_err_invalid_arg;  
00577     }  
00578     enc->initialized = k_state_uninitialized;  
00579     if (enc->initialized != k_state_uninitialized) {  
00580         return k_rx_err_validation_failed;  
00581     }  
00582     return k_rx_ok;  
00583 }  
00584 }
```

References [rx_frame_encoder_t::initialized](#), and [k_state_uninitialized](#).

4.1.4.14 rx_frame_encoder_init()

```
rx_err_t rx_frame_encoder_init (  
    rx\_frame\_encoder\_t * enc) [nodiscard]
```

Initialize frame encoder.

Parameters

out	<i>enc</i>	Pointer to encoder handle
-----	------------	---------------------------

Returns

`k_rx_ok` on success, `k_rx_err_invalid_arg` if `enc` is `nullptr`

Initialize frame encoder.

Prepares the encoder for use by setting the initialized flag and verifying the initialization completed successfully (redundant validation per Rule 5).

Parameters

out	<i>enc</i>	Encoder instance to initialize
-----	------------	--------------------------------

Returns

`k_rx_ok` on successful initialization

Return values

<i>k_rx_err_invalid_arg</i>	if <code>enc</code> is <code>nullptr</code>
<i>k_rx_err_validation_failed</i>	if initialization verification fails

Definition at line 548 of file [rx_frame.c](#).

```

00549 {
00550     if (enc == nullptr) {
00551         return k_rx_err_invalid_arg;
00552     }
00553     enc->initialized = k_state_initialized;
00554     if (enc->initialized != k_state_initialized) {
00555         return k_rx_err_validation_failed;
00556     }
00557     return k_rx_ok;
00558 }
00559 }
```

References [rx_frame_encoder_t::initialized](#), and [k_state_initialized](#).

4.1.4.15 rx_frame_read_le16()

```
uint16_t rx_frame_read_le16 (
    const uint8_t * buf)    [inline], [static]
```

Read uint16 from little-endian buffer.

Reads a 16-bit unsigned integer from a buffer in little-endian format. The low byte (LSB) is at index 0, the high byte (MSB) is at index 1.

Parameters

in	<i>buf</i>	Input buffer (at least 2 bytes)
----	------------	---------------------------------

Returns

Decoded 16-bit value in host byte order

Note

This is the preferred function for parsing multi-byte protocol fields.

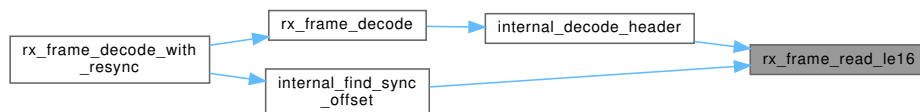
Definition at line 991 of file [rx_frame.h](#).

```
00992 {
00993     return (uint16_t)buf[k_le16_byte_low] | ((uint16_t)buf[k_le16_byte_high] « k_rx_le16_high_shift);
00994 }
```

References [k_le16_byte_high](#), [k_le16_byte_low](#), and [k_rx_le16_high_shift](#).

Referenced by [internal_decode_header\(\)](#), and [internal_find_sync_offset\(\)](#).

Here is the caller graph for this function:



4.1.4.16 rx_frame_read_le32()

```
uint32_t rx_frame_read_le32 (
    const uint8_t * buf)    [inline], [static]
```

Read uint32 from little-endian buffer.

Reads a 32-bit unsigned integer from a buffer in little-endian format.

Parameters

in	<i>buf</i>	Input buffer (at least 4 bytes)
----	------------	---------------------------------

Returns

Decoded 32-bit value in host byte order

Definition at line 1023 of file [rx_frame.h](#).

```
01024 {
01025     return (uint32_t)buf[k_le32_byte] | ((uint32_t)buf[k_le32_byte] « k_rx_le32_shift) |
01026         ((uint32_t)buf[k_le32_byte] « k_rx_le32_shift) |
01027         ((uint32_t)buf[k_le32_byte] « k_rx_le32_shift);
01028 }
```

References [k_le32_byte0](#), [k_le32_byte1](#), [k_le32_byte2](#), [k_le32_byte3](#), [k_rx_le32_shift0](#), [k_rx_le32_shift1](#), and [k_rx_le32_shift2](#).

4.1.4.17 rx_frame_type_valid()

```
bool rx_frame_type_valid (
    uint8_t type)  [inline], [static]
```

Check if frame type is valid.

Static inline for same reasons as [rx_frame_encoded_size\(\)](#) above.

Parameters

in	<i>type</i>	Frame type to check
----	-------------	---------------------

Returns

true if valid, false otherwise

Definition at line 1239 of file [rx_frame.h](#).

```
01240 {
01241     switch (type) {
01242     case k_frame_type_ping:
01243     case k_frame_type_pong:
01244     case k_frame_type_command:
01245     case k_frame_type_response:
01246     case k_frame_type_ack:
01247     case k_frame_type_nack:
01248     case k_frame_type_reset_ack:
01249     case k_frame_type_reset:
01250         return true;
01251     default:
01252         return false;
01253     }
01254 }
```

References [k_frame_type_ack](#), [k_frame_type_command](#), [k_frame_type_nack](#), [k_frame_type_ping](#), [k_frame_type_pong](#), [k_frame_type_reset](#), [k_frame_type_reset_ack](#), and [k_frame_type_response](#).

4.1.4.18 rx_frame_write_le16()

```
void rx_frame_write_le16 (
    uint8_t * buf,
    uint16_t val)  [inline], [static]
```

Write uint16 to little-endian buffer.

Writes a 16-bit unsigned integer to a buffer in little-endian format. The low byte (LSB) is written at index 0, the high byte (MSB) is written at index 1.

Parameters

out	<i>buf</i>	Output buffer (at least 2 bytes)
in	<i>val</i>	Value to write in host byte order

Note

This is the preferred function for serializing multi-byte protocol fields.

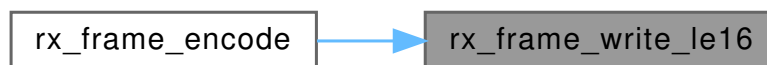
Definition at line 1009 of file rx_frame.h.

```
01010 {
01011     buf[k_le16_byte_low] = (uint8_t)(val & k_rx_byte_mask);
01012     buf[k_le16_byte_high] = (uint8_t)(val >> k_rx_le16_high_shift);
01013 }
```

References [k_le16_byte_high](#), [k_le16_byte_low](#), [k_rx_byte_mask](#), and [k_rx_le16_high_shift](#).

Referenced by [rx_frame_encode\(\)](#).

Here is the caller graph for this function:



4.2 rx_frame.h

[Go to the documentation of this file.](#)

```
00001 /* lib/rx_frame/inc/rx_frame.h */
00002
00003 /**
00004  * @file rx_frame.h
00005  * @brief Frame Layer Protocol for SPI Communication
00006  *
00007  * @details
00008  * ## Overview
00009  *
00010  * This module implements the frame-level protocol for reliable SPI communication
00011  * between the Raspberry Pi 5 gateway and the RX72N motor controller. It provides
00012  * frame encoding, decoding, and CRC-32 integrity verification.
00013  *
00014  * **Key Features:**
00015  * - Bit-exact compatible with star-gateway/internal/frame/ (Go implementation)
00016  * - IEEE 802.3 CRC-32 for data integrity
00017  * - Little-endian byte order for ALL multi-byte fields (SYNC, SEQ, LEN, CRC)
00018  * - Support for ACK/NACK flow control
00019  * - Optional forward error correction (FEC) flag
00020  *
00021  * ## Frame Format
00022  *
00023  * ```
00024  * +-----+-----+-----+-----+-----+-----+-----+
00025  * | SYNC  | SEQ   | LEN   | TYPE  | FLAGS  | PAYLOAD | CRC-32 |
00026  * | 2 bytes| 2 bytes| 2 bytes| 1 byte| 1 byte | 0-1024 B | 4 bytes |
00027  * | (LE)  | (LE)  | (LE)  |      |      |      | (LE)  |
00028  * +-----+-----+-----+-----+-----+-----+
00029  * ```
00030  *
00031  * **Field Descriptions:**
00032  *
00033  * | Field | Size | Byte Order | Description |
00034  * |-----|-----|-----|-----|
00035  * | SYNC | 2 | Little-endian | Synchronization marker (0x55AA, wire: 0xAA 0x55) |
00036  * | SEQ | 2 | Little-endian | Sequence number for ordering/retransmit |
00037  * | LEN | 2 | Little-endian | Payload length in bytes (0-1024) |
00038  * | TYPE | 1 | N/A | Frame type (command, response, ack, nack) |
00039  * | FLAGS | 1 | N/A | Control flags (requires_ack, retransmit, etc.) |
00040  * | PAYLOAD | 0-1024 | N/A | Encoded protobuf message or empty |
00041  * | CRC-32 | 4 | Little-endian | IEEE 802.3 CRC over SYNC+header+payload |
00042  *
00043  * ## Frame Size Calculations
00044  *
00045  * - **Minimum frame**: 12 bytes (SYNC + header + CRC, no payload)
00046  * - **Maximum frame**: 1036 bytes (SYNC + header + 1024 payload + CRC)
00047  * - **Overhead**: 12 bytes fixed per frame
00048  *
00049  * ## Protocol Flow
00050  *
```

```

00051 * @msc
00052 * RPi5, RX72N;
00053 *
00054 * --- [label="Normal Command/Response"];
00055 * RPi5 => RX72N [label="COMMAND (seq=1, requires_ack)"];
00056 * RPi5 <= RX72N [label="ACK (seq=1)"];
00057 * RPi5 <= RX72N [label="RESPONSE (seq=1)"];
00058 * RPi5 => RX72N [label="ACK (seq=1)"];
00059 *
00060 * --- [label="Retransmission on CRC Error"];
00061 * RPi5 => RX72N [label="COMMAND (seq=2)"];
00062 * RPi5 box RPi5 [label="Timeout"];
00063 * RPi5 => RX72N [label="COMMAND (seq=2, retransmit)"];
00064 * RPi5 <= RX72N [label="ACK (seq=2)"];
00065 *
00066 * --- [label="NACK on Decode Error"];
00067 * RPi5 => RX72N [label="COMMAND (seq=3, bad payload)"];
00068 * RPi5 <= RX72N [label="NACK (seq=3, soft_nack)"];
00069 * @endmsc
00070
00071 * ## CRC-32 Details
00072 *
00073 * The CRC-32 is computed using the IEEE 802.3 polynomial (0x04C11DB7) over
00074 * the entire frame excluding the CRC field itself:
00075 *
00076 * @f[
00077 * \text{CRC} = \text{CRC32}(\text{IEEE})(\text{SYNC} \parallel \text{Header} \parallel \text{Payload})
00078 * @f]
00079 *
00080 * **CRC Coverage:**
00081 * - SYNC word (2 bytes)
00082 * - SEQ (2 bytes)
00083 * - LEN (2 bytes)
00084 * - TYPE (1 byte)
00085 * - FLAGS (1 byte)
00086 * - PAYLOAD (0-1024 bytes)
00087 *
00088 * **Why Little-Endian CRC?**
00089 * The CRC-32 is stored in little-endian format (LSB first) to match the
00090 * IEEE 802.3 bit transmission order. This ensures compatibility with
00091 * Go's `crc32.ChecksumIEEE()` function.
00092 *
00093 * ## Thread Safety
00094 *
00095 * | Component | Thread Safe | Notes |
00096 * |-----|-----|-----|
00097 * | Encoder | [PASS] Yes | Stateless after init |
00098 * | Decoder | [PASS] Yes | Stateless after init |
00099 * | Utility functions | [PASS] Yes | Pure functions |
00100 *
00101 * ## Memory Usage
00102 *
00103 * | Component | Size | Notes |
00104 * |-----|-----|-----|
00105 * | rx_frame_t | 1034 bytes | Header + 1024-byte payload + CRC |
00106 * | rx_frame_encoder_t | 1 byte | Initialization flag only |
00107 * | rx_frame_decoder_t | 1 byte | Initialization flag only |
00108 * | Encoded frame buffer | 1036 bytes | Caller-provided |
00109 *
00110 * ## Performance Characteristics
00111 *
00112 * **Measured on RX72N @ 240 MHz, -O2 optimization:**
00113 *
00114 * | Operation | Empty Payload | 64B Payload | 255B Payload |
00115 * |-----|-----|-----|-----|
00116 * | Encode | ~5 us | ~15 us | ~50 us |
00117 * | Decode | ~5 us | ~15 us | ~50 us |
00118 * | CRC calc | ~1 us | ~6 us | ~25 us |
00119 *
00120 * ## NASA Power of 10 Compliance
00121 *
00122 * | Rule | Status | Notes |
00123 * |-----|-----|-----|
00124 * | 1. Simple control flow | [PASS] Pass | No goto/setjmp/recursion |
00125 * | 2. Fixed loop bounds | [PASS] Pass | Bounded by k_frame_max_payload |
00126 * | 3. No dynamic memory | [PASS] Pass | All buffers caller-provided |
00127 * | 4. Short functions | [PASS] Pass | All functions < 50 lines |
00128 * | 5. Assertions | [PASS] Pass | NULL checks, state validation |
00129 * | 6. Small scope | [PASS] Pass | Variables at minimal scope |
00130 * | 7. Check returns | [PASS] Pass | All returns propagated |
00131 * | 8. Limited preprocessor | [PASS] Pass | Only include guards |
00132 * | 9. Restrict pointers | [PASS] Pass | No function pointers |
00133 * | 10. Compiler warnings | [PASS] Pass | -Wall -Wextra -Werror |
00134 *
00135 * ## SOLID Principles
00136 *
00137 * | Principle | Implementation |

```



```

00138 * |-----|-----|
00139 * |**Single Responsibility** | Frame layer only - no protobuf knowledge |
00140 * |**Open/Closed** | New frame types via enum, no code changes |
00141 * |**Liskov Substitution** | Encoder/decoder handles are interchangeable |
00142 * |**Interface Segregation** | Separate encoder/decoder APIs |
00143 * |**Dependency Inversion** | Depends on rx_crc.h abstraction |
00144 *
00145 * ## Hardware Requirements
00146 *
00147 * | Resource | Requirement |
00148 * |-----|-----|
00149 * |**SPI** | 10 Mbps, Full-duplex |
00150 * |**DMA** | Optional, improves throughput |
00151 * |**CRC** | Uses rx_crc module (HW or SW) |
00152 *
00153 * ## Integration Example
00154 *
00155 * @code{.c}
00156 * #include "rx_frame.h"
00157 * #include "rx_spi_comm.h"
00158 *
00159 * // Transmit a command frame
00160 * void send_motor_command(uint8_t* protobuf_payload, uint32_t payload_len)
00161 * {
00162 *     rx_frame_encoder_t encoder;
00163 *     rx_frame_t frame;
00164 *     uint8_t wire_buffer[1036];
00165 *     uint32_t wire_len;
00166 *
00167 *     // Initialize encoder
00168 *     rx_frame_encoder_init(&encoder);
00169 *
00170 *     // Build frame
00171 *     frame.header.sequence = get_next_sequence();
00172 *     frame.header.length = payload_len;
00173 *     frame.header.type = k_frame_type_command;
00174 *     frame.header.flags = k_frame_flag_requires_ack;
00175 *     memcpy(frame.payload, protobuf_payload, payload_len);
00176 *
00177 *     // Encode to wire format
00178 *     rx_err_t err = rx_frame_encode(&encoder, &frame, wire_buffer, &wire_len);
00179 *     if (err != k_rx_ok) {
00180 *         rx_log_error("FRAME", "Encode failed: %d", err);
00181 *         return;
00182 *     }
00183 *
00184 *     // Send via SPI
00185 *     spi_transmit(wire_buffer, wire_len);
00186 * }
00187 *
00188 * // Receive and decode a frame
00189 * rx_err_t receive_frame(uint8_t* wire_data, uint32_t wire_len, rx_frame_t* frame)
00190 * {
00191 *     rx_frame_decoder_t decoder;
00192 *
00193 *     // Initialize decoder
00194 *     rx_frame_decoder_init(&decoder);
00195 *
00196 *     // Decode frame (validates SYNC, parses header, verifies CRC)
00197 *     rx_err_t err = rx_frame_decode(&decoder, wire_data, wire_len, frame);
00198 *     if (err == k_rx_err_crc_mismatch) {
00199 *         rx_log_error("FRAME", "CRC mismatch, requesting retransmit");
00200 *         send_nack(frame->header.sequence);
00201 *         return err;
00202 *     }
00203 *
00204 *     return err;
00205 * }
00206 * @endcode
00207 *
00208 * ## References
00209 *
00210 * - star-gateway/internal/frame/: Go reference implementation
00211 * - docs/sections/01_nanoph_protocol.tex: Protocol specification
00212 * - IEEE 802.3: CRC-32 polynomial definition
00213 * @see rx_frame.c Implementation file
00214 * @see rx_crc.h CRC-32 computation
00215 * @see rx_spi_comm.h SPI transport layer
00216 * @see rx_nanoph.h Protobuf encoding for payloads
00217 *
00218 * @author STAR Team
00219 * @date 2026-01-29
00220 * @copyright Copyright (c) 2026 STAR Project. MIT License.
00221 * @since Version 1.0.0
00222 */
00223
00224 #pragma once

```

```

00225
00226 #include <stdbool.h>
00227 #include <stddef.h>
00228 #include <stdint.h>
00229
00230 #include "rx_err.h"
00231
00232 #ifdef __cplusplus
00233 extern "C" {
00234 #endif
00235
00236 /*
=====
00237 * Frame Constants (must match Go implementation exactly)
00238 *
=====
00239 */
00240
00241 /**
00242  * @enum rx_frame_constants_t
00243  * @brief Frame structure constants for SPI protocol
00244  *
00245  * @details
00246  * These constants define the frame format parameters that must exactly match
00247  * the Go implementation in star-gateway/internal/frame/. Any mismatch will
00248  * cause protocol interoperability failures.
00249  *
00250  * ## Frame Layout
00251  *
00252  * ```
00253  * Offset: 0      2      4      6      7      8      8+len
00254  * Field:  SYNC   SEQ    LEN    TYPE  FLAGS PAYLOAD... CRC
00255  * Size:   2      2      2      1      1    0-1024   4
00256  * ```
00257  *
00258  * ## Size Calculations
00259  *
00260  * | Frame Type | Formula | Bytes |
00261  * |-----|-----|-----|
00262  * | Minimum (no payload) | SYNC + Header + CRC | 12 |
00263  * | With N-byte payload | 12 + N | 12 - 1036 |
00264  * | Maximum (1KB payload) | SYNC + Header + 1024 + CRC | 1036 |
00265  *
00266  * @par Usage Example:
00267  * @code{.c}
00268  * // Validate frame size
00269  * if (wire_len < k_frame_min_size || wire_len > k_frame_max_size) {
00270  *     return k_rx_err_invalid_size;
00271  * }
00272  *
00273  * // Check sync word
00274  * uint16_t sync = rx_frame_read_le16(wire_data);
00275  * if (sync != k_frame_sync_word) {
00276  *     return k_rx_err_protocol_error;
00277  * }
00278  *
00279  * // Calculate expected size from payload length
00280  * uint16_t payload_len = rx_frame_read_le16(&wire_data[4]);
00281  * uint32_t expected = rx_frame_encoded_size(payload_len);
00282  * @endcode
00283  *
00284  * @warning These values MUST match star-gateway/internal/frame/constants.go
00285  * @note Using uint16_t underlying type to accommodate k_frame_max_payload (1024)
00286  *
00287  * @see star-gateway/internal/frame/ Go reference implementation
00288  * @since Version 1.0.0
00289  */
00290 typedef enum : uint16_t {
00291     k_frame_sync_word = 0x55AA, /**< @brief Frame synchronization marker (0x55AA)
00292     * @details
00293     * The sync word marks the start of a valid frame. Receivers scan for this
00294     * pattern to establish frame boundaries. The value 0x55AA was chosen for:
00295     * - Alternating bit pattern aids clock recovery
00296     * - Unique enough to avoid false positives in random data
00297     * - Matches common SPI debug patterns
00298     * @par Wire Format: Little-endian (0xAA first, then 0x55)
00299     */
00300
00301     k_frame_sync_size = 2, /**< @brief SYNC field size in bytes (2)
00302     * @details Two-byte sync marker at frame start.
00303     */
00304
00305     k_frame_seq_size = 2, /**< @brief SEQ field size in bytes (2)
00306     * @details 16-bit sequence number for ordering and retransmit tracking.
00307     */
00308
00309     k_frame_len_size = 2, /**< @brief LEN field size in bytes (2)

```

```

00310             * @details 16-bit payload length (max 1024).
00311             */
00312
00313 k_frame_type_size = 1, /**< @brief TYPE field size in bytes (1)
00314             * @details Single byte for frame type (command, response, ack, nack).
00315             */
00316
00317 k_frame_flags_size = 1, /**< @brief FLAGS field size in bytes (1)
00318             * @details Single byte for control flags (requires_ack, retransmit, etc.).
00319             */
00320
00321 k_frame_crc_size = 4, /**< @brief CRC-32 field size in bytes (4)
00322             * @details IEEE 802.3 CRC-32 checksum.
00323             */
00324
00325 k_frame_header_size = 6, /**< @brief Header size excluding SYNC (6 bytes)
00326             * @details SEQ(2) + LEN(2) + TYPE(1) + FLAGS(1) = 6 bytes.
00327             */
00328
00329 k_frame_min_payload = 0, /**< @brief Minimum payload length (0 bytes)
00330             * @details ACK and NACK frames have empty payloads.
00331             */
00332
00333 k_frame_max_payload = 1024, /**< @brief Maximum payload length (1024 bytes)
00334             * @details Limited by SPI DMA buffer size and protobuf message limits.
00335             */
00336
00337 k_frame_min_size = 12, /**< @brief Minimum frame size (12 bytes)
00338             * @details SYNC(2) + Header(6) + CRC(4) = 12 bytes (no payload).
00339             */
00340
00341 k_frame_max_size = 1036, /**< @brief Maximum frame size (1036 bytes)
00342             * @details SYNC(2) + Header(6) + Payload(1024) + CRC(4) = 1036 bytes.
00343             */
00344
00345 k_frame_max_scan_bytes =
00346     k_frame_max_size, /**< @brief Maximum bytes to scan forward during sync recovery (1036)
00347             * @details
00348             * When the sync word is not found at offset 0, the decoder scans
00349             * forward up to this many bytes looking for the sync pattern. The
00350             * upper bound equals k_frame_max_size so recovery never reads more
00351             * data than one complete maximum-size frame. This ensures bounded
00352             * execution time (NASA Rule 2) and prevents unbounded stream
00353             * consumption on permanently corrupt inputs.
00354             *
00355             * The scan discards all bytes before the discovered sync word and
00356             * reattempts decoding from that position.
00357             */
00358 } rx_frame_constants_t;
00359
00360 /**
00361  * @enum rx_frame_type_t
00362  * @brief Frame types for SPI protocol (matches Go FrameType)
00363  *
00364  * @details
00365  * Defines the type of message contained in a frame. The frame type determines
00366  * the expected payload content and required response handling.
00367  *
00368  * ## Communication Pattern
00369  *
00370  * @startuml
00371  * [*] --> Idle
00372  *
00373  * Idle --> WaitingForAck : TX COMMAND
00374  * WaitingForAck --> ProcessingResponse : RX ACK
00375  * WaitingForAck --> Idle : Timeout (retransmit)
00376  * WaitingForAck --> Idle : RX NACK (error)
00377  *
00378  * ProcessingResponse --> SendingAck : RX RESPONSE
00379  * SendingAck --> Idle : TX ACK
00380  * @enduml
00381  *
00382  * ## Type Descriptions
00383  *
00384  * | Type | Direction | Payload | Response |
00385  * |-----|-----|-----|-----|
00386  * | COMMAND | RPi5 -> RX72N | Protobuf | ACK, then RESPONSE |
00387  * | RESPONSE | RX72N -> RPi5 | Protobuf | ACK |
00388  * | ACK | Either | None | None |
00389  * | NACK | Either | None (flags indicate reason) | Retransmit |
00390  *
00391  * @par Usage Example:
00392  * @code{.c}
00393  * switch (frame->header.type) {
00394  *     case k_frame_type_command:
00395  *         // Process command, send ACK then RESPONSE
00396  *         send_ack(frame->header.sequence);

```

```

00397 *     process_command(frame->payload, frame->header.length);
00398 *     send_response(frame->header.sequence);
00399 *     break;
00400 * case k_frame_type_ack:
00401 *     // Mark pending command as acknowledged
00402 *     mark_acked(frame->header.sequence);
00403 *     break;
00404 * case k_frame_type_nack:
00405 *     // Retransmit the failed frame
00406 *     retransmit(frame->header.sequence);
00407 *     break;
00408 * default:
00409 *     rx_log_error("FRAME", "Unknown type: %u", frame->header.type);
00410 * }
00411 * @endcode
00412 *
00413 * @warning Values MUST match star-gateway/internal/frame/types.go
00414 * @see rx_frame_type_valid() Validate frame type
00415 * @since Version 1.0.0
00416 */
00417 typedef enum : uint8_t {
00418 /**
00419  * @brief Ping request (0x00)
00420  * @details
00421  * Keepalive/heartbeat message sent to check connection status.
00422  * Receiver responds with PONG echoing the payload.
00423  * @par Payload: 4-byte counter (little-endian uint32) or empty
00424  */
00425 k_frame_type_ping = 0x00,
00426
00427 /**
00428  * @brief Pong response (0x01)
00429  * @details
00430  * Response to PING request, confirms connection is alive.
00431  * Echoes the PING counter payload for validation.
00432  * @par Payload: Echo of PING counter or empty
00433  */
00434 k_frame_type_pong = 0x01,
00435
00436 /**
00437  * @brief Command from controller to peripheral (0x10)
00438  * @details
00439  * Contains a protobuf-encoded command message from RPi5 to RX72N.
00440  * On SPI: typically requires ACK and generates a RESPONSE frame.
00441  * On USB CDC: no ACK expected (hardware reliability).
00442  * @par Typical Payloads: MotorCommand, TelemetryRequest, ConfigWrite
00443  */
00444 k_frame_type_command = 0x10,
00445
00446 /**
00447  * @brief Response from peripheral to controller (0x11)
00448  * @details
00449  * Contains a protobuf-encoded response message from RX72N to RPi5.
00450  * Sent after processing a COMMAND.
00451  * @par Typical Payloads: MotorStatus, TelemetryData, ConfigAck
00452  */
00453 k_frame_type_response = 0x11,
00454
00455 /**
00456  * @brief Positive acknowledgment (0x12, SPI only)
00457  * @details
00458  * Confirms successful reception and CRC validation of a frame.
00459  * Empty payload; sequence number matches the acknowledged frame.
00460  * Used only on SPI transport (USB CDC relies on hardware reliability).
00461  * @par Payload: None (length = 0)
00462  */
00463 k_frame_type_ack = 0x12,
00464
00465 /**
00466  * @brief Negative acknowledgment (0x13, SPI only)
00467  * @details
00468  * Indicates a problem with received frame (CRC error, decode failure).
00469  * Empty payload; flags indicate error type. Sender should retransmit.
00470  * Used only on SPI transport (USB CDC relies on hardware reliability).
00471  * @par Payload: None (length = 0)
00472  * @par Flags: May include k_frame_flag_soft_nack
00473  */
00474 k_frame_type_nack = 0x13,
00475
00476 /**
00477  * @brief Reset acknowledgment (0xFE)
00478  * @details
00479  * Confirms reset operation completed successfully.
00480  * Sequence number matches the reset request.
00481  * @par Payload: None (length = 0)
00482  */
00483 k_frame_type_reset_ack = 0xFE,

```

```

00484
00485 /**
00486  * @brief Reset request (0xFF)
00487  * @details
00488  * Request to reset communication state (sequence numbers, buffers, etc.).
00489  * Receiver responds with RESET_ACK.
00490  * @par Payload: None (length = 0)
00491  */
00492 k_frame_type_reset = 0xFF,
00493 } rx_frame_type_t;
00494
00495 /**
00496  * @enum rx_frame_flags_t
00497  * @brief Frame control flags (matches Go FrameFlags)
00498  *
00499  * @details
00500  * Bit flags that modify frame handling behavior. Multiple flags can be
00501  * combined using bitwise OR.
00502  *
00503  * ## Flag Descriptions
00504  *
00505  * | Flag | Bit | Purpose |
00506  * |-----|-----|-----|
00507  * | REQUIRES_ACK | 0 | Sender expects ACK/NACK response |
00508  * | RETRANSMIT | 1 | This is a retransmission of previous frame |
00509  * | PRIORITY | 2 | Process before normal-priority frames |
00510  * | FEC_ENABLED | 3 | Payload uses forward error correction |
00511  * | SOFT_NACK | 4 | NACK with recoverable error info |
00512  *
00513  * ## Flag Combinations
00514  *
00515  * @code{.c}
00516  * // Typical command frame
00517  * frame.header.flags = k_frame_flag_requires_ack;
00518  *
00519  * // Retransmission after timeout
00520  * frame.header.flags = k_frame_flag_requires_ack | k_frame_flag_retransmit;
00521  *
00522  * // High-priority motor command
00523  * frame.header.flags = k_frame_flag_requires_ack | k_frame_flag_priority;
00524  *
00525  * // FEC-protected data transfer
00526  * frame.header.flags = k_frame_flag_fec_enabled;
00527  *
00528  * // NACK with error details
00529  * nack_frame.header.flags = k_frame_flag_soft_nack;
00530  * @endcode
00531  *
00532  * @par Usage Example:
00533  * @code{.c}
00534  * // Check if ACK required
00535  * if (frame->header.flags & k_frame_flag_requires_ack) {
00536  *     send_ack(frame->header.sequence);
00537  * }
00538  *
00539  * // Check if retransmit
00540  * if (frame->header.flags & k_frame_flag_retransmit) {
00541  *     rx_log_warn("FRAME", "Received retransmit seq=%u",
00542  *         frame->header.sequence);
00543  *     stats.retransmit_count++;
00544  * }
00545  * @endcode
00546  *
00547  * @warning Values MUST match star-gateway/internal/frame/flags.go
00548  * @since Version 1.0.0
00549  */
00550 typedef enum : uint8_t {
00551     k_frame_flag_none = 0x00, /**< @brief No flags set (0x00)
00552                               * @details Default for frames that don't require special handling.
00553                               */
00554
00555     k_frame_flag_requires_ack = 0x01, /**< @brief Frame requires acknowledgment (0x01, bit 0)
00556                                       * @details
00557                                       * Sender expects an ACK or NACK response. If no response within
00558                                       * timeout, sender should retransmit with RETRANSMIT flag.
00559                                       */
00560
00561     k_frame_flag_retransmit = 0x02, /**< @brief Retransmission of previous frame (0x02, bit 1)
00562                                       * @details
00563                                       * Indicates this frame is a retry after timeout or NACK.
00564                                       * Receiver should check for duplicate sequence numbers.
00565                                       */
00566
00567     k_frame_flag_priority = 0x04, /**< @brief High-priority frame (0x04, bit 2)
00568                                       * @details
00569                                       * Process before normal-priority frames in queue.
00570                                       * Used for emergency stop and critical motor commands.

```

```

00571         */
00572
00573     k_frame_flag_fec_enabled = 0x08, /**< @brief Forward error correction enabled (0x08, bit 3)
00574         * @details
00575         * Payload is FEC-encoded (convolutional rate 1/2). FEC is enabled by default
00576         * on the HARQ path (e.g., SPI transport with rx_spi_link). This flag indicates
00577         * FEC encoding is present and receiver should apply Viterbi decoding before
00578         * processing payload. The flag is ignored for transports that do not use HARQ
00579         * (e.g., USB CDC, which relies on built-in hardware CRC and retry).
00580         */
00581
00582     k_frame_flag_soft_nack = 0x10, /**< @brief NACK with soft error information (0x10, bit 4)
00583         * @details
00584         * Used in NACK frames to indicate recoverable error.
00585         * May include confidence bits or partial decode info.
00586         */
00587 } rx_frame_flags_t;
00588
00589 /*
=====
00590 * Frame Structures
00591 *
=====
00592 */
00593
00594 /**
00595  * @struct rx_frame_header_t
00596  * @brief Frame header containing control fields
00597  *
00598  * @details
00599  * The frame header contains all control information needed to process a frame.
00600  * All 16-bit fields are stored in host byte order within this structure;
00601  * conversion to/from little-endian wire format is handled by the encoder/decoder.
00602  *
00603  * ## Memory Layout
00604  *
00605  * | Offset | Size | Field | Type | Description |
00606  * |-----|-----|-----|-----|-----|
00607  * | 0 | 2 | sequence | uint16_t | Sequence number |
00608  * | 2 | 2 | length | uint16_t | Payload length |
00609  * | 4 | 1 | type | uint8_t | Frame type |
00610  * | 5 | 1 | flags | uint8_t | Control flags |
00611  * | **Total** | **6** | | | No padding required |
00612  *
00613  * ## Wire Format vs In-Memory Format
00614  *
00615  * **Wire Format (little-endian):**
00616  * - sequence: LSB at lower address
00617  * - length: LSB at lower address
00618  *
00619  * **In-Memory (this struct):**
00620  * - sequence: Host byte order (little-endian on RX72N, matches wire)
00621  * - length: Host byte order
00622  *
00623  * @par Usage Example:
00624  * @code{.c}
00625  * rx_frame_header_t header = {
00626  *     .sequence = 0x1234, // Will be sent as 0x34, 0x12 on wire (LE)
00627  *     .length = 64, // 64-byte payload
00628  *     .type = k_frame_type_command,
00629  *     .flags = k_frame_flag_requires_ack,
00630  * };
00631  * @endcode
00632  *
00633  * @invariant length <= k_frame_max_payload (1024)
00634  * @invariant type is valid rx_frame_type_t value
00635  *
00636  * @see rx_frame_t Complete frame structure
00637  * @since Version 1.0.0
00638  */
00639 typedef struct {
00640     /**
00641      * @brief Frame sequence number
00642      * @details
00643      * 16-bit sequence number for ordering, duplicate detection, and
00644      * ACK/NACK correlation. Wraps around at 65535.
00645      * @par Wire Format: Little-endian
00646      * @par Valid Range: 0-65535
00647      */
00648     uint16_t sequence;
00649
00650     /**
00651      * @brief Payload length in bytes
00652      * @details
00653      * Length of the payload field (not including header or CRC).
00654      * Zero for ACK/NACK frames.
00655      * @par Wire Format: Little-endian

```

```

00656     * @par Valid Range: 0-1024 (k_frame_max_payload)
00657     * @invariant length <= k_frame_max_payload
00658     */
00659     uint16_t length;
00660
00661     /**
00662      * @brief Frame type identifier
00663      * @details
00664      * Identifies the message type (command, response, ack, nack).
00665      * @par Valid Values: rx_frame_type_t enumeration
00666      * @see rx_frame_type_valid() Validate type
00667      */
00668     uint8_t type;
00669
00670     /**
00671      * @brief Control flags
00672      * @details
00673      * Bitfield of rx_frame_flags_t values controlling frame handling.
00674      * Multiple flags can be combined with bitwise OR.
00675      * @par Valid Values: Combination of rx_frame_flags_t values
00676      */
00677     uint8_t flags;
00678 } rx_frame_header_t;
00679
00680 /**
00681  * @struct rx_frame_t
00682  * @brief Complete frame structure containing header, payload, and CRC
00683  *
00684  * @details
00685  * This structure represents a complete decoded frame ready for processing.
00686  * The encoder serializes this to wire format; the decoder populates it
00687  * from wire format.
00688  *
00689  * ## Memory Layout
00690  *
00691  * | Offset | Size | Field | Description |
00692  * |-----|-----|-----|-----|
00693  * | 0 | 6 | header | Frame header (rx_frame_header_t) |
00694  * | 6 | 1024 | payload | Payload buffer (max size) |
00695  * | 1030 | 4 | crc | CRC-32 checksum (for decoded frames) |
00696  * | **Total** | **1034** | | |
00697  *
00698  * **Note:** Structure size is 1034 bytes. The actual payload length is
00699  * indicated by header.length, not the buffer size.
00700  *
00701  * ## Usage Patterns
00702  *
00703  * **Encoding (TX):**
00704  * 1. Populate header fields (sequence, length, type, flags)
00705  * 2. Copy payload data to payload[] array
00706  * 3. Call rx_frame_encode() - CRC is computed automatically
00707  *
00708  * **Decoding (RX):**
00709  * 1. Call rx_frame_decode() with wire data
00710  * 2. Access header fields to determine frame type
00711  * 3. Read payload[0..header.length-1] for message content
00712  * 4. CRC field contains the received checksum (already verified)
00713  *
00714  * @par Usage Example - Building a Frame:
00715  * @code{.c}
00716  * rx_frame_t frame;
00717  * memset(&frame, 0, sizeof(frame));
00718  *
00719  * // Set header
00720  * frame.header.sequence = 1;
00721  * frame.header.length = protobuf_len;
00722  * frame.header.type = k_frame_type_command;
00723  * frame.header.flags = k_frame_flag_requires_ack;
00724  *
00725  * // Copy payload
00726  * memcpy(frame.payload, protobuf_data, protobuf_len);
00727  *
00728  * // Encode (CRC computed internally)
00729  * uint8_t wire[1036];
00730  * uint32_t wire_len;
00731  * rx_frame_encode(&encoder, &frame, wire, &wire_len);
00732  * @endcode
00733  *
00734  * @par Usage Example - Processing Decoded Frame:
00735  * @code{.c}
00736  * rx_frame_t frame;
00737  * rx_err_t err = rx_frame_decode(&decoder, wire_data, wire_len, &frame);
00738  *
00739  * if (err == k_rx_ok) {
00740  *     // Frame valid, CRC verified
00741  *     if (frame.header.type == k_frame_type_command) {
00742  *         // Process payload[0..header.length-1]

```



```

00743 *      handle_command(frame.payload, frame.header.length);
00744 *    }
00745 *  }
00746 *  @endcode
00747 *
00748 *  @invariant header.length <= k_frame_max_payload
00749 *  @note The crc field is populated during decode, not used during encode
00750 *  @warning Payload data beyond header.length is undefined
00751 *
00752 *  @see rx_frame_header_t Header structure
00753 *  @see rx_frame_encode() Encode frame to wire format
00754 *  @see rx_frame_decode() Decode frame from wire format
00755 *  @since Version 1.0.0
00756 */
00757 typedef struct {
00758 /**
00759  * @brief Frame header containing control fields
00760  * @details Populated by caller (encode) or decoder (decode).
00761  */
00762 rx_frame_header_t header;
00763
00764 /**
00765  * @brief Payload data buffer (maximum 1024 bytes)
00766  * @details
00767  * For transmit: Contains data to send (length in header.length).
00768  * For receive: Contains decoded payload (length in header.length).
00769  * @note Only payload[0..header.length-1] is valid data.
00770  */
00771 uint8_t payload[k_frame_max_payload];
00772
00773 /**
00774  * @brief CRC-32 checksum (IEEE 802.3)
00775  * @details
00776  * For received frames: Contains the CRC read from wire (verified).
00777  * For transmitted frames: Not used (computed by encoder).
00778  * @par Wire Format: Little-endian
00779  */
00780 uint32_t crc;
00781 } rx_frame_t;
00782
00783 /**
00784  * @struct rx_frame_encoder_t
00785  * @brief Frame encoder handle
00786  *
00787  * @details
00788  * Lightweight handle structure for frame encoding operations.
00789  * Must be initialized before use with rx_frame_encoder_init().
00790  *
00791  * ## Lifecycle
00792  *
00793  * 1. Declare: `rx_frame_encoder_t enc;`
00794  * 2. Initialize: `rx_frame_encoder_init(&enc);`
00795  * 3. Use: `rx_frame_encode(&enc, &frame, output, &output_len);`
00796  * 4. Deinitialize: `rx_frame_encoder_deinit(&enc);` (optional)
00797  *
00798  * @par Memory Layout:
00799  * | Offset | Size | Field | Description |
00800  * |-----|-----|-----|-----|
00801  * | 0 | 1 | initialized | Initialization flag |
00802  * | **Total** | **1** | | No padding |
00803  *
00804  * @invariant initialized is 0 or 1
00805  * @note Structure is stateless after init; thread-safe for concurrent use
00806  *
00807  * @see rx_frame_encoder_init() Initialize encoder
00808  * @see rx_frame_encode() Encode frame
00809  * @since Version 1.0.0
00810 */
00811 typedef struct {
00812 /**
00813  * @brief Initialization state flag
00814  * @details Non-zero (1) if initialized and ready for use.
00815  * @par Valid Values: 0 (uninitialized), 1 (initialized)
00816  */
00817 uint8_t initialized;
00818 } rx_frame_encoder_t;
00819
00820 /**
00821  * @struct rx_frame_decoder_t
00822  * @brief Frame decoder handle
00823  *
00824  * @details
00825  * Lightweight handle structure for frame decoding operations.
00826  * Must be initialized before use with rx_frame_decoder_init().
00827  *
00828  * ## Lifecycle
00829  *

```



```

00830 * 1. Declare: `rx_frame_decoder_t dec;`
00831 * 2. Initialize: `rx_frame_decoder_init(&dec);`
00832 * 3. Use: `rx_frame_decode(&dec, data, len, &frame);`
00833 * 4. Deinitialize: `rx_frame_decoder_deinit(&dec);` (optional)
00834 *
00835 * @par Memory Layout:
00836 * | Offset | Size | Field | Description |
00837 * |-----|-----|-----|-----|
00838 * | 0 | 1 | initialized | Initialization flag |
00839 * | **Total** | **1** | | No padding |
00840 *
00841 * @invariant initialized is 0 or 1
00842 * @note Structure is stateless after init; thread-safe for concurrent use
00843 *
00844 * @see rx_frame_decoder_init() Initialize decoder
00845 * @see rx_frame_decode() Decode frame
00846 * @since Version 1.0.0
00847 */
00848 typedef struct {
00849     /**
00850      * @brief Initialization state flag
00851      * @details Non-zero (1) if initialized and ready for use.
00852      * @par Valid Values: 0 (uninitialized), 1 (initialized)
00853      */
00854     uint8_t initialized;
00855 } rx_frame_decoder_t;
00856
00857 /*
=====
00858 * Encoder API
00859 *
=====
00860 */
00861 /**
00862 * @brief Initialize frame encoder
00863 *
00864 * @param[out] enc Pointer to encoder handle
00865 * @return k_rx_ok on success, k_rx_err_invalid_arg if enc is nullptr
00866 */
00867 [[nodiscard]] rx_err_t rx_frame_encoder_init(rx_frame_encoder_t* enc);
00868
00869 /**
00870 * @brief Deinitialize frame encoder
00871 *
00872 * @param[in,out] enc Pointer to encoder handle
00873 * @return k_rx_ok on success
00874 */
00875 [[nodiscard]] rx_err_t rx_frame_encoder_deinit(rx_frame_encoder_t* enc);
00876
00877 /**
00878 * @brief Encode frame to wire format
00879 *
00880 * Serializes frame to wire format with SYNC, header, payload, and CRC-32.
00881 * All multi-byte fields are little-endian (SYNC, SEQ, LEN, CRC-32).
00882 *
00883 * @param[in] enc Encoder handle
00884 * @param[in] frame Frame to encode
00885 * @param[out] output Output buffer (min k_frame_min_size + payload bytes)
00886 * @param[out] output_len Actual number of bytes written
00887 *
00888 * @return k_rx_ok on success
00889 * @return k_rx_err_invalid_arg if any pointer is nullptr
00890 * @return k_rx_err_invalid_state if encoder not initialized
00891 * @return k_rx_err_invalid_size if payload exceeds max
00892 */
00893 [[nodiscard]] rx_err_t rx_frame_encode(const rx_frame_encoder_t* enc,
00894                                       const rx_frame_t* frame,
00895                                       uint8_t* output,
00896                                       uint32_t* output_len);
00897
00898 /*
=====
00900 * Utility Functions (Static Inline)
00901 *
00902 * NOTE: These are intentionally static inline in the header because:
00903 * 1. They are trivial one-liner calculations (single arithmetic operation)
00904 * 2. They may be called in performance-sensitive code paths
00905 * 3. They need to be available across multiple translation units
00906 * 4. Function call overhead would exceed the actual computation cost
00907 *
00908 * For such trivial operations, static inline is the standard C pattern and
00909 * results in better performance with no binary size increase.
00910 *
=====
00911 */
00912

```

```

00913 /**
00914  * @brief Calculate encoded frame size
00915  *
00916  * @param[in] payload_len Payload length
00917  * @return Total frame size in bytes
00918  */
00919 static inline uint32_t rx_frame_encoded_size(uint32_t payload_len)
00920 {
00921     return k_frame_sync_size + k_frame_header_size + payload_len + k_frame_crc_size;
00922 }
00923
00924 /**
=====
00925  * Byte Ordering Utilities (Static Inline)
00926  *
00927  * Little-endian read/write for wire format serialization.
00928  * Used by rx_frame, rx_usb_comm, and rx_spi_comm modules.
00929  *
00930  * Little-endian format stores the least significant byte (LSB) at the lowest
00931  * memory address. For a 16-bit value:
00932  * - buf[0] = low byte (LSB)
00933  * - buf[1] = high byte (MSB)
00934  *
00935  * Example: 0x55AA stored in little-endian:
00936  *   buf[0] = 0xAA (low byte)
00937  *   buf[1] = 0x55 (high byte)
00938  *
=====
00939  */
00940
00941 /**
00942  * @brief Byte indices for 16-bit little-endian serialization
00943  *
00944  * In little-endian format, the low byte (LSB) is stored at index 0 and the
00945  * high byte (MSB) is stored at index 1.
00946  */
00947 typedef enum : uint8_t {
00948     k_le16_byte_low = 0, /**< Low byte (LSB) at index 0 */
00949     k_le16_byte_high = 1, /**< High byte (MSB) at index 1 */
00950 } rx_le16_byte_idx_t;
00951
00952 /**
00953  * @brief Byte manipulation constants for endianness conversions
00954  */
00955 typedef enum : uint8_t {
00956     k_rx_le16_high_shift = (8), /**< Bit shift for high byte in 16-bit value */
00957     k_rx_byte_mask = (0xFFU), /**< Mask to extract one byte */
00958 } rx_byte_order_constants_t;
00959
00960 /**
00961  * @brief Byte indices for 32-bit little-endian serialization
00962  */
00963 typedef enum : uint8_t {
00964     k_le32_byte = 0, /**< Byte 0 (LSB) */
00965     k_le32_byte = 1, /**< Byte 1 */
00966     k_le32_byte = 2, /**< Byte 2 */
00967     k_le32_byte = 3, /**< Byte 3 (MSB) */
00968 } rx_le32_byte_idx_t;
00969
00970 /**
00971  * @brief Bit shift amounts for extracting bytes from 32-bit values
00972  */
00973 typedef enum : uint8_t {
00974     k_rx_le32_shift = 0, /**< Shift 0 bits for byte 0 */
00975     k_rx_le32_shift = 8, /**< Shift 8 bits for byte 1 */
00976     k_rx_le32_shift = 16, /**< Shift 16 bits for byte 2 */
00977     k_rx_le32_shift = 24, /**< Shift 24 bits for byte 3 */
00978 } rx_le32_shift_t;
00979
00980 /**
00981  * @brief Read uint16 from little-endian buffer
00982  *
00983  * Reads a 16-bit unsigned integer from a buffer in little-endian format.
00984  * The low byte (LSB) is at index 0, the high byte (MSB) is at index 1.
00985  *
00986  * @param[in] buf Input buffer (at least 2 bytes)
00987  * @return Decoded 16-bit value in host byte order
00988  *
00989  * @note This is the preferred function for parsing multi-byte protocol fields.
00990  */
00991 static inline uint16_t rx_frame_read_le16(const uint8_t* buf)
00992 {
00993     return (uint16_t)buf[k_le16_byte_low] | ((uint16_t)buf[k_le16_byte_high] « k_rx_le16_high_shift);
00994 }
00995
00996 /**
00997  * @brief Write uint16 to little-endian buffer

```

```

00998 *
00999 * Writes a 16-bit unsigned integer to a buffer in little-endian format.
01000 * The low byte (LSB) is written at index 0, the high byte (MSB) is written
01001 * at index 1.
01002 *
01003 * @param[out] buf Output buffer (at least 2 bytes)
01004 * @param[in] val Value to write in host byte order
01005 *
01006 * @note This is the preferred function for serializing multi-byte protocol
01007 * fields.
01008 */
01009 static inline void rx_frame_write_le16(uint8_t* buf, uint16_t val)
01010 {
01011     buf[k_le16_byte_low] = (uint8_t)(val & k_rx_byte_mask);
01012     buf[k_le16_byte_high] = (uint8_t)(val >> k_rx_le16_high_shift);
01013 }
01014
01015 /**
01016 * @brief Read uint32 from little-endian buffer
01017 *
01018 * Reads a 32-bit unsigned integer from a buffer in little-endian format.
01019 *
01020 * @param[in] buf Input buffer (at least 4 bytes)
01021 * @return Decoded 32-bit value in host byte order
01022 */
01023 static inline uint32_t rx_frame_read_le32(const uint8_t* buf)
01024 {
01025     return (uint32_t)buf[k_le32_byte] | ((uint32_t)buf[k_le32_byte] << k_rx_le32_shift) |
01026           ((uint32_t)buf[k_le32_byte] << k_rx_le32_shift) |
01027           ((uint32_t)buf[k_le32_byte] << k_rx_le32_shift);
01028 }
01029
01030 /*
=====
01031 * Decoder API
01032 *
=====
01033 */
01034
01035 /**
01036 * @brief Initialize frame decoder
01037 *
01038 * @param[out] dec Pointer to decoder handle
01039 * @return k_rx_ok on success, k_rx_err_invalid_arg if dec is nullptr
01040 */
01041 [[nodiscard]] rx_err_t rx_frame_decoder_init(rx_frame_decoder_t* dec);
01042
01043 /**
01044 * @brief Deinitialize frame decoder
01045 *
01046 * @param[in,out] dec Pointer to decoder handle
01047 * @return k_rx_ok on success
01048 */
01049 [[nodiscard]] rx_err_t rx_frame_decoder_deinit(rx_frame_decoder_t* dec);
01050
01051 /**
01052 * @brief Decode wire format to frame
01053 *
01054 * Parses wire format and validates SYNC word and CRC-32.
01055 *
01056 * @param[in] dec Decoder handle
01057 * @param[in] data Wire format data
01058 * @param[in] data_len Data length
01059 * @param[out] frame Decoded frame
01060 *
01061 * @return k_rx_ok on success
01062 * @return k_rx_err_invalid_arg if any pointer is nullptr
01063 * @return k_rx_err_invalid_state if decoder not initialized
01064 * @return k_rx_err_invalid_size if data too short
01065 * @return k_rx_err_protocol_error if SYNC word invalid
01066 * @return k_rx_err_crc_mismatch if CRC validation fails
01067 */
01068 [[nodiscard]] rx_err_t rx_frame_decode(const rx_frame_decoder_t* dec,
01069                                       const uint8_t* data,
01070                                       const uint32_t data_len,
01071                                       rx_frame_t* frame);
01072
01073 /**
01074 * @brief Decode wire format to frame with bounded sync-word resynchronization
01075 *
01076 * @details
01077 * Extends rx_frame_decode() with automatic stream recovery. When the sync word
01078 * is not found at offset 0 (i.e., the stream is byte-misaligned due to a
01079 * dropped or inserted byte), this function scans forward up to
01080 * k_frame_max_scan_bytes positions looking for the next sync word occurrence.
01081 * Bytes discarded during the scan are reported via bytes_discarded_out for
01082 * diagnostic logging by the caller.

```

```

01083 *
01084 * If the sync word is located within the scan window, decoding proceeds from
01085 * that offset as if data had been provided aligned. If no sync word is found
01086 * within the bounded window, k_rx_err_protocol_error is returned.
01087 *
01088 * Algorithm:
01089 * 1. Attempt normal decode at offset 0.
01090 * 2. On k_rx_err_protocol_error (sync mismatch), invoke
01091 *    internal_find_sync_offset() to locate the next sync word.
01092 * 3. If found, set *bytes_discarded_out = sync_offset -- caller must advance the
01093 *    stream read pointer by this value even if the subsequent decode fails
01094 *    (e.g., k_rx_err_crc_mismatch), to avoid infinite re-scan.
01095 * 4. Decode from the discovered offset (trimmed view of data).
01096 * 5. All other errors (CRC, size) are propagated unchanged.
01097 *
01098 * @param[in] dec          Initialized frame decoder handle
01099 * @param[in] data          Raw byte buffer (may be misaligned)
01100 * @param[in] data_len      Length of data buffer in bytes
01101 * @param[out] frame        Decoded frame (header, payload, CRC)
01102 * @param[out] bytes_discarded_out Number of bytes skipped to reach sync word
01103 *                          (0 when sync was found at offset 0)
01104 *
01105 * @return rx_err_t Error code
01106 * @retval k_rx_ok          Frame decoded and CRC verified
01107 * @retval k_rx_err_invalid_arg Any pointer parameter is nullptr
01108 * @retval k_rx_err_invalid_state Decoder not initialized
01109 * @retval k_rx_err_invalid_size Data buffer too short to contain a valid frame
01110 * @retval k_rx_err_protocol_error No sync word found within k_frame_max_scan_bytes
01111 * @retval k_rx_err_crc_mismatch Sync found but CRC validation failed
01112 *
01113 * @pre dec must be initialized via rx_frame_decoder_init()
01114 * @pre data must point to a buffer of at least data_len bytes
01115 * @pre frame must be a non-null pointer to a writable rx_frame_t
01116 * @pre bytes_discarded_out must be a non-null pointer to a writable uint32_t
01117 * @post On k_rx_ok, frame contains the decoded frame with verified CRC
01118 * @post bytes_discarded_out contains the number of bytes skipped (>= 0)
01119 * @post On k_rx_err_crc_mismatch, *bytes_discarded_out == bytes skipped; caller must still advance stream pointer
01120 *
01121 * @note Thread-safe after init; the decoder handle is read-only and the function
01122 *       uses no shared mutable state (consistent with the file-level "Stateless after init").
01123 * @note The scan window is bounded at k_frame_max_scan_bytes (NASA Rule 2).
01124 * @note Caller should log bytes_discarded_out > 0 as a diagnostic warning.
01125 *
01126 * @warning Caller must advance its read pointer by *bytes_discarded_out bytes after any
01127 *          call that returns k_rx_ok or k_rx_err_crc_mismatch to avoid infinite re-scan;
01128 *          *bytes_discarded_out is unmodified only when k_rx_err_invalid_arg is returned.
01129 *
01130 * @par Performance:
01131 * Worst case: scans k_frame_max_scan_bytes (1036) before failing.
01132 * Best case: identical to rx_frame_decode() (0 bytes discarded).
01133 *
01134 * @par Example:
01135 * @code{.c}
01136 * rx_frame_decoder_t dec;
01137 * rx_frame_decoder_init(&dec);
01138 *
01139 * rx_frame_t frame;
01140 * uint32_t discarded = 0;
01141 * rx_err_t err = rx_frame_decode_with_resync(&dec, wire_buf, wire_len,
01142 *                                           &frame, &discarded);
01143 * if (discarded > 0) {
01144 *     rx_log_warn("FRAME", "Stream resync: discarded %u bytes", discarded);
01145 * }
01146 * if (err == k_rx_ok) {
01147 *     process_frame(&frame);
01148 * }
01149 * @endcode
01150 *
01151 * @see rx_frame_decode() Simple decode without resynchronization
01152 * @see k_frame_max_scan_bytes Maximum scan window (1036 bytes)
01153 * @since Version 1.0.0
01154 */
01155 [[nodiscard]] rx_err_t rx_frame_decode_with_resync(const rx_frame_decoder_t* dec,
01156                                                    const uint8_t* data,
01157                                                    const uint32_t data_len,
01158                                                    rx_frame_t* frame,
01159                                                    uint32_t* bytes_discarded_out);
01160
01161 /*
=====
01162 * Frame Helper Functions
01163 *
=====
01164 */
01165 /**
01166 * @brief Create ACK frame for given sequence

```

```

01168 *
01169 * @param[out] frame Frame to initialize
01170 * @param[in] sequence Sequence number to acknowledge
01171 * @return k_rx_ok on success
01172 */
01173 [[nodiscard]] rx_err_t rx_frame_create_ack(rx_frame_t* frame, uint16_t sequence);
01174
01175 /**
01176 * @brief Create NACK frame for given sequence
01177 *
01178 * @param[out] frame Frame to initialize
01179 * @param[in] sequence Sequence number
01180 * @param[in] flags Additional flags (e.g., k_frame_flag_soft_nack)
01181 * @return k_rx_ok on success
01182 */
01183 [[nodiscard]] rx_err_t rx_frame_create_nack(rx_frame_t* frame, uint16_t sequence, uint8_t flags);
01184
01185 /**
01186 * @brief Create PING frame for keepalive/heartbeat
01187 *
01188 * @param[out] frame Frame to initialize
01189 * @param[in] sequence Sequence number
01190 * @param[in] payload Optional payload to include (may be NULL if payload_len is 0)
01191 * @param[in] payload_len Payload length in bytes
01192 * @return k_rx_ok on success
01193 */
01194 [[nodiscard]] rx_err_t rx_frame_create_ping(rx_frame_t* frame,
01195                                             const uint16_t sequence,
01196                                             const uint8_t* payload,
01197                                             const uint32_t payload_len);
01198
01199 /**
01200 * @brief Create PONG frame (response to PING)
01201 *
01202 * @param[out] frame Frame to initialize
01203 * @param[in] sequence Sequence number (matches ping)
01204 * @param[in] payload Optional payload to include (may be NULL if payload_len is 0)
01205 * @param[in] payload_len Payload length in bytes
01206 * @return k_rx_ok on success
01207 */
01208 [[nodiscard]] rx_err_t rx_frame_create_pong(rx_frame_t* frame,
01209                                             const uint16_t sequence,
01210                                             const uint8_t* payload,
01211                                             const uint32_t payload_len);
01212
01213 /**
01214 * @brief Create RESET frame to reset communication state
01215 *
01216 * @param[out] frame Frame to initialize
01217 * @param[in] sequence Sequence number
01218 * @return k_rx_ok on success
01219 */
01220 [[nodiscard]] rx_err_t rx_frame_create_reset(rx_frame_t* frame, const uint16_t sequence);
01221
01222 /**
01223 * @brief Create RESET_ACK frame (response to RESET)
01224 *
01225 * @param[out] frame Frame to initialize
01226 * @param[in] sequence Sequence number (matches reset)
01227 * @return k_rx_ok on success
01228 */
01229 [[nodiscard]] rx_err_t rx_frame_create_reset_ack(rx_frame_t* frame, const uint16_t sequence);
01230
01231 /**
01232 * @brief Check if frame type is valid
01233 *
01234 * Static inline for same reasons as rx_frame_encoded_size() above.
01235 *
01236 * @param[in] type Frame type to check
01237 * @return true if valid, false otherwise
01238 */
01239 static inline bool rx_frame_type_valid(uint8_t type)
01240 {
01241     switch (type) {
01242         case k_frame_type_ping:
01243         case k_frame_type_pong:
01244         case k_frame_type_command:
01245         case k_frame_type_response:
01246         case k_frame_type_ack:
01247         case k_frame_type_nack:
01248         case k_frame_type_reset_ack:
01249         case k_frame_type_reset:
01250             return true;
01251         default:
01252             return false;
01253     }
01254 }

```

```

01255
01256 #ifdef __cplusplus
01257 }
01258 #endif

```

4.3 libs/rx_frame/src/rx_frame.c File Reference

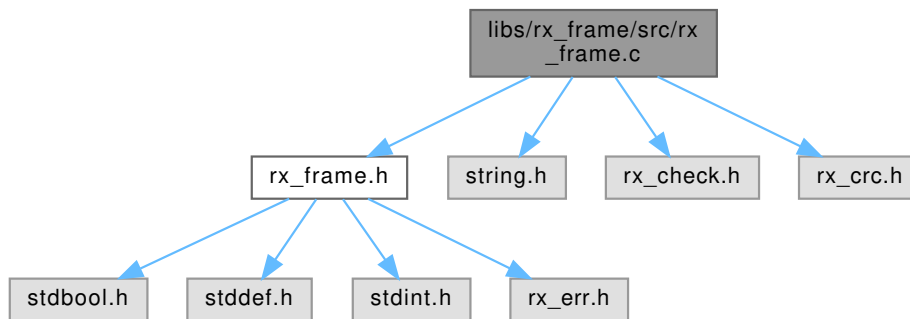
Frame Layer Protocol Implementation.

```

#include "rx_frame.h"
#include <string.h>
#include "rx_check.h"
#include "rx_crc.h"

```

Include dependency graph for rx_frame.c:



Enumerations

- enum `byte_shift_t`: `uint8_t { k_shift_byte7 = 0, k_shift_byte8 = 8, k_shift_byte9 = 16, k_shift_byte1 = 24 }`
Bit shift amounts for extracting bytes from multi-byte values.
- enum `init_state_t`: `uint8_t { k_state_uninitialized = 0, k_state_initialized = 1 }`
Initialization state values.
- enum `frame_offset_t`: `uint8_t { k_frame_offset_start = 0, k_frame_scan_start_offset = 1 }`
Frame byte offsets used during encoding, decoding, and sync scanning.
- enum `frame_resync_discarded_t`: `uint32_t { k_bytes_discarded_none = 0U }`
Byte-discard counter initial value for `rx_frame_decode_with_resync()`.

Functions

- static `rx_err_t internal_write_le32` (`uint8_t *buf, const uint32_t buf_len, const uint32_t val`)
Write uint32 in little-endian format (for CRC-32).
- static `rx_err_t internal_read_le32` (`const uint8_t *buf, const uint32_t buf_len, uint32_t *out_val`)
Read uint32 in little-endian format (for CRC-32).
- static `rx_err_t internal_decode_header` (`const uint8_t *data, const uint32_t data_len, rx_frame_t *frame, uint32_t *offset_out`)
Decode frame header from raw data buffer.
- static `rx_err_t internal_verify_crc` (`const uint8_t *data, uint32_t data_len, uint32_t offset, uint32_t *crc_out`)
Verify CRC-32 checksum at specified offset in data buffer.
- static `rx_err_t internal_find_sync_offset` (`const uint8_t *data, const uint32_t data_len, uint32_t *offset_out`)
Scan a byte buffer for the frame sync word with a fixed upper bound.

- `rx_err_t rx_frame_encoder_init(rx_frame_encoder_t *enc)`
Initialize a frame encoder instance.
- `rx_err_t rx_frame_encoder_deinit(rx_frame_encoder_t *enc)`
Deinitialize a frame encoder instance.
- `rx_err_t rx_frame_encode(const rx_frame_encoder_t *enc, const rx_frame_t *frame, uint8_t *output, uint32_t *output_len)`
Encode frame to wire format.
- `rx_err_t rx_frame_decoder_init(rx_frame_decoder_t *dec)`
Initialize a frame decoder instance.
- `rx_err_t rx_frame_decoder_deinit(rx_frame_decoder_t *dec)`
Deinitialize a frame decoder instance.
- `rx_err_t rx_frame_decode(const rx_frame_decoder_t *dec, const uint8_t *data, const uint32_t data_len, rx_frame_t *frame)`
Decode a frame from raw data.
- `rx_err_t rx_frame_decode_with_resync(const rx_frame_decoder_t *dec, const uint8_t *data, const uint32_t data_len, rx_frame_t *frame, uint32_t *bytes_discarded_out)`
Decode a frame from raw data with bounded sync-word resynchronization.
- `rx_err_t rx_frame_create_ack(rx_frame_t *frame, const uint16_t sequence)`
Create an ACK (acknowledgment) frame.
- `rx_err_t rx_frame_create_nack(rx_frame_t *frame, const uint16_t sequence, uint8_t flags)`
Create a NACK (negative acknowledgment) frame.
- `rx_err_t rx_frame_create_ping(rx_frame_t *frame, const uint16_t sequence, const uint8_t *payload, const uint32_t payload_len)`
Create a PING keepalive frame.
- `rx_err_t rx_frame_create_pong(rx_frame_t *frame, const uint16_t sequence, const uint8_t *payload, const uint32_t payload_len)`
Create a PONG response frame.
- `rx_err_t rx_frame_create_reset(rx_frame_t *frame, const uint16_t sequence)`
Create a RESET request frame.
- `rx_err_t rx_frame_create_reset_ack(rx_frame_t *frame, const uint16_t sequence)`
Create a RESET_ACK confirmation frame.

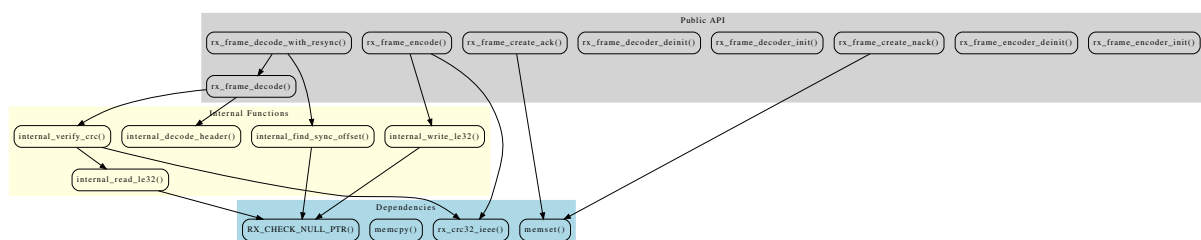
4.3.1 Detailed Description

Frame Layer Protocol Implementation.

4.3.1.1 Overview

This file implements the frame encoding and decoding operations for the SPI communication protocol between RPi5 and RX72N. The implementation is bit-exact compatible with the Go reference in `star-gateway/internal/frame/`.

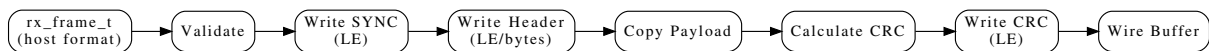
4.3.1.2 Module Architecture



4.3.1.3 Encoding Process

The encoder serializes a `rx_frame_t` structure to wire format:

1. **Validate inputs** - Check pointers, initialization state, payload size
2. **Write SYNC** - 2-byte marker (0x55AA) in little-endian (wire: 0xAA, 0x55)
3. **Write header** - SEQ, LEN in little-endian; TYPE, FLAGS as bytes
4. **Copy payload** - Direct byte copy (no endian conversion)
5. **Compute CRC** - IEEE 802.3 CRC-32 over SYNC+header+payload
6. **Write CRC** - 4-byte CRC in little-endian format



4.3.1.4 Decoding Process

The decoder parses wire format into a `rx_frame_t` structure:

1. **Validate inputs** - Check pointers, initialization state
2. **Verify SYNC** - Check for 0x55AA marker
3. **Parse header** - Extract SEQ, LEN, TYPE, FLAGS with endian conversion
4. **Validate length** - Ensure payload fits in buffer
5. **Copy payload** - Direct byte copy
6. **Verify CRC** - Compute expected CRC, compare with received

4.3.1.5 Memory Usage

Function	Stack	Notes
<code>rx_frame_encode()</code>	32 bytes	Variables, return address
<code>rx_frame_decode()</code>	32 bytes	Variables, return address
<code>internal_decode_header()</code>	24 bytes	Local variables
<code>internal_verify_crc()</code>	20 bytes	CRC values

4.3.1.6 Performance (RX72N @ 240 MHz)

Operation	Empty	64B	256B	1KB
Encode	5 us	12 us	35 us	130 us
Decode	5 us	12 us	35 us	130 us

Bottleneck: CRC calculation dominates for larger payloads. Hardware CRC reduces time by $\sim 4\times$.

4.3.1.7 NASA Power of 10 Compliance

Rule	Status	Notes
1. Simple control flow	[PASS]	Sequential processing, no goto

Rule	Status	Notes
2. Fixed loop bounds	[PASS]	Payload loop bounded by payload length; sync scanner bounded by k_frame_max_scan_bytes
3. No dynamic memory	[PASS]	All buffers caller-provided
4. Short functions	[PASS]	Largest function ~50 lines
5. Assertions	[PASS]	All inputs validated, post-conditions checked
6. Small scope	[PASS]	Variables at point of use
7. Check returns	[PASS]	All errors propagated
8. Limited preprocessor	[PASS]	Only includes
9. Restrict pointers	[PASS]	No function pointers
10. Compiler warnings	[PASS]	Clean build with -Wall -Wextra -Werror

4.3.1.8 SOLID Principles

Principle	Implementation
Single Responsibility	Encode/decode frames only
Open/Closed	New types/flags via enums
Liskov Substitution	N/A (no inheritance)
Interface Segregation	Separate encoder/decoder
Dependency Inversion	Depends on rx_crc abstraction

4.3.1.9 Test Coverage

See tests/test_rx_frame.c for comprehensive unit tests covering:

- Encode/decode round-trip
- CRC validation
- Boundary conditions (min/max payload)
- Error cases (NULL pointers, invalid sizes)

See also

[rx_frame.h](#) Public API and frame format documentation
[rx_crc.h](#) CRC-32 computation
[star-gateway/internal/frame/](#) Go reference implementation

Author

STAR Team

Date

2026-01-29

Copyright

Copyright (c) 2026 STAR Project. MIT License.

Since

Version 1.0.0

Definition in file [rx_frame.c](#).

4.3.2 Enumeration Type Documentation

4.3.2.1 byte_shift_t

enum [byte_shift_t](#) : uint8_t

Bit shift amounts for extracting bytes from multi-byte values.

Each byte position requires shifting by (position * 8) bits.

Enumerator

k_shift_byteζ	No shift needed for byte 0 (LSB)
k_shift_byten	Shift 8 bits for byte 1
k_shift_byteθ	Shift 16 bits for byte 2
k_shift_byteι	Shift 24 bits for byte 3 (MSB)

Definition at line 196 of file [rx_frame.c](#).

```
00196      : uint8_t {
00197  k_shift_byte = 0, /**< No shift needed for byte 0 (LSB) */
00198  k_shift_byte = 8, /**< Shift 8 bits for byte 1 */
00199  k_shift_byte = 16, /**< Shift 16 bits for byte 2 */
00200  k_shift_byte = 24, /**< Shift 24 bits for byte 3 (MSB) */
00201 } byte_shift_t;
```

4.3.2.2 frame_offset_t

enum [frame_offset_t](#) : uint8_t

Frame byte offsets used during encoding, decoding, and sync scanning.

Defines named byte-offset constants used by the frame encoder, decoder, and the bounded sync-word scanner. k_frame_offset_start marks the beginning of a frame buffer for normal aligned decoding. k_frame_scan_start_offset is the index at which [internal_find_sync_offset\(\)](#) begins its forward scan – offset 0 is omitted because [rx_frame_decode\(\)](#) has already tested it before invoking the scanner, so the scan starts at the next candidate position.

Invariant

`k_frame_scan_start_offset > k_frame_offset_start`, ensuring the scanner never re-tests the byte that `rx_frame_decode()` already checked.

```
// Normal aligned decode starts at k_frame_offset_start (0)
rx_err_t err = rx_frame_decode(dec, data, data_len, frame);

// On sync mismatch, internal_find_sync_offset() scans from offset 1:
uint8_t sync_off = k_frame_scan_start_offset;
// ... scanner advances sync_off until sync word found or scan limit reached
```

See also

`rx_frame_decode()` Uses `k_frame_offset_start` for normal aligned decoding
`internal_find_sync_offset()` Begins scan at `k_frame_scan_start_offset`

Since

Version 1.0.0

Enumerator

<code>k_frame_offset_start</code>	Start offset for frame parsing
<code>k_frame_scan_start_offset</code>	First offset checked in <code>internal_find_sync_offset()</code> ; offset 0 is presumed already tested by <code>rx_frame_decode()</code>

Definition at line 240 of file `rx_frame.c`.

```
00240     : uint8_t {
00241 k_frame_offset_start    = 0, /**< Start offset for frame parsing */
00242 k_frame_scan_start_offset = 1, /**< First offset checked in internal_find_sync_offset();
00243                                offset 0 is presumed already tested by rx_frame_decode() */
00244 } frame_offset_t;
```

4.3.2.3 frame_resync_discarded_t

enum `frame_resync_discarded_t` : `uint32_t`

Byte-discard counter initial value for `rx_frame_decode_with_resync()`.

`k_bytes_discarded_none` is the zero sentinel written to `*bytes_discarded_out` at the start of `rx_frame_decode_with_resync()` before any sync search. Using a named constant makes the intent auditable and prevents a raw 0 literal from appearing in the implementation (NASA Rule 8 / STAR no-magic-numbers policy).

Invariant

`k_bytes_discarded_none == 0` (matches the initial no-discard state)

```
uint32_t bytes_discarded = k_bytes_discarded_none;
rx_err_t err = rx_frame_decode_with_resync(&dec, data, data_len, &frame, &bytes_discarded);
```

See also

`rx_frame_decode_with_resync()` Only caller of this constant

Since

Version 1.0.0

Enumerator

<code>k_bytes_discarded_none</code>	No bytes have been discarded; stream is aligned
-------------------------------------	---

Definition at line 267 of file `rx_frame.c`.

```
00267     : uint32_t {
00268 k_bytes_discarded_none = 0U, /**< No bytes have been discarded; stream is aligned */
00269 } frame_resync_discarded_t;
```

4.3.2.4 init_state_t

enum [init_state_t](#) : uint8_t

Initialization state values.

Enumerator

k_state_uninitialized	Object not initialized
k_state_initialized	Object initialized and ready

Definition at line 206 of file [rx_frame.c](#).

```
00206      : uint8_t {
00207  k_state_uninitialized = 0, /**< Object not initialized */
00208  k_state_initialized  = 1, /**< Object initialized and ready */
00209 } init_state_t;
```

4.3.3 Function Documentation

4.3.3.1 internal_decode_header()

```
rx_err_t internal_decode_header (
    const uint8_t * data,
    const uint32_t data_len,
    rx\_frame\_t * frame,
    uint32_t * offset_out) [static]
```

Decode frame header from raw data buffer.

Extracts and validates frame header fields (sync word, sequence, length, type, flags) from the beginning of a raw data buffer. Verifies the sync word matches expected value and validates payload length is within bounds. Outputs the offset where payload data begins for subsequent processing.

Parameters

in	<i>data</i>	Input data buffer containing encoded frame
in	<i>data_len</i>	Length of input buffer in bytes
out	<i>frame</i>	Frame structure to populate with decoded header fields
out	<i>offset_out</i>	Offset in buffer where payload begins (after header and sync)

Returns

k_rx_ok on successful header decode

Return values

<i>k_rx_err_invalid_arg</i>	if data, frame, or offset_out is nullptr
<i>k_rx_err_invalid_size</i>	if data_len is too small or decoded payload length is out of bounds
<i>k_rx_err_protocol_error</i>	if sync word does not match expected value (k_frame_sync_word)

Definition at line 344 of file [rx_frame.c](#).

```

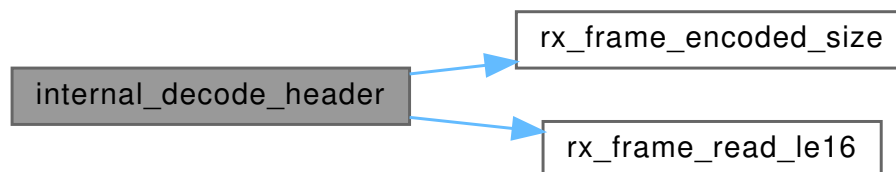
00348 {
00349     if (data == nullptr || frame == nullptr || offset_out == nullptr) {
00350         return k_rx_err_invalid_arg;
00351     }
00352     if (data_len < k_frame_min_size) {
00353         return k_rx_err_invalid_size;
00354     }
00355     uint32_t offset = k_frame_offset_start;
00356     uint16_t sync_word = rx_frame_read_le16(&data[offset]);
00357     if (sync_word != k_frame_sync_word) {
00358         return k_rx_err_protocol_error;
00359     }
00360     offset += k_frame_sync_size;
00361     frame->header.sequence = rx_frame_read_le16(&data[offset]);
00362     offset += k_frame_seq_size;
00363     frame->header.length = rx_frame_read_le16(&data[offset]);
00364     offset += k_frame_len_size;
00365     if (frame->header.length > k_frame_max_payload) {
00366         return k_rx_err_invalid_size;
00367     }
00368     uint32_t expected_size = rx_frame_encoded_size(frame->header.length);
00369     if (data_len < expected_size) {
00370         return k_rx_err_invalid_size;
00371     }
00372     frame->header.type = data[offset];
00373     offset += k_frame_type_size;
00374     frame->header.flags = data[offset];
00375     offset += k_frame_flags_size;
00376     if (offset_out != nullptr) {
00377         *offset_out = offset;
00378     }
00379     return k_rx_ok;
00380 }

```

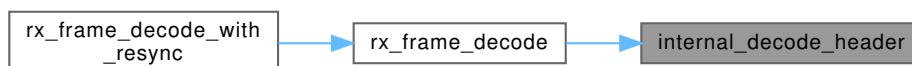
References [rx_frame_header_t::flags](#), [rx_frame_t::header](#), [k_frame_flags_size](#), [k_frame_len_size](#), [k_frame_max_payload](#), [k_frame_min_size](#), [k_frame_offset_start](#), [k_frame_seq_size](#), [k_frame_sync_size](#), [k_frame_sync_word](#), [k_frame_type_size](#), [rx_frame_header_t::length](#), [rx_frame_encoded_size\(\)](#), [rx_frame_read_le16\(\)](#), [rx_frame_header_t::sequence](#), and [rx_frame_header_t::type](#).

Referenced by [rx_frame_decode\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.2 internal_find_sync_offset()

```
rx_err_t internal_find_sync_offset (
    const uint8_t * data,
    const uint32_t data_len,
    uint32_t * offset_out) [static]
```

Scan a byte buffer for the frame sync word with a fixed upper bound.

Performs a linear scan of the buffer starting at offset 1 (offset 0 is presumed to have already failed sync check) looking for the 2-byte sync pattern `k_frame_sync_word` in little-endian wire encoding (first byte 0xAA, second byte 0x55). The scan is bounded at `k_frame_max_scan_bytes` to satisfy NASA Rule 2 (fixed loop upper-bounds) and prevent infinite consumption of a permanently corrupt stream.

The function stops and reports success at the first occurrence of the sync pattern. If no pattern is found within the window, `k_rx_err_protocol_error` is returned.

Algorithm:

1. Validate preconditions (non-null pointers, minimum buffer size).
2. Compute scan limit = min(`data_len - k_frame_sync_size`, `k_frame_max_scan_bytes`).
3. Iterate `i` from 1 to scan_limit inclusive: a. Read 16-bit LE value at `data[i]`. b. If it matches `k_frame_sync_word`, write `i` to `offset_out` and return `k_rx_ok`.
4. Return `k_rx_err_protocol_error` if no match found.

Parameters

in	<i>data</i>	Buffer to scan (at least 2 bytes)
in	<i>data_len</i>	Length of buffer in bytes (must be ≥ 2)
out	<i>offset_out</i>	Byte offset of the first sync word found (≥ 1)

Returns

`rx_err_t` Error code

Return values

<i>k_rx_ok</i>	Sync word found; offset written to <code>offset_out</code>
<i>k_rx_err_invalid_arg</i>	<code>data</code> or <code>offset_out</code> is nullptr
<i>k_rx_err_invalid_size</i>	<code>data_len</code> < 2 (cannot contain a sync word)
<i>k_rx_err_protocol_error</i>	No sync word found within <code>k_frame_max_scan_bytes</code>

Precondition

`data` must point to a readable buffer of at least `data_len` bytes

`data_len` must be $\geq k_frame_sync_size$ (2)

Postcondition

On `k_rx_ok`, `offset_out` is in range `[1, min(data_len - k_frame_sync_size, k_frame_max_scan_bytes)]`

On error, `offset_out` is unchanged

Note

Not thread-safe. Caller must ensure exclusive access to data buffer.

The loop bound is `min(data_len - k_frame_sync_size, k_frame_max_scan_bytes)`, which is statically bounded by `k_frame_max_scan_bytes = 1036` (NASA Rule 2).

Performance:

$O(N)$ where $N \leq k_frame_max_scan_bytes$. Each iteration performs one 16-bit LE read and one comparison.
Worst case: 1036 iterations (no sync found).

Example:

```
// Buffer where offset k_frame_offset_start failed sync; sync is at index 1
uint8_t data[] = {0x00, 0xAA, 0x55, 0x01, 0x00};
uint32_t offset = k_frame_offset_start;
rx_err_t err = internal_find_sync_offset(data, sizeof(data), &offset);
if (err == k_rx_ok) {
    // offset == 1: decode from data[offset]
} else {
    // k_rx_err_protocol_error: no sync found within scan window
}
```

See also

[rx_frame_decode_with_resync\(\)](#) Public API that calls this function

[k_frame_sync_word](#) Sync word value (0x55AA)

[k_frame_max_scan_bytes](#) Maximum scan window (1036 bytes)

Since

Version 1.0.0

Definition at line 503 of file [rx_frame.c](#).

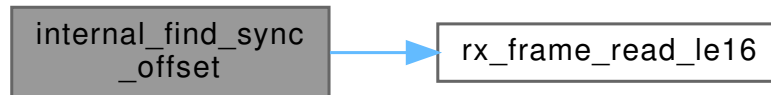
```
00504 {
00505     if (data == nullptr || offset_out == nullptr) {
00506         return k_rx_err_invalid_arg;
00507     }
00508
00509     if (data_len < k_frame_sync_size) {
00510         return k_rx_err_invalid_size;
00511     }
00512
00513     /* Bound scan to at most k_frame_max_scan_bytes positions beyond offset 0 */
00514     uint32_t scan_limit = data_len - k_frame_sync_size;
00515     if (scan_limit > k_frame_max_scan_bytes) {
00516         scan_limit = k_frame_max_scan_bytes;
00517     }
00518
00519     /* Scan starting at k_frame_scan_start_offset; offset 0 already failed sync check */
00520     for (uint32_t i = k_frame_scan_start_offset; i <= scan_limit; i++) {
00521         const uint16_t candidate = rx_frame_read_le16(&data[i]);
00522         if (candidate == k_frame_sync_word) {
00523             *offset_out = i;
00524             return k_rx_ok;
00525         }
00526     }
00527
00528     return k_rx_err_protocol_error;
}
```

```
00529 }
```

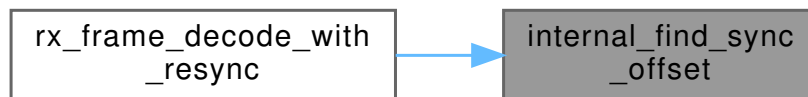
References [k_frame_max_scan_bytes](#), [k_frame_scan_start_offset](#), [k_frame_sync_size](#), [k_frame_sync_word](#), and [rx_frame_read_le16\(\)](#).

Referenced by [rx_frame_decode_with_resync\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.3 internal_read_le32()

```

rx_err_t internal_read_le32 (
    const uint8_t * buf,
    const uint32_t buf_len,
    uint32_t * out_val) [static]
  
```

Read uint32 in little-endian format (for CRC-32).

Parameters

in	<i>buf</i>	Input buffer (at least 4 bytes)
in	<i>buf_len</i>	Size of input buffer in bytes (must be ≥ 4)
out	<i>out_val</i>	Pointer to store decoded value

Returns

k_rx_ok on success

Return values

<i>k_rx_err_invalid_arg</i>	if <i>buf</i> or <i>out_val</i> is nullptr
<i>k_rx_err_invalid_size</i>	if <i>buf_len</i> < 4

Definition at line 312 of file [rx_frame.c](#).

```

00313 {
00314     RX_CHECK_NULL_PTR(buf, "FRAME", "LE32 read buffer is nullptr");
00315     RX_CHECK_NULL_PTR(out_val, "FRAME", "LE32 output pointer is nullptr");
00316     if (buf_len < k_frame_crc_size) {
00317         return k_rx_err_invalid_size;
  
```



```

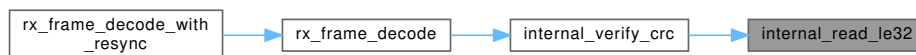
00318 }
00319
00320 *out_val = (uint32_t)buf[k_le32_byte] | ((uint32_t)buf[k_le32_byte] « k_shift_byte) |
00321 ((uint32_t)buf[k_le32_byte] « k_shift_byte) |
00322 ((uint32_t)buf[k_le32_byte] « k_shift_byte);
00323 return k_rx_ok;
00324 }

```

References [k_frame_crc_size](#), [k_le32_byteζ](#), [k_le32_byten](#), [k_le32_byteθ](#), [k_le32_byteι](#), [k_shift_byten](#), [k_shift_byteθ](#), and [k_shift_byteι](#).

Referenced by [internal_verify_crc\(\)](#).

Here is the caller graph for this function:



4.3.3.4 internal_verify_crc()

```

rx_err_t internal_verify_crc (
    const uint8_t * data,
    uint32_t data_len,
    uint32_t offset,
    uint32_t * crc_out) [static]

```

Verify CRC-32 checksum at specified offset in data buffer.

Reads the CRC-32 value stored at the specified offset in the buffer, calculates the expected CRC over the data preceding it, and compares for match. Uses IEEE 802.3 CRC-32 polynomial (0x04C11DB7), compatible with Go's `crc32.ChecksumIEEE()`. On success, outputs the received CRC value for caller inspection.

Parameters

in	<i>data</i>	Input data buffer containing payload followed by CRC
in	<i>data_len</i>	Total length of input buffer in bytes
in	<i>offset</i>	Byte offset in buffer where CRC value is stored
out	<i>crc_out</i>	Pointer to store received CRC value from buffer

Returns

`k_rx_ok` on successful CRC verification (received matches calculated)

Return values

<i>k_rx_err_invalid_arg</i>	if data or crc_out is nullptr
<i>k_rx_err_invalid_size</i>	if offset + CRC size exceeds buffer length or data_len too small
<i>k_rx_err_crc_mismatch</i>	if calculated CRC does not match received CRC value

Definition at line 411 of file [rx_frame.c](#).

```

00412 {
00413     if (data == nullptr || crc_out == nullptr) {

```

```

00414     return k_rx_err_invalid_arg;
00415 }
00416
00417 if (data_len < k_frame_min_size || (offset + k_frame_crc_size) > data_len) {
00418     return k_rx_err_invalid_size;
00419 }
00420
00421 uint32_t received_crc = 0;
00422 rx_err_t err = internal_read_le32(&data[offset], data_len - offset, &received_crc);
00423 if (err != k_rx_ok) {
00424     return err;
00425 }
00426 uint32_t calculated_crc = rx_crc32_ieee(data, offset);
00427
00428 if (received_crc != calculated_crc) {
00429     return k_rx_err_crc_mismatch;
00430 }
00431
00432 *crc_out = received_crc;
00433
00434 return k_rx_ok;
00435 }

```

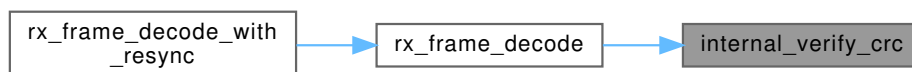
References [internal_read_le32\(\)](#), [k_frame_crc_size](#), and [k_frame_min_size](#).

Referenced by [rx_frame_decode\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.5 internal_write_le32()

```

rx_err_t internal_write_le32 (
    uint8_t * buf,
    const uint32_t buf_len,
    const uint32_t val) [static]

```

Write uint32 in little-endian format (for CRC-32).

Parameters

out	<i>buf</i>	Output buffer (at least 4 bytes)
in	<i>buf_len</i>	Size of output buffer in bytes (must be ≥ 4)
in	<i>val</i>	Value to write

Returns

`k_rx_ok` on success

Return values

<code>k_rx_err_invalid_arg</code>	if <code>buf</code> is nullptr
-----------------------------------	--------------------------------

<code>k_rx_err_invalid_size</code>	if buf_len < 4
------------------------------------	----------------

Definition at line 287 of file [rx_frame.c](#).

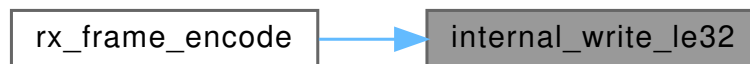
```

00288 {
00289     RX_CHECK_NULL_PTR(buf, "FRAME", "LE32 write buffer is nullptr");
00290     if (buf_len < k_frame_crc_size) {
00291         return k_rx_err_invalid_size;
00292     }
00293     buf[k_le32_byte] = (uint8_t)(val & k_rx_byte_mask);
00294     buf[k_le32_byte] = (uint8_t)((val » k_shift_byte) & k_rx_byte_mask);
00295     buf[k_le32_byte] = (uint8_t)((val » k_shift_byte) & k_rx_byte_mask);
00296     buf[k_le32_byte] = (uint8_t)((val » k_shift_byte) & k_rx_byte_mask);
00297     buf[k_le32_byte] = (uint8_t)((val » k_shift_byte) & k_rx_byte_mask);
00298     return k_rx_ok;
00299 }
```

References [k_frame_crc_size](#), [k_le32_byte7](#), [k_le32_byte8](#), [k_le32_byte9](#), [k_le32_byte1](#), [k_rx_byte_mask](#), [k_shift_byte8](#), [k_shift_byte9](#), and [k_shift_byte1](#).

Referenced by [rx_frame_encode\(\)](#).

Here is the caller graph for this function:



4.3.3.6 rx_frame_create_ack()

```

rx_err_t rx_frame_create_ack (
    rx_frame_t * frame,
    const uint16_t sequence) [nodiscard]
```

Create an ACK (acknowledgment) frame.

Create ACK frame for given sequence.

Constructs a frame with type=ACK, empty payload, and the specified sequence number. The ACK response frames are used by the receiver to confirm successful reception of a command or data frame from the sender.

Parameters

out	<i>frame</i>	Frame structure to populate with ACK
in	<i>sequence</i>	Sequence number of the frame being acknowledged

Returns

`k_rx_ok` on successful frame creation

Return values

<code>k_rx_err_invalid_arg</code>	if frame is nullptr
-----------------------------------	---------------------

<i>k_rx_err_validation_failed</i>	if ACK type verification fails
-----------------------------------	--------------------------------

Definition at line 892 of file [rx_frame.c](#).

```

00893 {
00894     if (frame == nullptr) {
00895         return k_rx_err_invalid_arg;
00896     }
00897
00898     *frame          = (rx_frame_t){0};
00899     frame->header.sequence = sequence;
00900     frame->header.length  = 0;
00901     frame->header.type    = k_frame_type_ack;
00902     frame->header.flags   = k_frame_flag_none;
00903
00904     if (frame->header.type != k_frame_type_ack) {
00905         return k_rx_err_validation_failed;
00906     }
00907
00908     return k_rx_ok;
00909 }

```

References [rx_frame_header_t::flags](#), [rx_frame_t::header](#), [k_frame_flag_none](#), [k_frame_type_ack](#), [rx_frame_header_t::length](#), [rx_frame_header_t::sequence](#), and [rx_frame_header_t::type](#).

4.3.3.7 rx_frame_create_nack()

```

rx_err_t rx_frame_create_nack (
    rx_frame_t * frame,
    const uint16_t sequence,
    uint8_t flags) [nodiscard]

```

Create a NACK (negative acknowledgment) frame.

Create NACK frame for given sequence.

Constructs a frame with type=NACK, empty payload, the specified sequence number, and error/status flags. NACK frames are sent by the receiver to indicate that a frame was not processed successfully due to an error condition (specified in flags).

Parameters

out	<i>frame</i>	Frame structure to populate with NACK
in	<i>sequence</i>	Sequence number of the frame being negative-acknowledged
in	<i>flags</i>	Status/error flags indicating reason for NACK

Returns

k_rx_ok on successful frame creation

Return values

<i>k_rx_err_invalid_arg</i>	if frame is nullptr
<i>k_rx_err_validation_failed</i>	if NACK type verification fails

Definition at line 926 of file [rx_frame.c](#).

```

00927 {
00928     if (frame == nullptr) {
00929         return k_rx_err_invalid_arg;
00930     }
00931
00932     *frame = (rx_frame_t){0};
00933     frame->header.sequence = sequence;
00934     frame->header.length = 0;
00935     frame->header.type = k_frame_type_nack;
00936     frame->header.flags = flags;
00937
00938     if (frame->header.type != k_frame_type_nack) {
00939         return k_rx_err_validation_failed;
00940     }
00941
00942     return k_rx_ok;
00943 }

```

References [rx_frame_header_t::flags](#), [rx_frame_t::header](#), [k_frame_type_nack](#), [rx_frame_header_t::length](#), [rx_frame_header_t::sequence](#), and [rx_frame_header_t::type](#).

4.3.3.8 rx_frame_create_ping()

```

rx_err_t rx_frame_create_ping (
    rx_frame_t * frame,
    const uint16_t sequence,
    const uint8_t * payload,
    const uint32_t payload_len) [nodiscard]

```

Create a PING keepalive frame.

Create PING frame for keepalive/heartbeat.

Initializes frame as a PING request for connection liveness checks. Receiver should respond with a PONG frame echoing the payload. Payload is optional; a 4-byte LE counter is typical.

Parameters

out	<i>frame</i>	Frame to populate (zeroed then filled)
in	<i>sequence</i>	Sequence number for this frame
in	<i>payload</i>	Optional payload (NULL when payload_len is 0)
in	<i>payload_len</i>	Payload length in bytes [0, k_frame_max_payload]

Return values

<i>k_rx_ok</i>	Frame created successfully
<i>k_rx_err_invalid_arg</i>	frame is NULL, or payload NULL with len > 0
<i>k_rx_err_invalid_size</i>	payload_len exceeds k_frame_max_payload
<i>k_rx_err_validation_failed</i>	Post-condition type check failed

Precondition

frame != nullptr

payload != NULL when payload_len > 0

Postcondition

```
frame->header.type == k_frame_type_ping
frame->header.length == payload_len
```

Note

Stateless; safe to call from any context.

See also

[rx_frame_create_pong\(\)](#) Corresponding response creator

Since

Version 1.0.0

Definition at line 972 of file [rx_frame.c](#).

```
00976 {
00977     if (frame == nullptr) {
00978         return k_rx_err_invalid_arg;
00979     }
00980
00981     if (payload_len > 0 && payload == nullptr) {
00982         return k_rx_err_invalid_arg;
00983     }
00984
00985     if (payload_len > k_frame_max_payload) {
00986         return k_rx_err_invalid_size;
00987     }
00988
00989     *frame = (rx_frame_t){0};
00990     frame->header.sequence = sequence;
00991     frame->header.length = (uint16_t)payload_len;
00992     frame->header.type = k_frame_type_ping;
00993     frame->header.flags = k_frame_flag_none;
00994
00995     if (payload_len > 0) {
00996         memcpy(frame->payload, payload, payload_len);
00997     }
00998
00999     if (frame->header.type != k_frame_type_ping) {
01000         return k_rx_err_validation_failed;
01001     }
01002
01003     return k_rx_ok;
01004 }
```

References [rx_frame_header_t::flags](#), [rx_frame_t::header](#), [k_frame_flag_none](#), [k_frame_max_payload](#), [k_frame_type_ping](#), [rx_frame_header_t::length](#), [rx_frame_t::payload](#), [rx_frame_header_t::sequence](#), and [rx_frame_header_t::type](#).

4.3.3.9 rx_frame_create_pong()

```
rx_err_t rx_frame_create_pong (
    rx_frame_t * frame,
    const uint16_t sequence,
    const uint8_t * payload,
    const uint32_t payload_len) [nodiscard]
```

Create a PONG response frame.

Create PONG frame (response to PING).

Initializes frame as a PONG response to a received PING. Echoes the PING payload so the sender can validate round-trip data.

Parameters

out	<i>frame</i>	Frame to populate (zeroed then filled)
-----	--------------	--

in	<i>sequence</i>	Sequence number for this frame
in	<i>payload</i>	Payload to echo (NULL when payload_len is 0)
in	<i>payload_len</i>	Payload length in bytes [0, k_frame_max_payload]

Return values

<i>k_rx_ok</i>	Frame created successfully
<i>k_rx_err_invalid_arg</i>	frame is NULL, or payload NULL with len > 0
<i>k_rx_err_invalid_size</i>	payload_len exceeds k_frame_max_payload
<i>k_rx_err_validation_failed</i>	Post-condition type check failed

Precondition

frame != nullptr
 payload != NULL when payload_len > 0

Postcondition

frame->header.type == k_frame_type_pong
 frame->header.length == payload_len

Note

Stateless; safe to call from any context.

See also

[rx_frame_create_ping\(\)](#) Corresponding request creator

Since

Version 1.0.0

Definition at line 1032 of file [rx_frame.c](#).

```

01036 {
01037     if (frame == nullptr) {
01038         return k_rx_err_invalid_arg;
01039     }
01040
01041     if (payload_len > 0 && payload == nullptr) {
01042         return k_rx_err_invalid_arg;
01043     }
01044
01045     if (payload_len > k_frame_max_payload) {
01046         return k_rx_err_invalid_size;
01047     }
01048
01049     *frame = (rx_frame_t){0};
01050     frame->header.sequence = sequence;
01051     frame->header.length = (uint16_t)payload_len;
01052     frame->header.type = k_frame_type_pong;
01053     frame->header.flags = k_frame_flag_none;
01054
01055     if (payload_len > 0) {
01056         memcpy(frame->payload, payload, payload_len);

```

```

01057 }
01058
01059 if (frame->header.type != k_frame_type_pong) {
01060     return k_rx_err_validation_failed;
01061 }
01062
01063 return k_rx_ok;
01064 }

```

References [rx_frame_header_t::flags](#), [rx_frame_t::header](#), [k_frame_flag_none](#), [k_frame_max_payload](#), [k_frame_type_pong](#), [rx_frame_header_t::length](#), [rx_frame_t::payload](#), [rx_frame_header_t::sequence](#), and [rx_frame_header_t::type](#).

4.3.3.10 rx_frame_create_reset()

```

rx_err_t rx_frame_create_reset (
    rx_frame_t * frame,
    const uint16_t sequence) [nodiscard]

```

Create a RESET request frame.

Create RESET frame to reset communication state.

Initializes frame as a RESET request that asks the peer to reset communication state (sequence numbers, buffers). The receiver should respond with a RESET_ACK frame. Payload is always empty.

Parameters

out	<i>frame</i>	Frame to populate (zeroed then filled)
in	<i>sequence</i>	Sequence number for this frame

Return values

<i>k_rx_ok</i>	Frame created successfully
<i>k_rx_err_invalid_arg</i>	frame is nullptr
<i>k_rx_err_validation_failed</i>	Post-condition type check failed

Precondition

`frame != nullptr`

Caller has determined that a reset is necessary

Postcondition

`frame->header.type == k_frame_type_reset`

`frame->header.length == 0`

Note

Stateless; safe to call from any context.

Warning

Reset clears all sequence number state on both sides.

See also

[rx_frame_create_reset_ack\(\)](#) Corresponding acknowledgment creator

Since

Version 1.0.0

Definition at line 1091 of file [rx_frame.c](#).

```

01092 {
01093     if (frame == nullptr) {
01094         return k_rx_err_invalid_arg;
01095     }
01096     *frame = (rx_frame_t){0};
01097     frame->header.sequence = sequence;
01098     frame->header.length = 0;
01099     frame->header.type = k_frame_type_reset;
01100     frame->header.flags = k_frame_flag_none;
01101     if (frame->header.type != k_frame_type_reset) {
01102         return k_rx_err_validation_failed;
01103     }
01104     return k_rx_ok;
01105 }

```

References [rx_frame_header_t::flags](#), [rx_frame_t::header](#), [k_frame_flag_none](#), [k_frame_type_reset](#), [rx_frame_header_t::length](#), [rx_frame_header_t::sequence](#), and [rx_frame_header_t::type](#).

4.3.3.11 rx_frame_create_reset_ack()

```

rx_err_t rx_frame_create_reset_ack (
    rx_frame_t * frame,
    const uint16_t sequence) [nodiscard]

```

Create a RESET_ACK confirmation frame.

Create RESET_ACK frame (response to RESET).

Initializes frame as a RESET_ACK response to a received RESET request. Confirms that the receiver has completed its reset procedure. Payload is always empty; sequence matches the RESET request.

Parameters

out	<i>frame</i>	Frame to populate (zeroed then filled)
in	<i>sequence</i>	Sequence number (should match received RESET)

Return values

<i>k_rx_ok</i>	Frame created successfully
<i>k_rx_err_invalid_arg</i>	frame is nullptr

<code>k_rx_err_validation_failed</code>	Post-condition type check failed
---	----------------------------------

Precondition

`frame != nullptr`

A RESET frame was received and processed

Postcondition

`frame->header.type == k_frame_type_reset_ack`

`frame->header.length == 0`

Note

Stateless; safe to call from any context.

See also

[rx_frame_create_reset\(\)](#) Corresponding request creator

Since

Version 1.0.0

Definition at line 1134 of file [rx_frame.c](#).

```

01135 {
01136     if (frame == nullptr) {
01137         return k_rx_err_invalid_arg;
01138     }
01139     *frame = (rx_frame_t){0};
01140     frame->header.sequence = sequence;
01141     frame->header.length = 0;
01142     frame->header.type = k_frame_type_reset_ack;
01143     frame->header.flags = k_frame_flag_none;
01144     if (frame->header.type != k_frame_type_reset_ack) {
01145         return k_rx_err_validation_failed;
01146     }
01147     return k_rx_ok;
01148 }
01149
01150 return k_rx_ok;
01151 }
```

References [rx_frame_header_t::flags](#), [rx_frame_t::header](#), [k_frame_flag_none](#), [k_frame_type_reset_ack](#), [rx_frame_header_t::length](#), [rx_frame_header_t::sequence](#), and [rx_frame_header_t::type](#).

4.3.3.12 rx_frame_decode()

```

rx_err_t rx_frame_decode (
    const rx_frame_decoder_t * dec,
    const uint8_t * data,
    const uint32_t data_len,
    rx_frame_t * frame) [nodiscard]
```

Decode a frame from raw data.

Decode wire format to frame.

Validates the decoder state, extracts the frame header, copies the payload, and verifies the CRC-32 checksum.

Parameters

in	<i>dec</i>	Initialized frame decoder instance
----	------------	------------------------------------

in	<i>data</i>	Raw frame data buffer
in	<i>data_len</i>	Length of data buffer in bytes
out	<i>frame</i>	Decoded frame (header, payload, CRC)

Return values

<i>k_rx_ok</i>	Success - frame decoded and CRC verified
<i>k_rx_err_invalid_arg</i>	Any pointer parameter is nullptr or other invalid arguments
<i>k_rx_err_invalid_state</i>	Decoder not initialized
<i>k_rx_err_crc</i>	CRC-32 verification failed
<i>k_rx_err_invalid_size</i>	Payload exceeds maximum frame size

Note

This function performs validation at multiple points:

- Null pointer checks on all parameters
- Decoder initialization check
- Header parsing via [internal_decode_header\(\)](#)
- CRC verification via [internal_verify_crc\(\)](#)

Definition at line 726 of file [rx_frame.c](#).

```

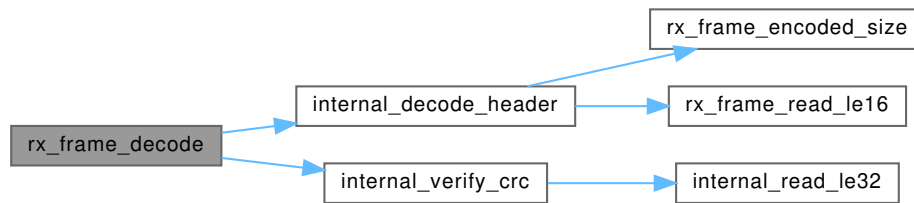
00730 {
00731     if (dec == nullptr || data == nullptr || frame == nullptr) {
00732         return k_rx_err_invalid_arg;
00733     }
00734
00735     if (dec->initialized != k_state_initialized) {
00736         return k_rx_err_invalid_state;
00737     }
00738
00739     uint32_t offset;
00740     rx_err_t err = internal_decode_header(data, data_len, frame, &offset);
00741     if (err != k_rx_ok) {
00742         return err;
00743     }
00744
00745     /* Read PAYLOAD */
00746     if (frame->header.length > 0) {
00747         memcpy(frame->payload, &data[offset], frame->header.length);
00748         offset += frame->header.length;
00749     }
00750
00751     err = internal_verify_crc(data, data_len, offset, &frame->crc);
00752     if (err != k_rx_ok) {
00753         return err;
00754     }
00755     return k_rx_ok;
00756 }

```

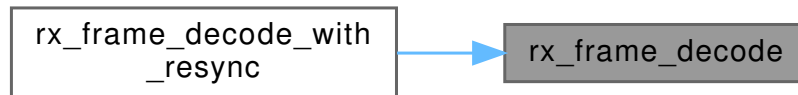
References [rx_frame_t::crc](#), [rx_frame_t::header](#), [rx_frame_decoder_t::initialized](#), [internal_decode_header\(\)](#), [internal_verify_crc\(\)](#), [k_state_initialized](#), [rx_frame_header_t::length](#), and [rx_frame_t::payload](#).

Referenced by [rx_frame_decode_with_resync\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.13 rx_frame_decode_with_resync()

```

rx_err_t rx_frame_decode_with_resync (
    const rx_frame_decoder_t * dec,
    const uint8_t * data,
    const uint32_t data_len,
    rx_frame_t * frame,
    uint32_t * bytes_discarded_out) [nodiscard]
  
```

Decode a frame from raw data with bounded sync-word resynchronization.

Decode wire format to frame with bounded sync-word resynchronization.

Provides stream recovery when the byte stream has become misaligned due to a dropped or inserted byte. On sync mismatch, scans forward up to `k_frame_max_scan_bytes` positions for the next occurrence of the sync word, discards the leading bytes, and reattempts decoding from that position.

This function is safe for repeated calls on a stream: the caller must advance its stream read pointer by `*bytes_discarded_out` on `k_rx_ok` and also on `k_rx_err_crc_mismatch` (any case where `bytes_discarded_out` is set) to avoid re-processing discarded bytes in subsequent calls.

Recovery algorithm:

1. Attempt `rx_frame_decode()` on `data[0..data_len-1]`.
2. If result is not `k_rx_err_protocol_error`, return immediately.
3. Call `internal_find_sync_offset()` to locate next sync word in `data[1..]`.
4. If not found, return `k_rx_err_protocol_error` with `*bytes_discarded_out = 0`.
5. Decode from `data[sync_offset..data_len-1]` using the trimmed sub-buffer.
6. Write `sync_offset` to `*bytes_discarded_out` and return result.

Parameters

in	<i>dec</i>	Initialized frame decoder handle
----	------------	----------------------------------

in	<i>data</i>	Raw byte buffer (may be stream-misaligned)
in	<i>data_len</i>	Length of data buffer in bytes
out	<i>frame</i>	Decoded frame (header, payload, CRC)
out	<i>bytes_discarded_out</i>	Bytes skipped before sync word (0 = aligned)

Returns

`rx_err_t` `k_rx_ok` on success, or `k_rx_err_invalid_arg`, `k_rx_err_invalid_state`, `k_rx_err_invalid_size`, `k_rx_err_protocol_error`, or `k_rx_err_crc_mismatch` on failure

Return values

<i>k_rx_ok</i>	Frame decoded and CRC verified
<i>k_rx_err_invalid_arg</i>	Any pointer parameter is nullptr
<i>k_rx_err_invalid_state</i>	Decoder not initialized
<i>k_rx_err_invalid_size</i>	Buffer too short to contain a valid frame
<i>k_rx_err_protocol_error</i>	No sync word found within scan window
<i>k_rx_err_crc_mismatch</i>	Sync found but CRC verification failed

Precondition

`dec` must be initialized via `rx_frame_decoder_init()`

`data` must point to a buffer of at least `data_len` bytes

`frame` must point to a valid `rx_frame_t` storage location (not NULL)

`bytes_discarded_out` must point to a valid `uint32_t` storage location (not NULL)

Postcondition

On `k_rx_ok`, `*bytes_discarded_out` == bytes skipped to reach sync word

On `k_rx_ok`, `frame` contains a fully verified decoded frame

On `k_rx_err_crc_mismatch`, `*bytes_discarded_out` == bytes skipped; caller must still advance stream pointer

Note

Parameter order follows the canonical convention: decoder input -> data input -> frame output -> diagnostic output (matching `rx_frame_decode()`).

Not thread-safe. Caller must synchronize access.

The scan is bounded at `k_frame_max_scan_bytes` (NASA Rule 2).

Log `*bytes_discarded_out > 0` as a stream-health diagnostic warning.

Warning

Caller must advance stream read pointer by `*bytes_discarded_out` after any call that returns `k_rx_ok` or `k_rx_err_crc_mismatch` to avoid infinite re-scan; `*bytes_discarded_out` is unmodified only when `k_rx_err_invalid_arg` is returned.

Example:

```
uint32_t discarded = k_bytes_discarded_none;
rx_err_t err = rx_frame_decode_with_resync(dec, data, data_len, &frame, &discarded);
if (err == k_rx_ok) {
    // Advance stream read pointer past any discarded bytes
    stream_read_ptr += discarded;
} else if (err == k_rx_err_crc_mismatch) {
    // Sync was found but CRC failed -- still advance to avoid infinite re-scan
    stream_read_ptr += discarded;
    rx_log_error(TAG, "sync found but CRC failed");
} else {
    rx_log_error(TAG, "no sync word within scan window");
}
```

See also

[rx_frame_decode\(\)](#) Simple decode without stream recovery

[internal_find_sync_offset\(\)](#) Bounded sync word scanner

[k_frame_max_scan_bytes](#) Scan window upper bound

Since

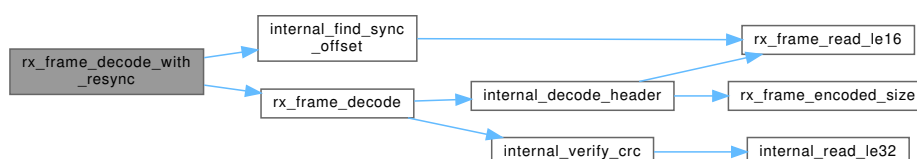
Version 1.0.0

Definition at line 836 of file [rx_frame.c](#).

```
00841 {
00842     if (dec == nullptr || data == nullptr || frame == nullptr || bytes_discarded_out == nullptr) {
00843         return k_rx_err_invalid_arg;
00844     }
00845
00846     *bytes_discarded_out = k_bytes_discarded_none;
00847
00848     /* Fast path: attempt aligned decode first */
00849     rx_err_t err = rx_frame_decode(dec, data, data_len, frame);
00850     if (err != k_rx_err_protocol_error) {
00851         /* Either success, or an error other than sync mismatch (CRC, size, etc.) */
00852         return err;
00853     }
00854
00855     /* Sync word not at offset 0: scan forward for next sync word */
00856     uint32_t sync_offset = (uint32_t)k_frame_offset_start;
00857     err = internal_find_sync_offset(data, data_len, &sync_offset);
00858     if (err != k_rx_ok) {
00859         /* No sync word found within bounded window */
00860         return k_rx_err_protocol_error;
00861     }
00862
00863     /* Report discarded bytes now: caller needs this to advance past the sync
00864      * even if the subsequent decode fails (e.g. CRC mismatch at new offset) */
00865     *bytes_discarded_out = sync_offset;
00866
00867     /* Reattempt decode from the discovered sync offset */
00868     err = rx_frame_decode(dec, &data[sync_offset], data_len - sync_offset, frame);
00869
00870     return err;
00871 }
```

References [internal_find_sync_offset\(\)](#), [k_bytes_discarded_none](#), [k_frame_offset_start](#), and [rx_frame_decode\(\)](#).

Here is the call graph for this function:



4.3.3.14 rx_frame_decoder_deinit()

```
rx_err_t rx_frame_decoder_deinit (
    rx_frame_decoder_t * dec) [nodiscard]
```

Deinitialize a frame decoder instance.

Deinitialize frame decoder.

Marks the decoder as uninitialized, preventing further use. Verifies deinitialization completed successfully (redundant validation per Rule 5).

Parameters

out	<i>dec</i>	Decoder instance to deinitialize
-----	------------	----------------------------------

Returns

k_rx_ok on successful deinitialization

Return values

<i>k_rx_err_invalid_arg</i>	if dec is nullptr
<i>k_rx_err_validation_failed</i>	if deinitialization verification fails

Definition at line 690 of file [rx_frame.c](#).

```
00691 {
00692     if (dec == nullptr) {
00693         return k_rx_err_invalid_arg;
00694     }
00695     dec->initialized = k_state_uninitialized;
00696     if (dec->initialized != k_state_uninitialized) {
00697         return k_rx_err_validation_failed;
00698     }
00699     return k_rx_ok;
00700 }
00701 }
```

References [rx_frame_decoder_t::initialized](#), and [k_state_uninitialized](#).

4.3.3.15 rx_frame_decoder_init()

```
rx_err_t rx_frame_decoder_init (
    rx_frame_decoder_t * dec) [nodiscard]
```

Initialize a frame decoder instance.

Initialize frame decoder.

Prepares the decoder for use by setting the initialized flag and verifying the initialization completed successfully (redundant validation per Rule 5).

Parameters

out	<i>dec</i>	Decoder instance to initialize
-----	------------	--------------------------------

Returns

k_rx_ok on successful initialization

Return values

<i>k_rx_err_invalid_arg</i>	if dec is nullptr
-----------------------------	-------------------

<code>k_rx_err_validation_failed</code>	if initialization verification fails
---	--------------------------------------

Definition at line 665 of file [rx_frame.c](#).

```

00666 {
00667     if (dec == nullptr) {
00668         return k_rx_err_invalid_arg;
00669     }
00670
00671     dec->initialized = k_state_initialized;
00672     if (dec->initialized != k_state_initialized) {
00673         return k_rx_err_validation_failed;
00674     }
00675     return k_rx_ok;
00676 }

```

References [rx_frame_decoder_t::initialized](#), and [k_state_initialized](#).

4.3.3.16 rx_frame_encode()

```

rx_err_t rx_frame_encode (
    const rx_frame_encoder_t * enc,
    const rx_frame_t * frame,
    uint8_t * output,
    uint32_t * output_len) [nodiscard]

```

Encode frame to wire format.

Serializes frame to wire format with SYNC, header, payload, and CRC-32. All multi-byte fields are little-endian (SYNC, SEQ, LEN, CRC-32).

Parameters

in	<i>enc</i>	Encoder handle
in	<i>frame</i>	Frame to encode
out	<i>output</i>	Output buffer (min k_frame_min_size + payload bytes)
out	<i>output_len</i>	Actual number of bytes written

Returns

- `k_rx_ok` on success
- `k_rx_err_invalid_arg` if any pointer is nullptr
- `k_rx_err_invalid_state` if encoder not initialized
- `k_rx_err_invalid_size` if payload exceeds max

Definition at line 586 of file [rx_frame.c](#).

```

00590 {
00591     if (enc == nullptr || frame == nullptr || output == nullptr || output_len == nullptr) {
00592         return k_rx_err_invalid_arg;
00593     }
00594
00595     if (enc->initialized != k_state_initialized) {
00596         return k_rx_err_invalid_state;
00597     }
00598
00599     /* Validate payload size */
00600     if (frame->header.length > k_frame_max_payload) {
00601         return k_rx_err_invalid_size;
00602     }

```



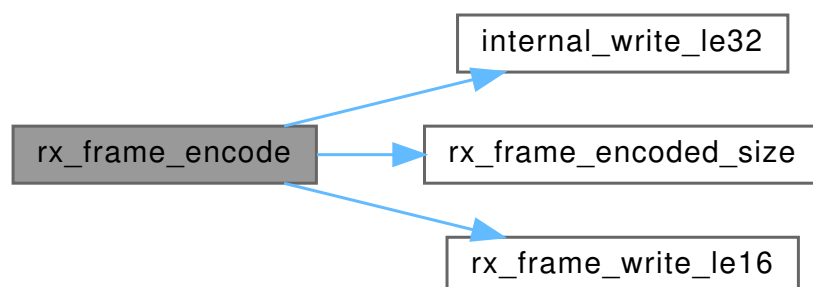
```

00603
00604 /* Calculate total frame size */
00605 uint32_t frame_size = rx_frame_encoded_size(frame->header.length);
00606 uint32_t offset = k_frame_offset_start;
00607
00608 /* Write SYNC word (little-endian: wire bytes 0xAA, 0x55) */
00609 rx_frame_write_le16(&output[offset], k_frame_sync_word);
00610 offset += k_frame_sync_size;
00611
00612 /* Write SEQ (little-endian) */
00613 rx_frame_write_le16(&output[offset], frame->header.sequence);
00614 offset += k_frame_seq_size;
00615
00616 /* Write LEN (little-endian) */
00617 rx_frame_write_le16(&output[offset], frame->header.length);
00618 offset += k_frame_len_size;
00619
00620 /* Write TYPE (1 byte) */
00621 output[offset] = frame->header.type;
00622 offset += k_frame_type_size;
00623
00624 /* Write FLAGS (1 byte) */
00625 output[offset] = frame->header.flags;
00626 offset += k_frame_flags_size;
00627
00628 /* Write PAYLOAD */
00629 if (frame->header.length > 0) {
00630     memcpy(&output[offset], frame->payload, frame->header.length);
00631     offset += frame->header.length;
00632 }
00633
00634 /* Calculate CRC-32 over SYNC + Header + Payload (IEEE 802.3 polynomial) */
00635 uint32_t crc = rx_crc32_ieee(output, offset);
00636
00637 /* Write CRC-32 (little-endian to match IEEE 802.3 LSB-first order) */
00638 rx_err_t err = internal_write_le32(&output[offset], frame_size - offset, crc);
00639 if (err != k_rx_ok) {
00640     return err;
00641 }
00642 offset += k_frame_crc_size;
00643
00644 *output_len = frame_size;
00645 return k_rx_ok;
00646 }

```

References [rx_frame_header_t::flags](#), [rx_frame_t::header](#), [rx_frame_encoder_t::initialized](#), [internal_write_le32\(\)](#), [k_frame_crc_size](#), [k_frame_flags_size](#), [k_frame_len_size](#), [k_frame_max_payload](#), [k_frame_offset_start](#), [k_frame_seq_size](#), [k_frame_sync_size](#), [k_frame_sync_word](#), [k_frame_type_size](#), [k_state_initialized](#), [rx_frame_header_t::length](#), [rx_frame_t::payload](#), [rx_frame_encoded_size\(\)](#), [rx_frame_write_le16\(\)](#), [rx_frame_header_t::sequence](#), and [rx_frame_header_t::type](#).

Here is the call graph for this function:



4.3.3.17 rx_frame_encoder_deinit()

```

rx_err_t rx_frame_encoder_deinit (
    rx_frame_encoder_t * enc) [nodiscard]

```

Deinitialize a frame encoder instance.

Deinitialize frame encoder.

Marks the encoder as uninitialized, preventing further use. Verifies deinitialization completed successfully (redundant validation per Rule 5).

Parameters

out	<i>enc</i>	Encoder instance to deinitialize
-----	------------	----------------------------------

Returns

`k_rx_ok` on successful deinitialization

Return values

<i>k_rx_err_invalid_arg</i>	if <code>enc</code> is <code>nullptr</code>
<i>k_rx_err_validation_failed</i>	if deinitialization verification fails

Definition at line 573 of file [rx_frame.c](#).

```
00574 {  
00575     if (enc == nullptr) {  
00576         return k_rx_err_invalid_arg;  
00577     }  
00578  
00579     enc->initialized = k_state_uninitialized;  
00580     if (enc->initialized != k_state_uninitialized) {  
00581         return k_rx_err_validation_failed;  
00582     }  
00583     return k_rx_ok;  
00584 }
```

References [rx_frame_encoder_t::initialized](#), and [k_state_uninitialized](#).

4.3.3.18 rx_frame_encoder_init()

```
rx_err_t rx_frame_encoder_init (  
    rx\_frame\_encoder\_t * enc) [nodiscard]
```

Initialize a frame encoder instance.

Initialize frame encoder.

Prepares the encoder for use by setting the initialized flag and verifying the initialization completed successfully (redundant validation per Rule 5).

Parameters

out	<i>enc</i>	Encoder instance to initialize
-----	------------	--------------------------------

Returns

`k_rx_ok` on successful initialization

Return values

<i>k_rx_err_invalid_arg</i>	if <code>enc</code> is <code>nullptr</code>
-----------------------------	---

<code>k_rx_err_validation_failed</code>	if initialization verification fails
---	--------------------------------------

Definition at line 548 of file `rx_frame.c`.

```
00549 {
00550     if (enc == nullptr) {
00551         return k_rx_err_invalid_arg;
00552     }
00553
00554     enc->initialized = k_state_initialized;
00555     if (enc->initialized != k_state_initialized) {
00556         return k_rx_err_validation_failed;
00557     }
00558     return k_rx_ok;
00559 }
```

References `rx_frame_encoder_t::initialized`, and `k_state_initialized`.

4.4 rx_frame.c

[Go to the documentation of this file.](#)

```
00001 /* lib/rx_frame/src/rx_frame.c */
00002
00003 /**
00004  * @file rx_frame.c
00005  * @brief Frame Layer Protocol Implementation
00006  *
00007  * @details
00008  * ## Overview
00009  *
00010  * This file implements the frame encoding and decoding operations for the
00011  * SPI communication protocol between RPi5 and RX72N. The implementation is
00012  * bit-exact compatible with the Go reference in star-gateway/internal/frame/.
00013  *
00014  * ## Module Architecture
00015  *
00016  * @dot
00017  * digraph frame_arch {
00018  *     rankdir=TB;
00019  *     node [shape=box, style=rounded];
00020  *
00021  *     subgraph cluster_public {
00022  *         label="Public API";
00023  *         style=filled;
00024  *         color=lightgrey;
00025  *         encoder_init [label="rx_frame_encoder_init()"];
00026  *         encoder_deinit [label="rx_frame_encoder_deinit()"];
00027  *         encode [label="rx_frame_encode()"];
00028  *         decoder_init [label="rx_frame_decoder_init()"];
00029  *         decoder_deinit [label="rx_frame_decoder_deinit()"];
00030  *         decode [label="rx_frame_decode()"];
00031  *         decode_resync [label="rx_frame_decode_with_resync()"];
00032  *         create_ack [label="rx_frame_create_ack()"];
00033  *         create_nack [label="rx_frame_create_nack()"];
00034  *     }
00035  *
00036  *     subgraph cluster_internal {
00037  *         label="Internal Functions";
00038  *         style=filled;
00039  *         color=lightyellow;
00040  *         write_le32 [label="internal_write_le32()"];
00041  *         read_le32 [label="internal_read_le32()"];
00042  *         decode_header [label="internal_decode_header()"];
00043  *         verify_crc [label="internal_verify_crc()"];
00044  *         find_sync [label="internal_find_sync_offset()"];
00045  *     }
00046  *
00047  *     subgraph cluster_deps {
00048  *         label="Dependencies";
00049  *         style=filled;
00050  *         color=lightblue;
00051  *         rx_crc [label="rx_crc32_ieee()"];
00052  *         rx_check [label="RX_CHECK_NULL_PTR()"];
00053  *         memcpy_fn [label="memcpy()"];
00054  *         memset_fn [label="memset()"];
00055  *     }
00056  * }
```

```

00057 * encode -> write_le32;
00058 * encode -> rx_crc;
00059 * decode -> decode_header;
00060 * decode -> verify_crc;
00061 * decode_resync -> decode;
00062 * decode_resync -> find_sync;
00063 * find_sync -> rx_check;
00064 * decode_header -> rx_crc [style=invis];
00065 * verify_crc -> read_le32;
00066 * verify_crc -> rx_crc;
00067 * write_le32 -> rx_check;
00068 * read_le32 -> rx_check;
00069 * create_ack -> memset_fn;
00070 * create_nack -> memset_fn;
00071 * }
00072 * @enddot
00073
00074 * ## Encoding Process
00075 *
00076 * The encoder serializes a rx_frame_t structure to wire format:
00077 *
00078 * 1. **Validate inputs** - Check pointers, initialization state, payload size
00079 * 2. **Write SYNC** - 2-byte marker (0x55AA) in little-endian (wire: 0xAA, 0x55)
00080 * 3. **Write header** - SEQ, LEN in little-endian; TYPE, FLAGS as bytes
00081 * 4. **Copy payload** - Direct byte copy (no endian conversion)
00082 * 5. **Compute CRC** - IEEE 802.3 CRC-32 over SYNC+header+payload
00083 * 6. **Write CRC** - 4-byte CRC in little-endian format
00084 *
00085 * @dot
00086 * digraph encode_flow {
00087 *   rankdir=LR;
00088 *   node [shape=box, style=rounded];
00089 *
00090 *   input [label="rx_frame_t\n(host format)"];
00091 *   validate [label="Validate"];
00092 *   sync [label="Write SYNC\n(LE)"];
00093 *   header [label="Write Header\n(LE/bytes)"];
00094 *   payload [label="Copy Payload"];
00095 *   crc_calc [label="Calculate CRC"];
00096 *   crc_write [label="Write CRC\n(LE)"];
00097 *   output [label="Wire Buffer"];
00098 *
00099 *   input -> validate -> sync -> header -> payload -> crc_calc -> crc_write -> output;
00100 * }
00101 * @enddot
00102
00103 * ## Decoding Process
00104 *
00105 * The decoder parses wire format into a rx_frame_t structure:
00106 *
00107 * 1. **Validate inputs** - Check pointers, initialization state
00108 * 2. **Verify SYNC** - Check for 0x55AA marker
00109 * 3. **Parse header** - Extract SEQ, LEN, TYPE, FLAGS with endian conversion
00110 * 4. **Validate length** - Ensure payload fits in buffer
00111 * 5. **Copy payload** - Direct byte copy
00112 * 6. **Verify CRC** - Compute expected CRC, compare with received
00113 *
00114 * ## Memory Usage
00115 *
00116 * | Function | Stack | Notes |
00117 * |-----|-----|-----|
00118 * | rx_frame_encode() | 32 bytes | Variables, return address |
00119 * | rx_frame_decode() | 32 bytes | Variables, return address |
00120 * | internal_decode_header() | 24 bytes | Local variables |
00121 * | internal_verify_crc() | 20 bytes | CRC values |
00122 *
00123 * ## Performance (RX72N @ 240 MHz)
00124 *
00125 * | Operation | Empty | 64B | 256B | 1KB |
00126 * |-----|-----|-----|-----|
00127 * | Encode | 5 us | 12 us | 35 us | 130 us |
00128 * | Decode | 5 us | 12 us | 35 us | 130 us |
00129 *
00130 * **Bottleneck:** CRC calculation dominates for larger payloads.
00131 * Hardware CRC reduces time by ~4x.
00132 *
00133 * ## NASA Power of 10 Compliance
00134 *
00135 * | Rule | Status | Notes |
00136 * |-----|-----|-----|
00137 * | 1. Simple control flow | [PASS] | Sequential processing, no goto |
00138 * | 2. Fixed loop bounds | [PASS] | Payload loop bounded by payload length; sync scanner bounded by  
k_frame_max_scan_bytes |
00139 * | 3. No dynamic memory | [PASS] | All buffers caller-provided |
00140 * | 4. Short functions | [PASS] | Largest function ~50 lines |
00141 * | 5. Assertions | [PASS] | All inputs validated, post-conditions checked |
00142 * | 6. Small scope | [PASS] | Variables at point of use |

```

```

00143 * | 7. Check returns | [PASS] | All errors propagated |
00144 * | 8. Limited preprocessor | [PASS] | Only includes |
00145 * | 9. Restrict pointers | [PASS] | No function pointers |
00146 * | 10. Compiler warnings | [PASS] | Clean build with -Wall -Wextra -Werror |
00147 *
00148 * ## SOLID Principles
00149 *
00150 * | Principle | Implementation |
00151 * |-----|-----|
00152 * | **Single Responsibility** | Encode/decode frames only |
00153 * | **Open/Closed** | New types/flags via enums |
00154 * | **Liskov Substitution** | N/A (no inheritance) |
00155 * | **Interface Segregation** | Separate encoder/decoder |
00156 * | **Dependency Inversion** | Depends on rx_crc abstraction |
00157 *
00158 * ## Test Coverage
00159 *
00160 * See tests/test_rx_frame.c for comprehensive unit tests covering:
00161 * - Encode/decode round-trip
00162 * - CRC validation
00163 * - Boundary conditions (min/max payload)
00164 * - Error cases (NULL pointers, invalid sizes)
00165 *
00166 * @see rx_frame.h Public API and frame format documentation
00167 * @see rx_crc.h CRC-32 computation
00168 * @see star-gateway/internal/frame/ Go reference implementation
00169 *
00170 * @author STAR Team
00171 * @date 2026-01-29
00172 * @copyright Copyright (c) 2026 STAR Project. MIT License.
00173 * @since Version 1.0.0
00174 */
00175
00176 #include "rx_frame.h"
00177
00178 #include <string.h>
00179
00180 #include "rx_check.h"
00181 #include "rx_crc.h"
00182
00183 /*
=====
00184 * Byte Serialization Constants
00185 *
00186 * Named constants for byte manipulation in serialization functions.
00187 * Eliminates magic numbers and documents the bit/byte operations.
00188 *
=====
00189 */
00190
00191 /**
00192 * @brief Bit shift amounts for extracting bytes from multi-byte values
00193 *
00194 * Each byte position requires shifting by (position * 8) bits.
00195 */
00196 typedef enum : uint8_t {
00197     k_shift_byte = 0, /**< No shift needed for byte 0 (LSB) */
00198     k_shift_byte = 8, /**< Shift 8 bits for byte 1 */
00199     k_shift_byte = 16, /**< Shift 16 bits for byte 2 */
00200     k_shift_byte = 24, /**< Shift 24 bits for byte 3 (MSB) */
00201 } byte_shift_t;
00202
00203 /**
00204 * @brief Initialization state values
00205 */
00206 typedef enum : uint8_t {
00207     k_state_uninitialized = 0, /**< Object not initialized */
00208     k_state_initialized = 1, /**< Object initialized and ready */
00209 } init_state_t;
00210
00211 /**
00212 * @enum frame_offset_t
00213 * @brief Frame byte offsets used during encoding, decoding, and sync scanning
00214 *
00215 * @details
00216 * Defines named byte-offset constants used by the frame encoder, decoder, and
00217 * the bounded sync-word scanner. k_frame_offset_start marks the beginning of a
00218 * frame buffer for normal aligned decoding. k_frame_scan_start_offset is the
00219 * index at which internal_find_sync_offset() begins its forward scan -- offset 0
00220 * is omitted because rx_frame_decode() has already tested it before invoking
00221 * the scanner, so the scan starts at the next candidate position.
00222 *
00223 * @invariant k_frame_scan_start_offset > k_frame_offset_start, ensuring the
00224 * scanner never re-tests the byte that rx_frame_decode() already checked.
00225 *
00226 * @code{.c}
00227 * // Normal aligned decode starts at k_frame_offset_start (0)

```

```

00228 * rx_err_t err = rx_frame_decode(dec, data, data_len, frame);
00229 *
00230 * // On sync mismatch, internal_find_sync_offset() scans from offset 1:
00231 * uint8_t sync_off = k_frame_scan_start_offset;
00232 * // ... scanner advances sync_off until sync word found or scan limit reached
00233 * @endcode
00234 *
00235 * @see rx_frame_decode() Uses k_frame_offset_start for normal aligned decoding
00236 * @see internal_find_sync_offset() Begins scan at k_frame_scan_start_offset
00237 *
00238 * @since Version 1.0.0
00239 */
00240 typedef enum : uint8_t {
00241     k_frame_offset_start = 0, /**< Start offset for frame parsing */
00242     k_frame_scan_start_offset = 1, /**< First offset checked in internal_find_sync_offset();
00243                                     offset 0 is presumed already tested by rx_frame_decode() */
00244 } frame_offset_t;
00245
00246 /**
00247 * @enum frame_resync_discarded_t
00248 * @brief Byte-discard counter initial value for rx_frame_decode_with_resync()
00249 *
00250 * @details
00251 * k_bytes_discarded_none is the zero sentinel written to *bytes_discarded_out
00252 * at the start of rx_frame_decode_with_resync() before any sync search. Using
00253 * a named constant makes the intent auditable and prevents a raw 0 literal from
00254 * appearing in the implementation (NASA Rule 8 / STAR no-magic-numbers policy).
00255 *
00256 * @invariant k_bytes_discarded_none == 0 (matches the initial no-discard state)
00257 *
00258 * @code{.c}
00259 * uint32_t bytes_discarded = k_bytes_discarded_none;
00260 * rx_err_t err = rx_frame_decode_with_resync(&dec, data, data_len, &frame, &bytes_discarded);
00261 * @endcode
00262 *
00263 * @see rx_frame_decode_with_resync() Only caller of this constant
00264 *
00265 * @since Version 1.0.0
00266 */
00267 typedef enum : uint32_t {
00268     k_bytes_discarded_none = 0U, /**< No bytes have been discarded; stream is aligned */
00269 } frame_resync_discarded_t;
00270
00271 /*
=====
00272 * Private Helper Functions
00273 *
=====
00274 */
00275
00276 /**
00277 * @brief Write uint32 in little-endian format (for CRC-32)
00278 *
00279 * @param[out] buf Output buffer (at least 4 bytes)
00280 * @param[in] buf_len Size of output buffer in bytes (must be >= 4)
00281 * @param[in] val Value to write
00282 *
00283 * @return k_rx_ok on success
00284 * @retval k_rx_err_invalid_arg if buf is nullptr
00285 * @retval k_rx_err_invalid_size if buf_len < 4
00286 */
00287 static rx_err_t internal_write_le32(uint8_t* buf, const uint32_t buf_len, const uint32_t val)
00288 {
00289     RX_CHECK_NULL_PTR(buf, "FRAME", "LE32 write buffer is nullptr");
00290     if (buf_len < k_frame_crc_size) {
00291         return k_rx_err_invalid_size;
00292     }
00293
00294     buf[k_le32_byte] = (uint8_t)(val & k_rx_byte_mask);
00295     buf[k_le32_byte + 1] = (uint8_t)((val >> k_shift_byte) & k_rx_byte_mask);
00296     buf[k_le32_byte + 2] = (uint8_t)((val >> k_shift_byte) & k_rx_byte_mask);
00297     buf[k_le32_byte + 3] = (uint8_t)((val >> k_shift_byte) & k_rx_byte_mask);
00298     return k_rx_ok;
00299 }
00300
00301 /**
00302 * @brief Read uint32 in little-endian format (for CRC-32)
00303 *
00304 * @param[in] buf Input buffer (at least 4 bytes)
00305 * @param[in] buf_len Size of input buffer in bytes (must be >= 4)
00306 * @param[out] out_val Pointer to store decoded value
00307 *
00308 * @return k_rx_ok on success
00309 * @retval k_rx_err_invalid_arg if buf or out_val is nullptr
00310 * @retval k_rx_err_invalid_size if buf_len < 4
00311 */
00312 static rx_err_t internal_read_le32(const uint8_t* buf, const uint32_t buf_len, uint32_t* out_val)

```

```

00313 {
00314     RX_CHECK_NULL_PTR(buf, "FRAME", "LE32 read buffer is nullptr");
00315     RX_CHECK_NULL_PTR(out_val, "FRAME", "LE32 output pointer is nullptr");
00316     if (buf_len < k_frame_crc_size) {
00317         return k_rx_err_invalid_size;
00318     }
00319
00320     *out_val = (uint32_t)buf[k_le32_byte] | ((uint32_t)buf[k_le32_byte] « k_shift_byte) |
00321         ((uint32_t)buf[k_le32_byte] « k_shift_byte) |
00322         ((uint32_t)buf[k_le32_byte] « k_shift_byte);
00323     return k_rx_ok;
00324 }
00325
00326 /**
00327  * @brief Decode frame header from raw data buffer
00328  *
00329  * Extracts and validates frame header fields (sync word, sequence, length, type, flags)
00330  * from the beginning of a raw data buffer. Verifies the sync word matches expected value
00331  * and validates payload length is within bounds. Outputs the offset where payload data
00332  * begins for subsequent processing.
00333  *
00334  * @param[in] data      Input data buffer containing encoded frame
00335  * @param[in] data_len  Length of input buffer in bytes
00336  * @param[out] frame    Frame structure to populate with decoded header fields
00337  * @param[out] offset_out Offset in buffer where payload begins (after header and sync)
00338  *
00339  * @return k_rx_ok on successful header decode
00340  * @retval k_rx_err_invalid_arg if data, frame, or offset_out is nullptr
00341  * @retval k_rx_err_invalid_size if data_len is too small or decoded payload length is out of bounds
00342  * @retval k_rx_err_protocol_error if sync word does not match expected value (k_frame_sync_word)
00343  */
00344 static rx_err_t internal_decode_header(const uint8_t* data,
00345                                       const uint32_t data_len,
00346                                       rx_frame_t* frame,
00347                                       uint32_t* offset_out)
00348 {
00349     if (data == nullptr || frame == nullptr || offset_out == nullptr) {
00350         return k_rx_err_invalid_arg;
00351     }
00352
00353     if (data_len < k_frame_min_size) {
00354         return k_rx_err_invalid_size;
00355     }
00356
00357     uint32_t offset = k_frame_offset_start;
00358     uint16_t sync_word = rx_frame_read_le16(&data[offset]);
00359     if (sync_word != k_frame_sync_word) {
00360         return k_rx_err_protocol_error;
00361     }
00362     offset += k_frame_sync_size;
00363
00364     frame->header.sequence = rx_frame_read_le16(&data[offset]);
00365     offset += k_frame_seq_size;
00366
00367     frame->header.length = rx_frame_read_le16(&data[offset]);
00368     offset += k_frame_len_size;
00369
00370     if (frame->header.length > k_frame_max_payload) {
00371         return k_rx_err_invalid_size;
00372     }
00373
00374     uint32_t expected_size = rx_frame_encoded_size(frame->header.length);
00375     if (data_len < expected_size) {
00376         return k_rx_err_invalid_size;
00377     }
00378
00379     frame->header.type = data[offset];
00380     offset += k_frame_type_size;
00381
00382     frame->header.flags = data[offset];
00383     offset += k_frame_flags_size;
00384
00385     if (offset_out != nullptr) {
00386         *offset_out = offset;
00387     }
00388
00389     return k_rx_ok;
00390 }
00391
00392 /**
00393  * @brief Verify CRC-32 checksum at specified offset in data buffer
00394  *
00395  * Reads the CRC-32 value stored at the specified offset in the buffer, calculates
00396  * the expected CRC over the data preceding it, and compares for match. Uses IEEE 802.3
00397  * CRC-32 polynomial (0x04C11DB7), compatible with Go's crc32.ChecksumIEEE(). On success,
00398  * outputs the received CRC value for caller inspection.
00399  */

```

```

00400 * @param[in] data Input data buffer containing payload followed by CRC
00401 * @param[in] data_len Total length of input buffer in bytes
00402 * @param[in] offset Byte offset in buffer where CRC value is stored
00403 * @param[out] crc_out Pointer to store received CRC value from buffer
00404 *
00405 * @return k_rx_ok on successful CRC verification (received matches calculated)
00406 * @retval k_rx_err_invalid_arg if data or crc_out is nullptr
00407 * @retval k_rx_err_invalid_size if offset + CRC size exceeds buffer length or data_len too small
00408 * @retval k_rx_err_crc_mismatch if calculated CRC does not match received CRC value
00409 */
00410 static rx_err_t
00411 internal_verify_crc(const uint8_t* data, uint32_t data_len, uint32_t offset, uint32_t* crc_out)
00412 {
00413     if (data == nullptr || crc_out == nullptr) {
00414         return k_rx_err_invalid_arg;
00415     }
00416
00417     if (data_len < k_frame_min_size || (offset + k_frame_crc_size) > data_len) {
00418         return k_rx_err_invalid_size;
00419     }
00420
00421     uint32_t received_crc = 0;
00422     rx_err_t err = internal_read_le32(&data[offset], data_len - offset, &received_crc);
00423     if (err != k_rx_ok) {
00424         return err;
00425     }
00426     uint32_t calculated_crc = rx_crc32_ieee(data, offset);
00427
00428     if (received_crc != calculated_crc) {
00429         return k_rx_err_crc_mismatch;
00430     }
00431
00432     *crc_out = received_crc;
00433
00434     return k_rx_ok;
00435 }
00436
00437 /**
00438 * @brief Scan a byte buffer for the frame sync word with a fixed upper bound
00439 *
00440 * @details
00441 * Performs a linear scan of the buffer starting at offset 1 (offset 0 is
00442 * presumed to have already failed sync check) looking for the 2-byte sync
00443 * pattern k_frame_sync_word in little-endian wire encoding (first byte 0xAA,
00444 * second byte 0x55). The scan is bounded at k_frame_max_scan_bytes to satisfy
00445 * NASA Rule 2 (fixed loop upper-bounds) and prevent infinite consumption of
00446 * a permanently corrupt stream.
00447 *
00448 * The function stops and reports success at the first occurrence of the sync
00449 * pattern. If no pattern is found within the window, k_rx_err_protocol_error
00450 * is returned.
00451 *
00452 * Algorithm:
00453 * 1. Validate preconditions (non-null pointers, minimum buffer size).
00454 * 2. Compute scan limit = min(data_len - k_frame_sync_size, k_frame_max_scan_bytes).
00455 * 3. Iterate i from 1 to scan_limit inclusive:
00456 *     a. Read 16-bit LE value at data[i].
00457 *     b. If it matches k_frame_sync_word, write i to offset_out and return k_rx_ok.
00458 * 4. Return k_rx_err_protocol_error if no match found.
00459 *
00460 * @param[in] data Buffer to scan (at least 2 bytes)
00461 * @param[in] data_len Length of buffer in bytes (must be >= 2)
00462 * @param[out] offset_out Byte offset of the first sync word found (>= 1)
00463 *
00464 * @return rx_err_t Error code
00465 * @retval k_rx_ok Sync word found; offset written to offset_out
00466 * @retval k_rx_err_invalid_arg data or offset_out is nullptr
00467 * @retval k_rx_err_invalid_size data_len < 2 (cannot contain a sync word)
00468 * @retval k_rx_err_protocol_error No sync word found within k_frame_max_scan_bytes
00469 *
00470 * @pre data must point to a readable buffer of at least data_len bytes
00471 * @pre data_len must be >= k_frame_sync_size (2)
00472 * @post On k_rx_ok, offset_out is in range [1, min(data_len - k_frame_sync_size, k_frame_max_scan_bytes)]
00473 * @post On error, offset_out is unchanged
00474 *
00475 * @note Not thread-safe. Caller must ensure exclusive access to data buffer.
00476 * @note The loop bound is min(data_len - k_frame_sync_size, k_frame_max_scan_bytes), which is
00477 *     statically bounded by k_frame_max_scan_bytes = 1036 (NASA Rule 2).
00478 *
00479 * @par Performance:
00480 * O(N) where N <= k_frame_max_scan_bytes. Each iteration performs one 16-bit
00481 * LE read and one comparison. Worst case: 1036 iterations (no sync found).
00482 *
00483 * @par Example:
00484 * @code{.c}
00485 * // Buffer where offset k_frame_offset_start failed sync; sync is at index 1
00486 * uint8_t data[] = {0x00, 0xAA, 0x55, 0x01, 0x00};

```



```

00487 * uint32_t offset = k_frame_offset_start;
00488 * rx_err_t err = internal_find_sync_offset(data, sizeof(data), &offset);
00489 * if (err == k_rx_ok) {
00490 *     // offset == 1: decode from data[offset]
00491 * } else {
00492 *     // k_rx_err_protocol_error: no sync found within scan window
00493 * }
00494 * @endcode
00495 *
00496 * @see rx_frame_decode_with_resync() Public API that calls this function
00497 * @see k_frame_sync_word Sync word value (0x55AA)
00498 * @see k_frame_max_scan_bytes Maximum scan window (1036 bytes)
00499 *
00500 * @since Version 1.0.0
00501 */
00502 static rx_err_t
00503 internal_find_sync_offset(const uint8_t* data, const uint32_t data_len, uint32_t* offset_out)
00504 {
00505     if (data == nullptr || offset_out == nullptr) {
00506         return k_rx_err_invalid_arg;
00507     }
00508
00509     if (data_len < k_frame_sync_size) {
00510         return k_rx_err_invalid_size;
00511     }
00512
00513     /* Bound scan to at most k_frame_max_scan_bytes positions beyond offset 0 */
00514     uint32_t scan_limit = data_len - k_frame_sync_size;
00515     if (scan_limit > k_frame_max_scan_bytes) {
00516         scan_limit = k_frame_max_scan_bytes;
00517     }
00518
00519     /* Scan starting at k_frame_scan_start_offset; offset 0 already failed sync check */
00520     for (uint32_t i = k_frame_scan_start_offset; i <= scan_limit; i++) {
00521         const uint16_t candidate = rx_frame_read_le16(&data[i]);
00522         if (candidate == k_frame_sync_word) {
00523             *offset_out = i;
00524             return k_rx_ok;
00525         }
00526     }
00527     return k_rx_err_protocol_error;
00528 }
00529 }
00530
00531 /*
=====
00532 * Encoder API Implementation
00533 *
=====
00534 */
00535 /**
00536 * @brief Initialize a frame encoder instance
00537 *
00538 * Prepares the encoder for use by setting the initialized flag and verifying
00539 * the initialization completed successfully (redundant validation per Rule 5).
00540 *
00541 * @param[out] enc Encoder instance to initialize
00542 *
00543 * @return k_rx_ok on successful initialization
00544 * @retval k_rx_err_invalid_arg if enc is nullptr
00545 * @retval k_rx_err_validation_failed if initialization verification fails
00546 */
00547 rx_err_t rx_frame_encoder_init(rx_frame_encoder_t* enc)
00548 {
00549     if (enc == nullptr) {
00550         return k_rx_err_invalid_arg;
00551     }
00552
00553     enc->initialized = k_state_initialized;
00554     if (enc->initialized != k_state_initialized) {
00555         return k_rx_err_validation_failed;
00556     }
00557     return k_rx_ok;
00558 }
00559
00560 /**
00561 * @brief Deinitialize a frame encoder instance
00562 *
00563 * Marks the encoder as uninitialized, preventing further use.
00564 * Verifies deinitialization completed successfully (redundant validation per Rule 5).
00565 *
00566 * @param[out] enc Encoder instance to deinitialize
00567 *
00568 * @return k_rx_ok on successful deinitialization
00569 * @retval k_rx_err_invalid_arg if enc is nullptr
00570 * @retval k_rx_err_validation_failed if deinitialization verification fails
00571 */

```

```

00572 */
00573 rx_err_t rx_frame_encoder_deinit(rx_frame_encoder_t* enc)
00574 {
00575     if (enc == nullptr) {
00576         return k_rx_err_invalid_arg;
00577     }
00578     enc->initialized = k_state_uninitialized;
00580     if (enc->initialized != k_state_uninitialized) {
00581         return k_rx_err_validation_failed;
00582     }
00583     return k_rx_ok;
00584 }
00585
00586 rx_err_t rx_frame_encode(const rx_frame_encoder_t* enc,
00587                         const rx_frame_t* frame,
00588                         uint8_t* output,
00589                         uint32_t* output_len)
00590 {
00591     if (enc == nullptr || frame == nullptr || output == nullptr || output_len == nullptr) {
00592         return k_rx_err_invalid_arg;
00593     }
00594     if (enc->initialized != k_state_initialized) {
00595         return k_rx_err_invalid_state;
00596     }
00597 }
00598
00599 /* Validate payload size */
00600 if (frame->header.length > k_frame_max_payload) {
00601     return k_rx_err_invalid_size;
00602 }
00603
00604 /* Calculate total frame size */
00605 uint32_t frame_size = rx_frame_encoded_size(frame->header.length);
00606 uint32_t offset = k_frame_offset_start;
00607
00608 /* Write SYNC word (little-endian: wire bytes 0xAA, 0x55) */
00609 rx_frame_write_le16(&output[offset], k_frame_sync_word);
00610 offset += k_frame_sync_size;
00611
00612 /* Write SEQ (little-endian) */
00613 rx_frame_write_le16(&output[offset], frame->header.sequence);
00614 offset += k_frame_seq_size;
00615
00616 /* Write LEN (little-endian) */
00617 rx_frame_write_le16(&output[offset], frame->header.length);
00618 offset += k_frame_len_size;
00619
00620 /* Write TYPE (1 byte) */
00621 output[offset] = frame->header.type;
00622 offset += k_frame_type_size;
00623
00624 /* Write FLAGS (1 byte) */
00625 output[offset] = frame->header.flags;
00626 offset += k_frame_flags_size;
00627
00628 /* Write PAYLOAD */
00629 if (frame->header.length > 0) {
00630     memcpy(&output[offset], frame->payload, frame->header.length);
00631     offset += frame->header.length;
00632 }
00633
00634 /* Calculate CRC-32 over SYNC + Header + Payload (IEEE 802.3 polynomial) */
00635 uint32_t crc = rx_crc32_ieee(output, offset);
00636
00637 /* Write CRC-32 (little-endian to match IEEE 802.3 LSB-first order) */
00638 rx_err_t err = internal_write_le32(&output[offset], frame_size - offset, crc);
00639 if (err != k_rx_ok) {
00640     return err;
00641 }
00642 offset += k_frame_crc_size;
00643
00644 *output_len = frame_size;
00645 return k_rx_ok;
00646 }
00647
00648 /*
=====
00649 * Decoder API Implementation
00650 *
=====
00651 */
00652
00653 /**
00654 * @brief Initialize a frame decoder instance
00655 *
00656 * Prepares the decoder for use by setting the initialized flag and verifying

```

```

00657 * the initialization completed successfully (redundant validation per Rule 5).
00658 *
00659 * @param[out] dec Decoder instance to initialize
00660 *
00661 * @return k_rx_ok on successful initialization
00662 * @retval k_rx_err_invalid_arg if dec is nullptr
00663 * @retval k_rx_err_validation_failed if initialization verification fails
00664 */
00665 rx_err_t rx_frame_decoder_init(rx_frame_decoder_t* dec)
00666 {
00667     if (dec == nullptr) {
00668         return k_rx_err_invalid_arg;
00669     }
00670     dec->initialized = k_state_initialized;
00671     if (dec->initialized != k_state_initialized) {
00672         return k_rx_err_validation_failed;
00673     }
00674     return k_rx_ok;
00675 }
00676
00677 /**
00678 * @brief Deinitialize a frame decoder instance
00679 *
00680 * Marks the decoder as uninitialized, preventing further use.
00681 * Verifies deinitialization completed successfully (redundant validation per Rule 5).
00682 *
00683 * @param[out] dec Decoder instance to deinitialize
00684 *
00685 * @return k_rx_ok on successful deinitialization
00686 * @retval k_rx_err_invalid_arg if dec is nullptr
00687 * @retval k_rx_err_validation_failed if deinitialization verification fails
00688 */
00689 rx_err_t rx_frame_decoder_deinit(rx_frame_decoder_t* dec)
00690 {
00691     if (dec == nullptr) {
00692         return k_rx_err_invalid_arg;
00693     }
00694     dec->initialized = k_state_uninitialized;
00695     if (dec->initialized != k_state_uninitialized) {
00696         return k_rx_err_validation_failed;
00697     }
00698     return k_rx_ok;
00699 }
00700
00701 /**
00702 * @brief Decode a frame from raw data
00703 *
00704 * Validates the decoder state, extracts the frame header, copies the payload,
00705 * and verifies the CRC-32 checksum.
00706 *
00707 * @param[in] dec      Initialized frame decoder instance
00708 * @param[in] data      Raw frame data buffer
00709 * @param[in] data_len  Length of data buffer in bytes
00710 * @param[out] frame    Decoded frame (header, payload, CRC)
00711 *
00712 * @retval k_rx_ok Success - frame decoded and CRC verified
00713 * @retval k_rx_err_invalid_arg Any pointer parameter is nullptr or other invalid arguments
00714 * @retval k_rx_err_invalid_state Decoder not initialized
00715 * @retval k_rx_err_crc CRC-32 verification failed
00716 * @retval k_rx_err_invalid_size Payload exceeds maximum frame size
00717 *
00718 * @note This function performs validation at multiple points:
00719 * - Null pointer checks on all parameters
00720 * - Decoder initialization check
00721 * - Header parsing via internal_decode_header()
00722 * - CRC verification via internal_verify_crc()
00723 */
00724 rx_err_t rx_frame_decode(const rx_frame_decoder_t* dec,
00725                          const uint8_t* data,
00726                          const uint32_t data_len,
00727                          rx_frame_t* frame)
00728 {
00729     if (dec == nullptr || data == nullptr || frame == nullptr) {
00730         return k_rx_err_invalid_arg;
00731     }
00732     if (dec->initialized != k_state_initialized) {
00733         return k_rx_err_invalid_state;
00734     }
00735     uint32_t offset;
00736     rx_err_t err = internal_decode_header(data, data_len, frame, &offset);
00737     if (err != k_rx_ok) {
00738         return err;
00739     }
00740 }

```

```

00744
00745 /* Read PAYLOAD */
00746 if (frame->header.length > 0) {
00747     memcpy(frame->payload, &data[offset], frame->header.length);
00748     offset += frame->header.length;
00749 }
00750
00751 err = internal_verify_crc(data, data_len, offset, &frame->crc);
00752 if (err != k_rx_ok) {
00753     return err;
00754 }
00755 return k_rx_ok;
00756 }
00757
00758 /**
00759  * @brief Decode a frame from raw data with bounded sync-word resynchronization
00760  *
00761  * @details
00762  * Provides stream recovery when the byte stream has become misaligned due to
00763  * a dropped or inserted byte. On sync mismatch, scans forward up to
00764  * k_frame_max_scan_bytes positions for the next occurrence of the sync word,
00765  * discards the leading bytes, and reattempts decoding from that position.
00766  *
00767  * This function is safe for repeated calls on a stream: the caller must advance
00768  * its stream read pointer by *bytes_discarded_out on k_rx_ok and also on
00769  * k_rx_err_crc_mismatch (any case where bytes_discarded_out is set) to avoid
00770  * re-processing discarded bytes in subsequent calls.
00771  *
00772  * Recovery algorithm:
00773  * 1. Attempt rx_frame_decode() on data[0..data_len-1].
00774  * 2. If result is not k_rx_err_protocol_error, return immediately.
00775  * 3. Call internal_find_sync_offset() to locate next sync word in data[1..].
00776  * 4. If not found, return k_rx_err_protocol_error with *bytes_discarded_out = 0.
00777  * 5. Decode from data[sync_offset..data_len-1] using the trimmed sub-buffer.
00778  * 6. Write sync_offset to *bytes_discarded_out and return result.
00779  *
00780  * @param[in] dec          Initialized frame decoder handle
00781  * @param[in] data          Raw byte buffer (may be stream-misaligned)
00782  * @param[in] data_len      Length of data buffer in bytes
00783  * @param[out] frame        Decoded frame (header, payload, CRC)
00784  * @param[out] bytes_discarded_out Bytes skipped before sync word (0 = aligned)
00785  *
00786  * @return rx_err_t k_rx_ok on success, or k_rx_err_invalid_arg,
00787  *         k_rx_err_invalid_state, k_rx_err_invalid_size,
00788  *         k_rx_err_protocol_error, or k_rx_err_crc_mismatch on failure
00789  * @retval k_rx_ok          Frame decoded and CRC verified
00790  * @retval k_rx_err_invalid_arg Any pointer parameter is nullptr
00791  * @retval k_rx_err_invalid_state Decoder not initialized
00792  * @retval k_rx_err_invalid_size Buffer too short to contain a valid frame
00793  * @retval k_rx_err_protocol_error No sync word found within scan window
00794  * @retval k_rx_err_crc_mismatch Sync found but CRC verification failed
00795  *
00796  * @pre dec must be initialized via rx_frame_decoder_init()
00797  * @pre data must point to a buffer of at least data_len bytes
00798  * @pre frame must point to a valid rx_frame_t storage location (not NULL)
00799  * @pre bytes_discarded_out must point to a valid uint32_t storage location (not NULL)
00800  * @post On k_rx_ok, *bytes_discarded_out == bytes skipped to reach sync word
00801  * @post On k_rx_err_crc_mismatch, *bytes_discarded_out == bytes skipped; caller must still advance stream pointer
00802  *
00803  * @note Parameter order follows the canonical convention: decoder input -> data input ->
00804  *         frame output -> diagnostic output (matching rx_frame_decode()).
00805  * @note Not thread-safe. Caller must synchronize access.
00806  * @note The scan is bounded at k_frame_max_scan_bytes (NASA Rule 2).
00807  * @note Log *bytes_discarded_out > 0 as a stream-health diagnostic warning.
00808  *
00809  * @warning Caller must advance stream read pointer by *bytes_discarded_out after any call
00810  *         that returns k_rx_ok or k_rx_err_crc_mismatch to avoid infinite re-scan;
00811  *         *bytes_discarded_out is unmodified only when k_rx_err_invalid_arg is returned.
00812  *
00813  * @par Example:
00814  * @code{.c}
00815  * uint32_t discarded = k_bytes_discarded_none;
00816  * rx_err_t err = rx_frame_decode_with_resync(dec, data, data_len, &frame, &discarded);
00817  * if (err == k_rx_ok) {
00818  *     // Advance stream read pointer past any discarded bytes
00819  *     stream_read_ptr += discarded;
00820  * } else if (err == k_rx_err_crc_mismatch) {
00821  *     // Sync was found but CRC failed -- still advance to avoid infinite re-scan
00822  *     stream_read_ptr += discarded;
00823  *     rx_log_error(TAG, "sync found but CRC failed");
00824  * } else {
00825  *     rx_log_error(TAG, "no sync word within scan window");
00826  * }
00827  * }
00828  * @endcode
00829  *
00830  * @see rx_frame_decode() Simple decode without stream recovery

```

```

00831 * @see internal_find_sync_offset() Bounded sync word scanner
00832 * @see k_frame_max_scan_bytes Scan window upper bound
00833 *
00834 * @since Version 1.0.0
00835 */
00836 rx_err_t rx_frame_decode_with_resync(const rx_frame_decoder_t* dec,
00837                                     const uint8_t* data,
00838                                     const uint32_t data_len,
00839                                     rx_frame_t* frame,
00840                                     uint32_t* bytes_discarded_out)
00841 {
00842     if (dec == nullptr || data == nullptr || frame == nullptr || bytes_discarded_out == nullptr) {
00843         return k_rx_err_invalid_arg;
00844     }
00845
00846     *bytes_discarded_out = k_bytes_discarded_none;
00847
00848     /* Fast path: attempt aligned decode first */
00849     rx_err_t err = rx_frame_decode(dec, data, data_len, frame);
00850     if (err != k_rx_err_protocol_error) {
00851         /* Either success, or an error other than sync mismatch (CRC, size, etc.) */
00852         return err;
00853     }
00854
00855     /* Sync word not at offset 0: scan forward for next sync word */
00856     uint32_t sync_offset = (uint32_t)k_frame_offset_start;
00857     err = internal_find_sync_offset(data, data_len, &sync_offset);
00858     if (err != k_rx_ok) {
00859         /* No sync word found within bounded window */
00860         return k_rx_err_protocol_error;
00861     }
00862
00863     /* Report discarded bytes now: caller needs this to advance past the sync
00864      * even if the subsequent decode fails (e.g. CRC mismatch at new offset) */
00865     *bytes_discarded_out = sync_offset;
00866
00867     /* Reattempt decode from the discovered sync offset */
00868     err = rx_frame_decode(dec, &data[sync_offset], data_len - sync_offset, frame);
00869
00870     return err;
00871 }
00872
00873 /*
=====
00874 * Utility Functions
00875 *
=====
00876 */
00877 /**
00878 * @brief Create an ACK (acknowledgment) frame
00879 *
00880 *
00881 * Constructs a frame with type=ACK, empty payload, and the specified sequence number.
00882 * The ACK response frames are used by the receiver to confirm successful reception
00883 * of a command or data frame from the sender.
00884 *
00885 * @param[out] frame Frame structure to populate with ACK
00886 * @param[in] sequence Sequence number of the frame being acknowledged
00887 *
00888 * @return k_rx_ok on successful frame creation
00889 * @retval k_rx_err_invalid_arg if frame is nullptr
00890 * @retval k_rx_err_validation_failed if ACK type verification fails
00891 */
00892 rx_err_t rx_frame_create_ack(rx_frame_t* frame, const uint16_t sequence)
00893 {
00894     if (frame == nullptr) {
00895         return k_rx_err_invalid_arg;
00896     }
00897
00898     *frame = (rx_frame_t){0};
00899     frame->header.sequence = sequence;
00900     frame->header.length = 0;
00901     frame->header.type = k_frame_type_ack;
00902     frame->header.flags = k_frame_flag_none;
00903
00904     if (frame->header.type != k_frame_type_ack) {
00905         return k_rx_err_validation_failed;
00906     }
00907
00908     return k_rx_ok;
00909 }
00910
00911 /**
00912 * @brief Create a NACK (negative acknowledgment) frame
00913 *
00914 * Constructs a frame with type=NACK, empty payload, the specified sequence number,
00915 * and error/status flags. NACK frames are sent by the receiver to indicate that a

```

```

00916 * frame was not processed successfully due to an error condition (specified in flags).
00917 *
00918 * @param[out] frame    Frame structure to populate with NACK
00919 * @param[in]  sequence Sequence number of the frame being negative-acknowledged
00920 * @param[in]  flags    Status/error flags indicating reason for NACK
00921 *
00922 * @return k_rx_ok on successful frame creation
00923 * @retval k_rx_err_invalid_arg if frame is nullptr
00924 * @retval k_rx_err_validation_failed if NACK type verification fails
00925 */
00926 rx_err_t rx_frame_create_nack(rx_frame_t* frame, const uint16_t sequence, uint8_t flags)
00927 {
00928     if (frame == nullptr) {
00929         return k_rx_err_invalid_arg;
00930     }
00931
00932     *frame = (rx_frame_t){0};
00933     frame->header.sequence = sequence;
00934     frame->header.length   = 0;
00935     frame->header.type     = k_frame_type_nack;
00936     frame->header.flags    = flags;
00937
00938     if (frame->header.type != k_frame_type_nack) {
00939         return k_rx_err_validation_failed;
00940     }
00941
00942     return k_rx_ok;
00943 }
00944
00945 /**
00946 * @brief Create a PING keepalive frame
00947 *
00948 * @details
00949 * Initializes frame as a PING request for connection liveness checks.
00950 * Receiver should respond with a PONG frame echoing the payload.
00951 * Payload is optional; a 4-byte LE counter is typical.
00952 *
00953 * @param[out] frame    Frame to populate (zeroed then filled)
00954 * @param[in]  sequence Sequence number for this frame
00955 * @param[in]  payload   Optional payload (NULL when payload_len is 0)
00956 * @param[in]  payload_len Payload length in bytes [0, k_frame_max_payload]
00957 *
00958 * @retval k_rx_ok          Frame created successfully
00959 * @retval k_rx_err_invalid_arg frame is NULL, or payload NULL with len > 0
00960 * @retval k_rx_err_invalid_size payload_len exceeds k_frame_max_payload
00961 * @retval k_rx_err_validation_failed Post-condition type check failed
00962 *
00963 * @pre frame != nullptr
00964 * @pre payload != NULL when payload_len > 0
00965 * @post frame->header.type == k_frame_type_ping
00966 * @post frame->header.length == payload_len
00967 *
00968 * @note Stateless; safe to call from any context.
00969 * @see rx_frame_create_pong() Corresponding response creator
00970 * @since Version 1.0.0
00971 */
00972 rx_err_t rx_frame_create_ping(rx_frame_t* frame,
00973                               const uint16_t sequence,
00974                               const uint8_t* payload,
00975                               const uint32_t payload_len)
00976 {
00977     if (frame == nullptr) {
00978         return k_rx_err_invalid_arg;
00979     }
00980
00981     if (payload_len > 0 && payload == nullptr) {
00982         return k_rx_err_invalid_arg;
00983     }
00984
00985     if (payload_len > k_frame_max_payload) {
00986         return k_rx_err_invalid_size;
00987     }
00988
00989     *frame = (rx_frame_t){0};
00990     frame->header.sequence = sequence;
00991     frame->header.length   = (uint16_t)payload_len;
00992     frame->header.type     = k_frame_type_ping;
00993     frame->header.flags    = k_frame_flag_none;
00994
00995     if (payload_len > 0) {
00996         memcpy(frame->payload, payload, payload_len);
00997     }
00998
00999     if (frame->header.type != k_frame_type_ping) {
01000         return k_rx_err_validation_failed;
01001     }
01002

```

```

01003     return k_rx_ok;
01004 }
01005
01006 /**
01007  * @brief Create a PONG response frame
01008  *
01009  * @details
01010  * Initializes frame as a PONG response to a received PING.
01011  * Echoes the PING payload so the sender can validate round-trip data.
01012  *
01013  * @param[out] frame    Frame to populate (zeroed then filled)
01014  * @param[in]  sequence  Sequence number for this frame
01015  * @param[in]  payload    Payload to echo (NULL when payload_len is 0)
01016  * @param[in]  payload_len Payload length in bytes [0, k_frame_max_payload]
01017  *
01018  * @retval k_rx_ok      Frame created successfully
01019  * @retval k_rx_err_invalid_arg  frame is NULL, or payload NULL with len > 0
01020  * @retval k_rx_err_invalid_size  payload_len exceeds k_frame_max_payload
01021  * @retval k_rx_err_validation_failed Post-condition type check failed
01022  *
01023  * @pre frame != nullptr
01024  * @pre payload != NULL when payload_len > 0
01025  * @post frame->header.type == k_frame_type_pong
01026  * @post frame->header.length == payload_len
01027  *
01028  * @note Stateless; safe to call from any context.
01029  * @see rx_frame_create_ping() Corresponding request creator
01030  * @since Version 1.0.0
01031  */
01032 rx_err_t rx_frame_create_pong(rx_frame_t* frame,
01033                               const uint16_t sequence,
01034                               const uint8_t* payload,
01035                               const uint32_t payload_len)
01036 {
01037     if (frame == nullptr) {
01038         return k_rx_err_invalid_arg;
01039     }
01040
01041     if (payload_len > 0 && payload == nullptr) {
01042         return k_rx_err_invalid_arg;
01043     }
01044
01045     if (payload_len > k_frame_max_payload) {
01046         return k_rx_err_invalid_size;
01047     }
01048
01049     *frame = (rx_frame_t){0};
01050     frame->header.sequence = sequence;
01051     frame->header.length = (uint16_t)payload_len;
01052     frame->header.type = k_frame_type_pong;
01053     frame->header.flags = k_frame_flag_none;
01054
01055     if (payload_len > 0) {
01056         memcpy(frame->payload, payload, payload_len);
01057     }
01058
01059     if (frame->header.type != k_frame_type_pong) {
01060         return k_rx_err_validation_failed;
01061     }
01062
01063     return k_rx_ok;
01064 }
01065
01066 /**
01067  * @brief Create a RESET request frame
01068  *
01069  * @details
01070  * Initializes frame as a RESET request that asks the peer to reset
01071  * communication state (sequence numbers, buffers). The receiver
01072  * should respond with a RESET_ACK frame. Payload is always empty.
01073  *
01074  * @param[out] frame    Frame to populate (zeroed then filled)
01075  * @param[in]  sequence Sequence number for this frame
01076  *
01077  * @retval k_rx_ok      Frame created successfully
01078  * @retval k_rx_err_invalid_arg  frame is nullptr
01079  * @retval k_rx_err_validation_failed Post-condition type check failed
01080  *
01081  * @pre frame != nullptr
01082  * @pre Caller has determined that a reset is necessary
01083  * @post frame->header.type == k_frame_type_reset
01084  * @post frame->header.length == 0
01085  *
01086  * @note Stateless; safe to call from any context.
01087  * @warning Reset clears all sequence number state on both sides.
01088  * @see rx_frame_create_reset_ack() Corresponding acknowledgment creator
01089  * @since Version 1.0.0

```

```

01090 */
01091 rx_err_t rx_frame_create_reset(rx_frame_t* frame, const uint16_t sequence)
01092 {
01093     if (frame == nullptr) {
01094         return k_rx_err_invalid_arg;
01095     }
01096     *frame = (rx_frame_t){0};
01097     frame->header.sequence = sequence;
01098     frame->header.length = 0;
01099     frame->header.type = k_frame_type_reset;
01100     frame->header.flags = k_frame_flag_none;
01101     if (frame->header.type != k_frame_type_reset) {
01102         return k_rx_err_validation_failed;
01103     }
01104     return k_rx_ok;
01105 }
01106
01107 /**
01108  * @brief Create a RESET_ACK confirmation frame
01109  * @details
01110  * @param[out] frame Frame to populate (zeroed then filled)
01111  * @param[in] sequence Sequence number (should match received RESET)
01112  * @retval k_rx_ok Frame created successfully
01113  * @retval k_rx_err_invalid_arg frame is nullptr
01114  * @retval k_rx_err_validation_failed Post-condition type check failed
01115  * @pre frame != nullptr
01116  * @pre A RESET frame was received and processed
01117  * @post frame->header.type == k_frame_type_reset_ack
01118  * @post frame->header.length == 0
01119  * @note Stateless; safe to call from any context.
01120  * @see rx_frame_create_reset() Corresponding request creator
01121  * @since Version 1.0.0
01122  */
01123 rx_err_t rx_frame_create_reset_ack(rx_frame_t* frame, const uint16_t sequence)
01124 {
01125     if (frame == nullptr) {
01126         return k_rx_err_invalid_arg;
01127     }
01128     *frame = (rx_frame_t){0};
01129     frame->header.sequence = sequence;
01130     frame->header.length = 0;
01131     frame->header.type = k_frame_type_reset_ack;
01132     frame->header.flags = k_frame_flag_none;
01133     if (frame->header.type != k_frame_type_reset_ack) {
01134         return k_rx_err_validation_failed;
01135     }
01136     return k_rx_ok;
01137 }

```